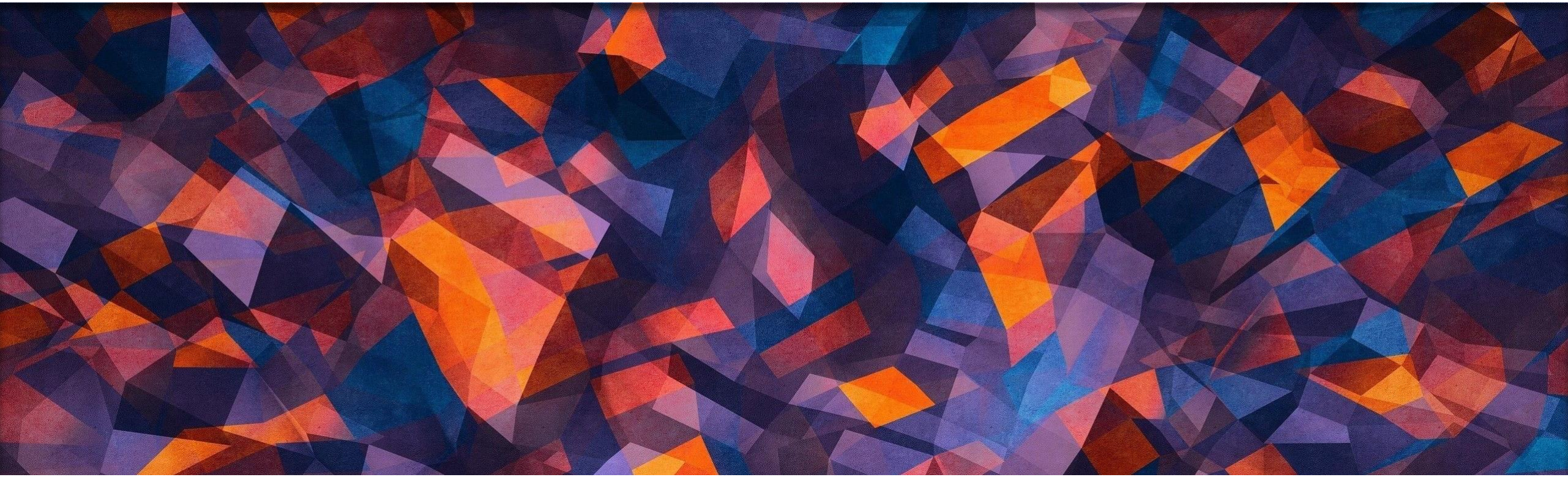
Universität St.Gallen



Event-driven and Process-oriented Architectures

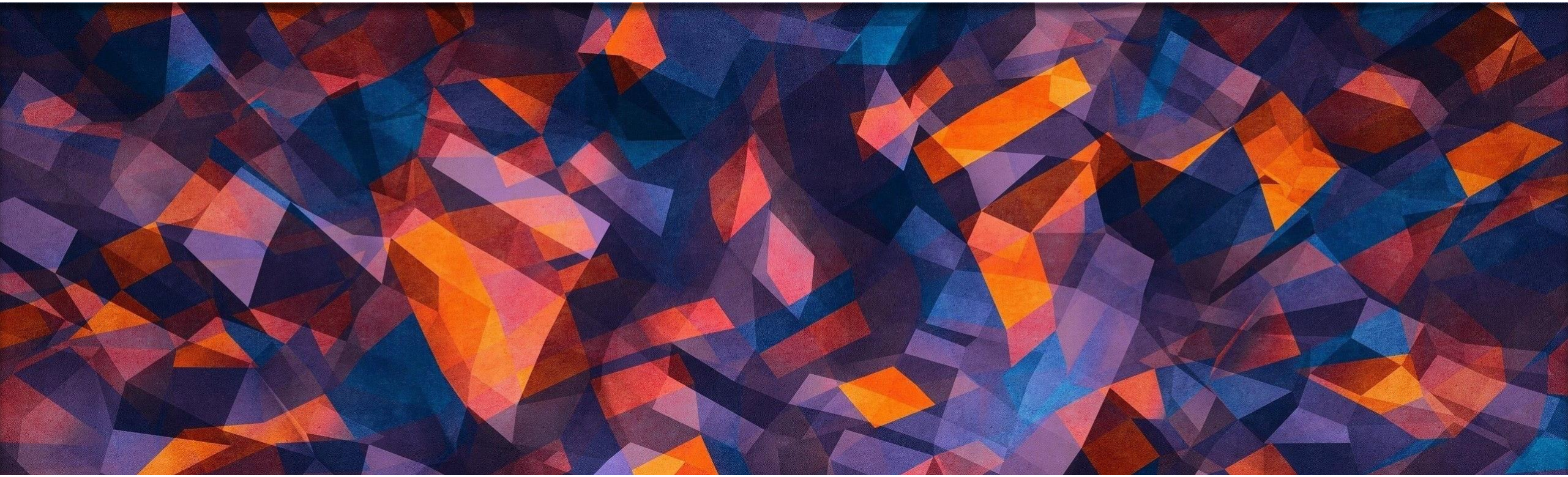
Event-driven Architecture

Lecture 2

Agenda

- Motivating Example: Flowing Retail Example (Part 1)
- Event-driven Architecture
  - Four Common Patterns
    - Event Notification
    - Event-carried State Transfer
    - Event Sourcing
    - Command Query Responsibility Segregation (CQRS)
- Motivating Example: Flowing Retail Example (Part 2)
- Coordination: Choreography

Universität St.Gallen

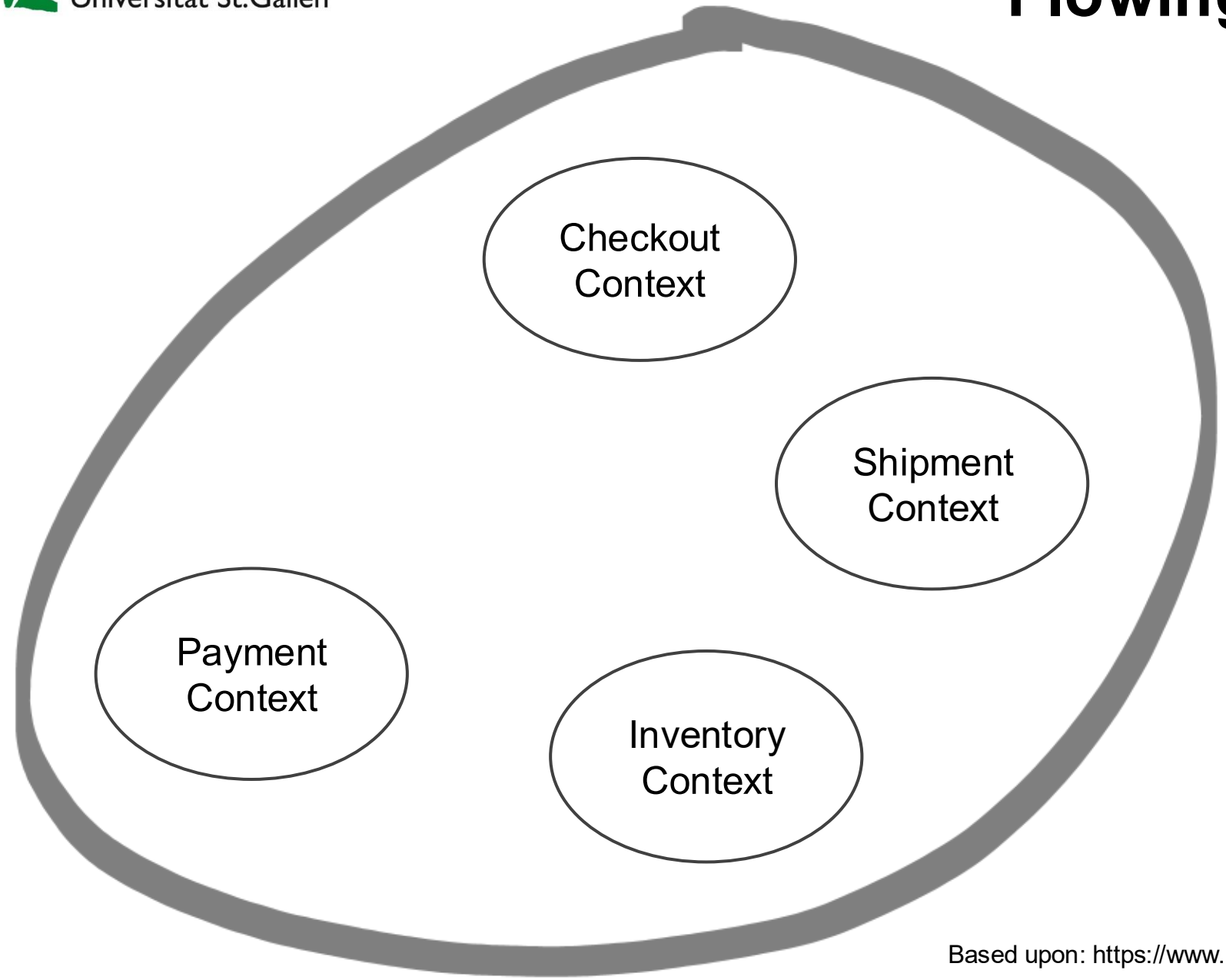


Event-driven and Process-oriented Architectures

Motivating Example (Part 1)

Lecture 2

Flowing Retail Example

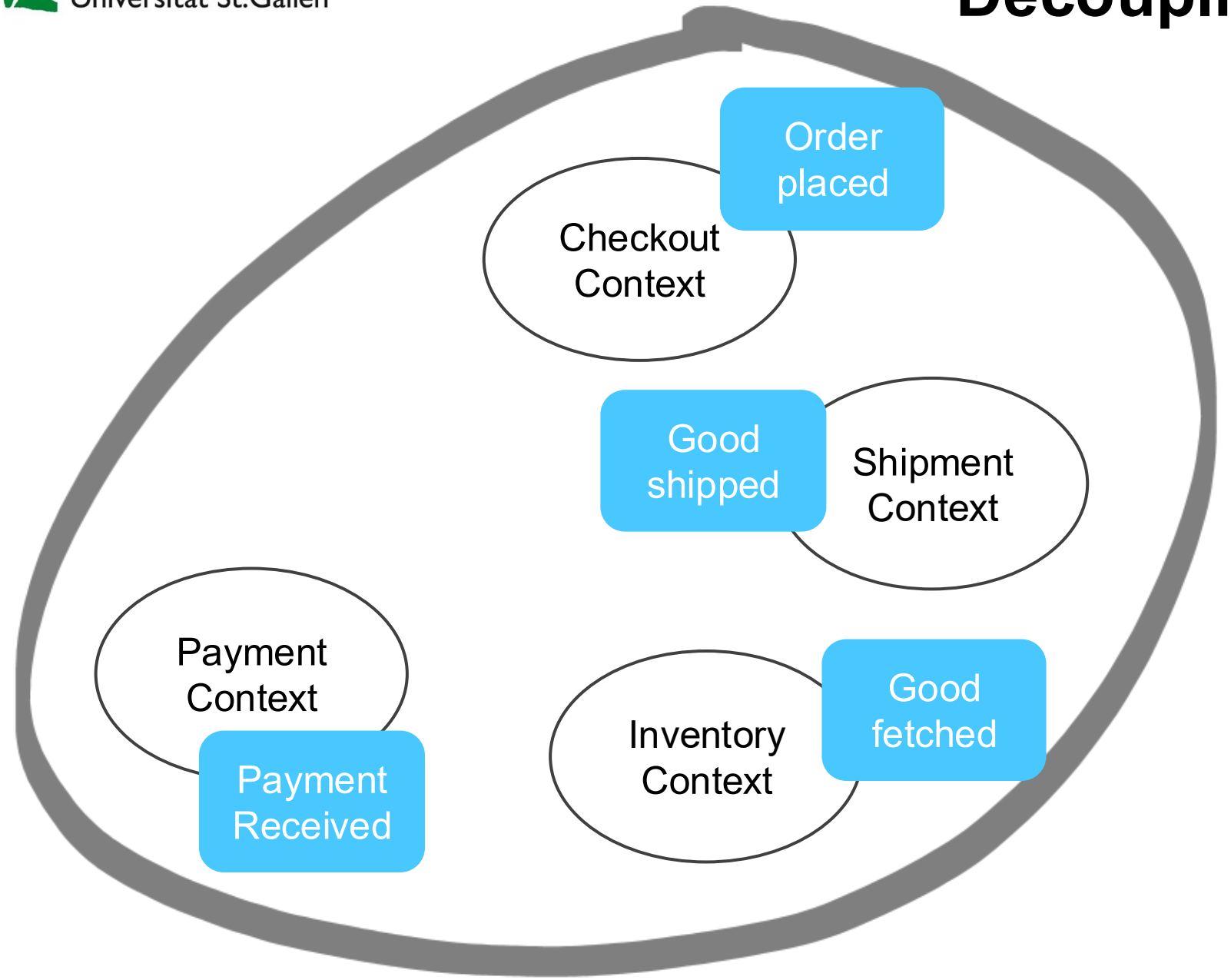


Four bounded contexts resulting into four microservices

Up to now all should sound familiar to you from ASSE.

Based upon: https://www.infoq.com/articles/events-workflow-automation/

Decoupling using Events



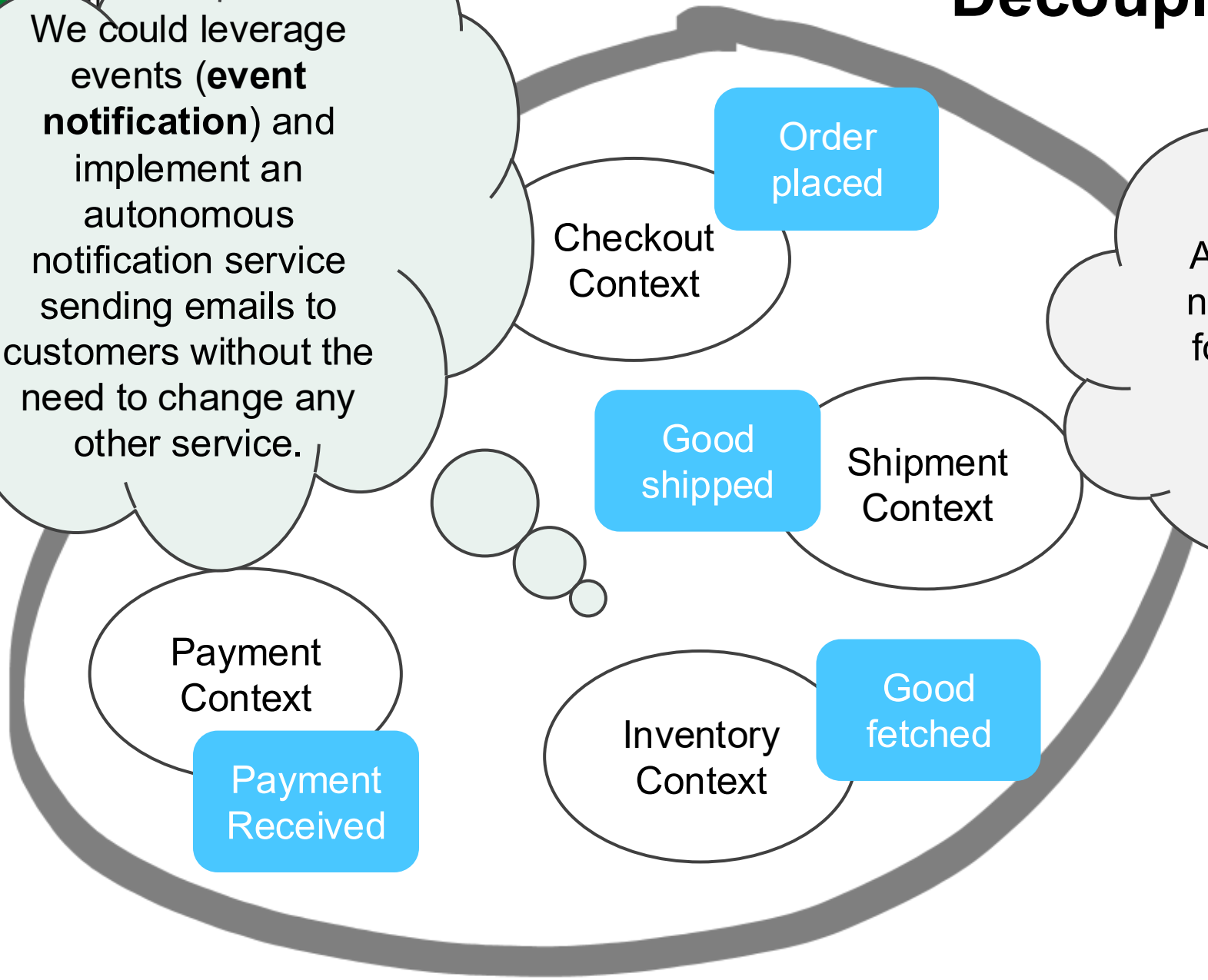
- Services emit events to inform about important state changes

Up to now all should sound familiar to you from ASSE.

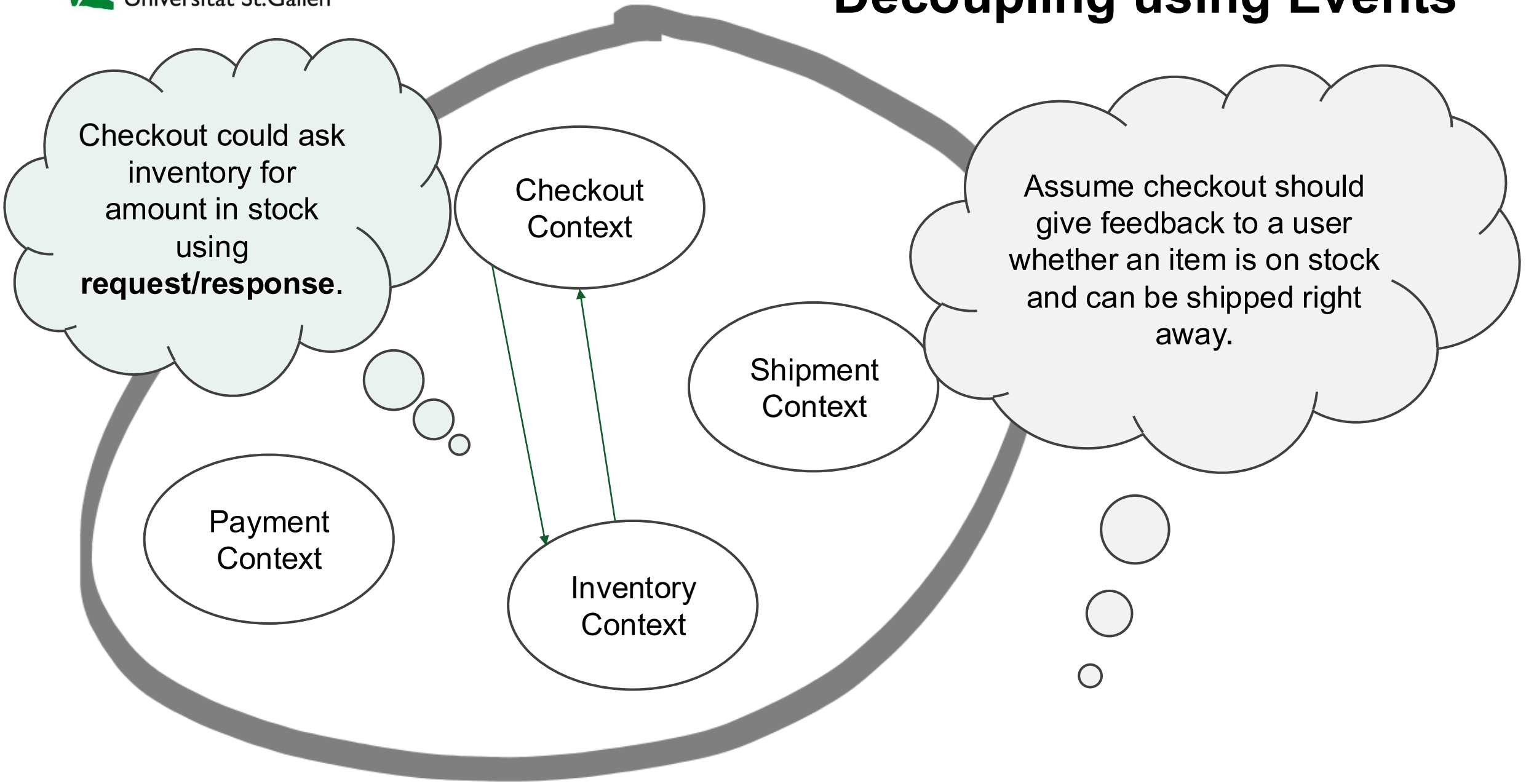
Decoupling using Events

We could leverage events (**event notification**) and implement an autonomous notification service sending emails to customers without the need to change any other service.

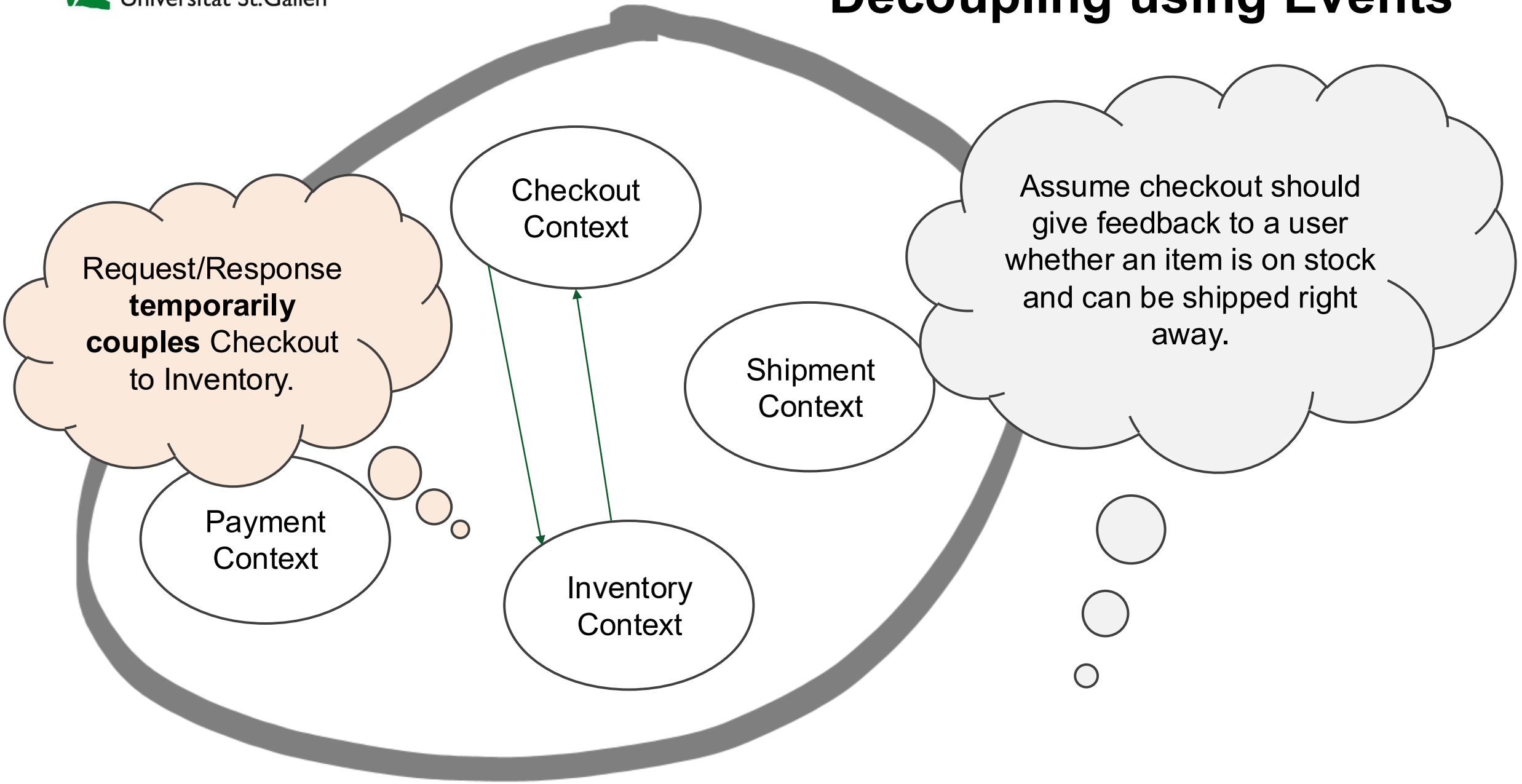
Assume we want to send notifications to customers for important steps in the order fullfillment.



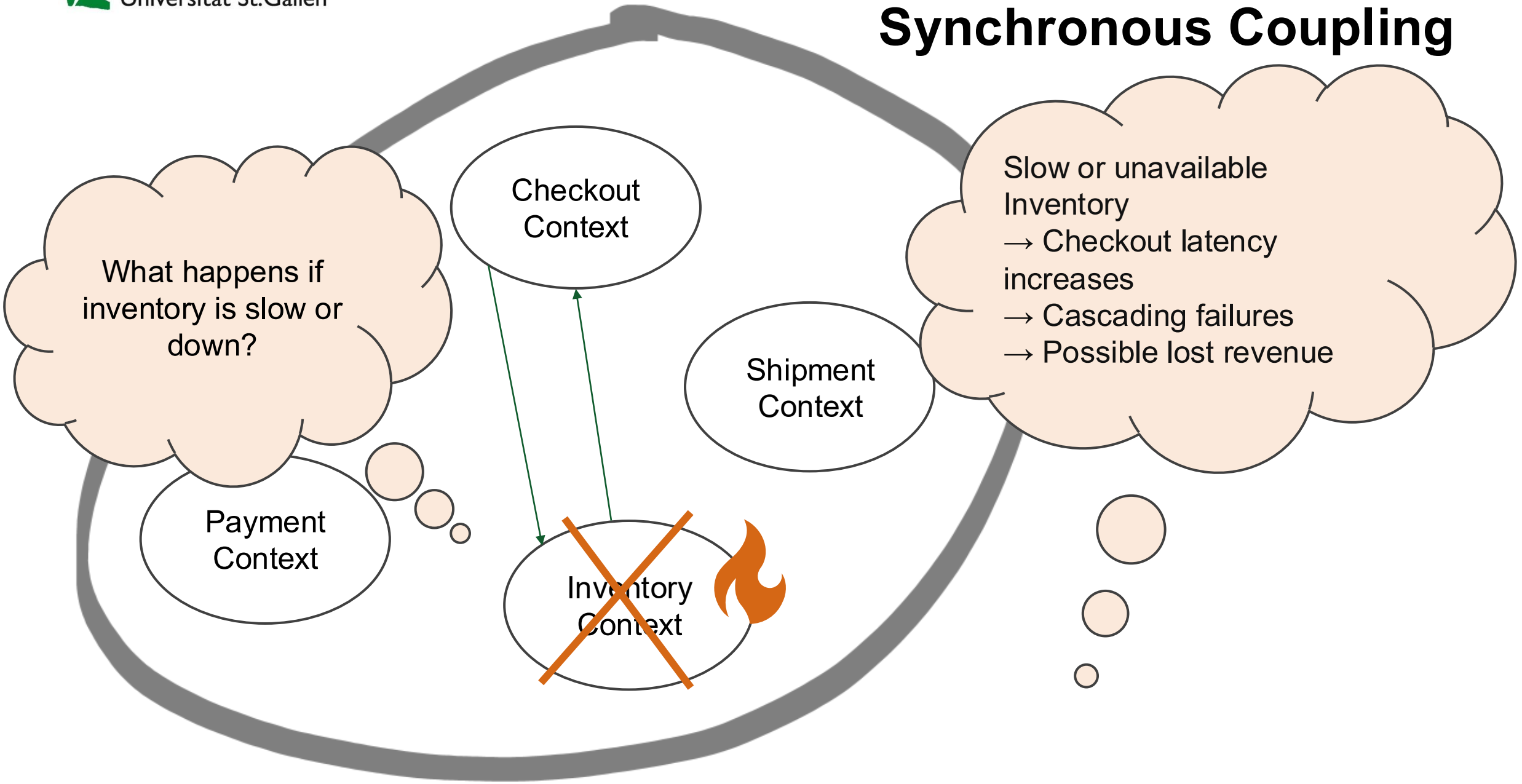
Decoupling using Events



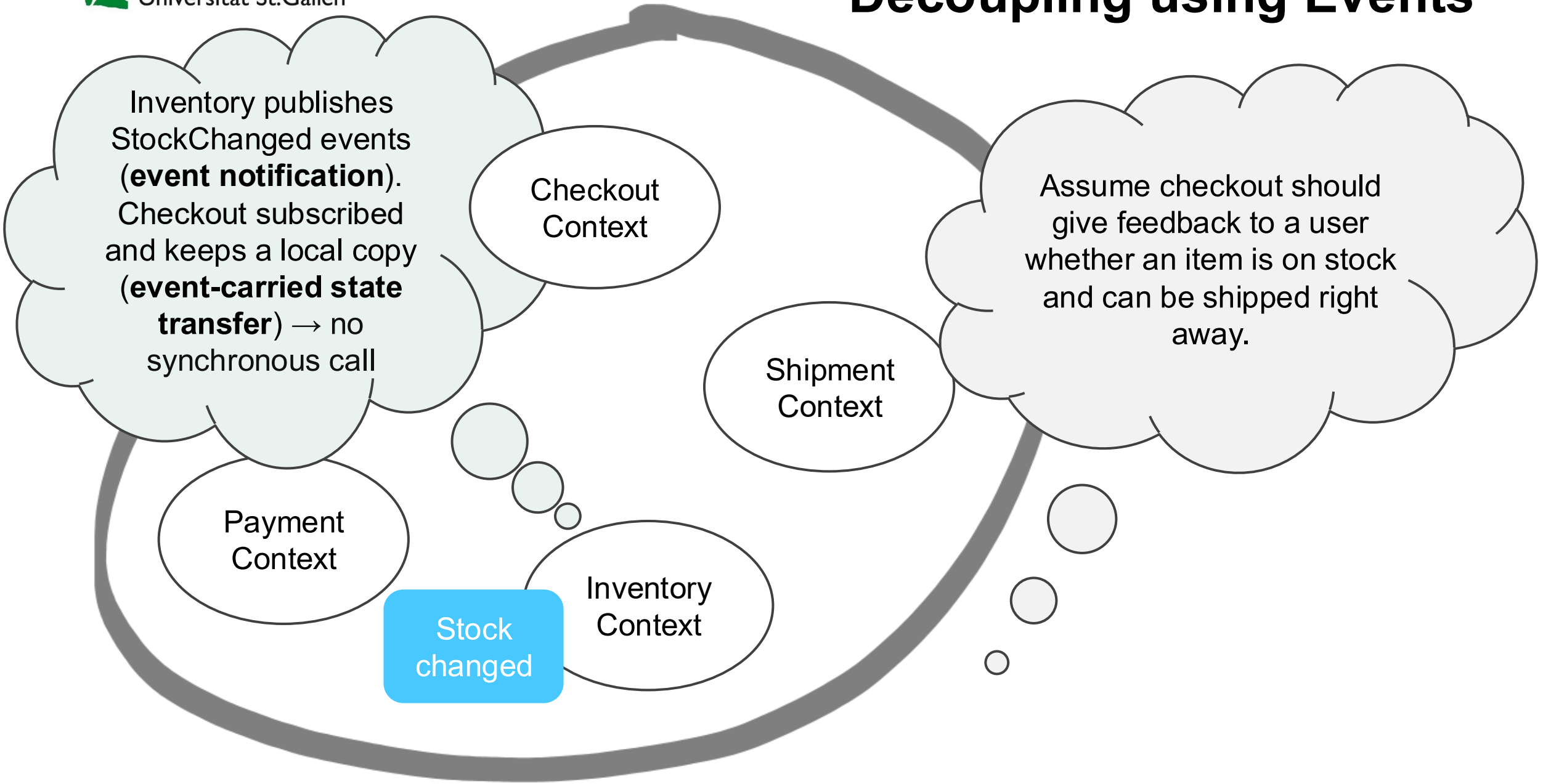
Decoupling using Events



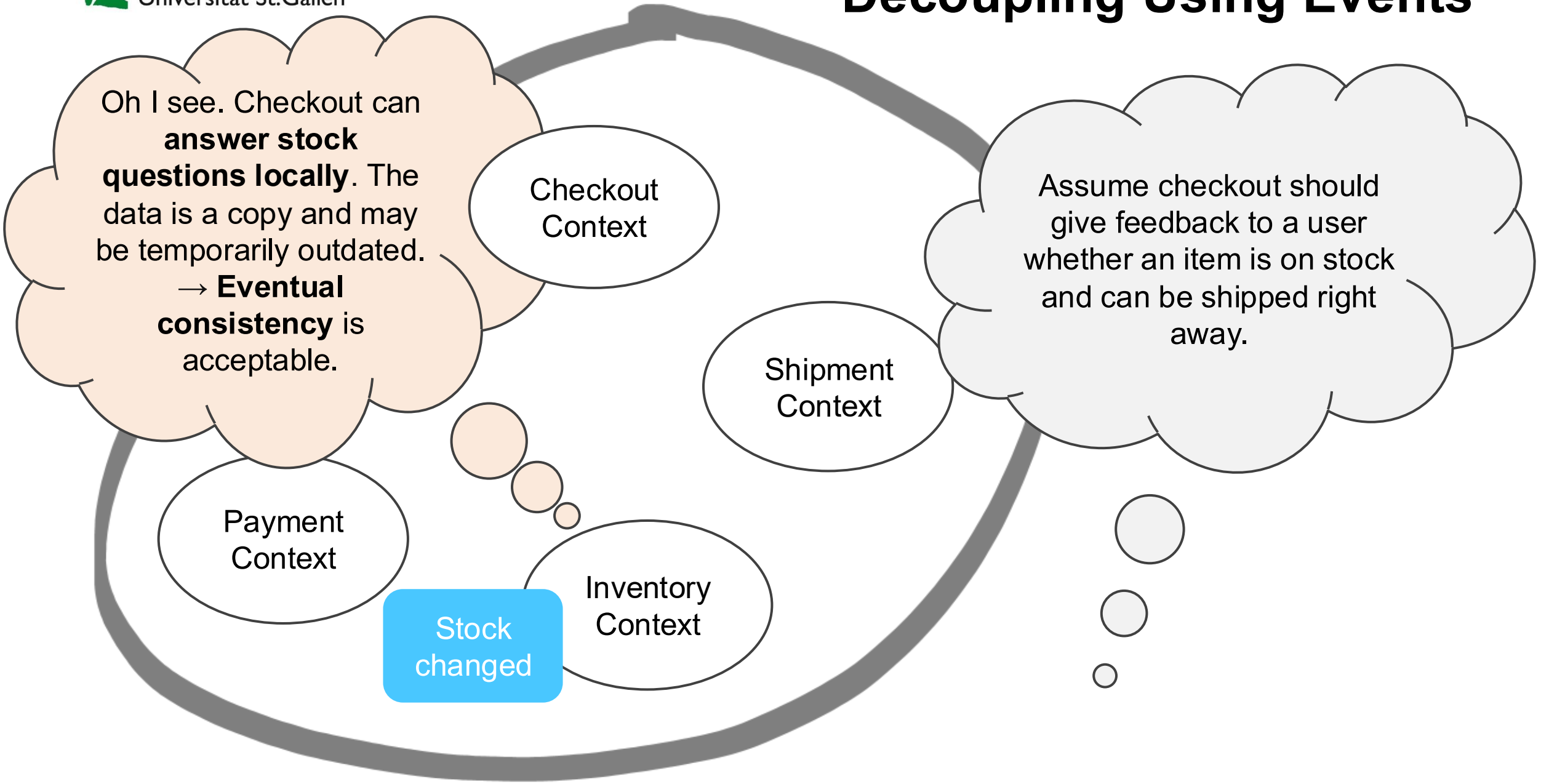
The Danger of Synchronous Coupling



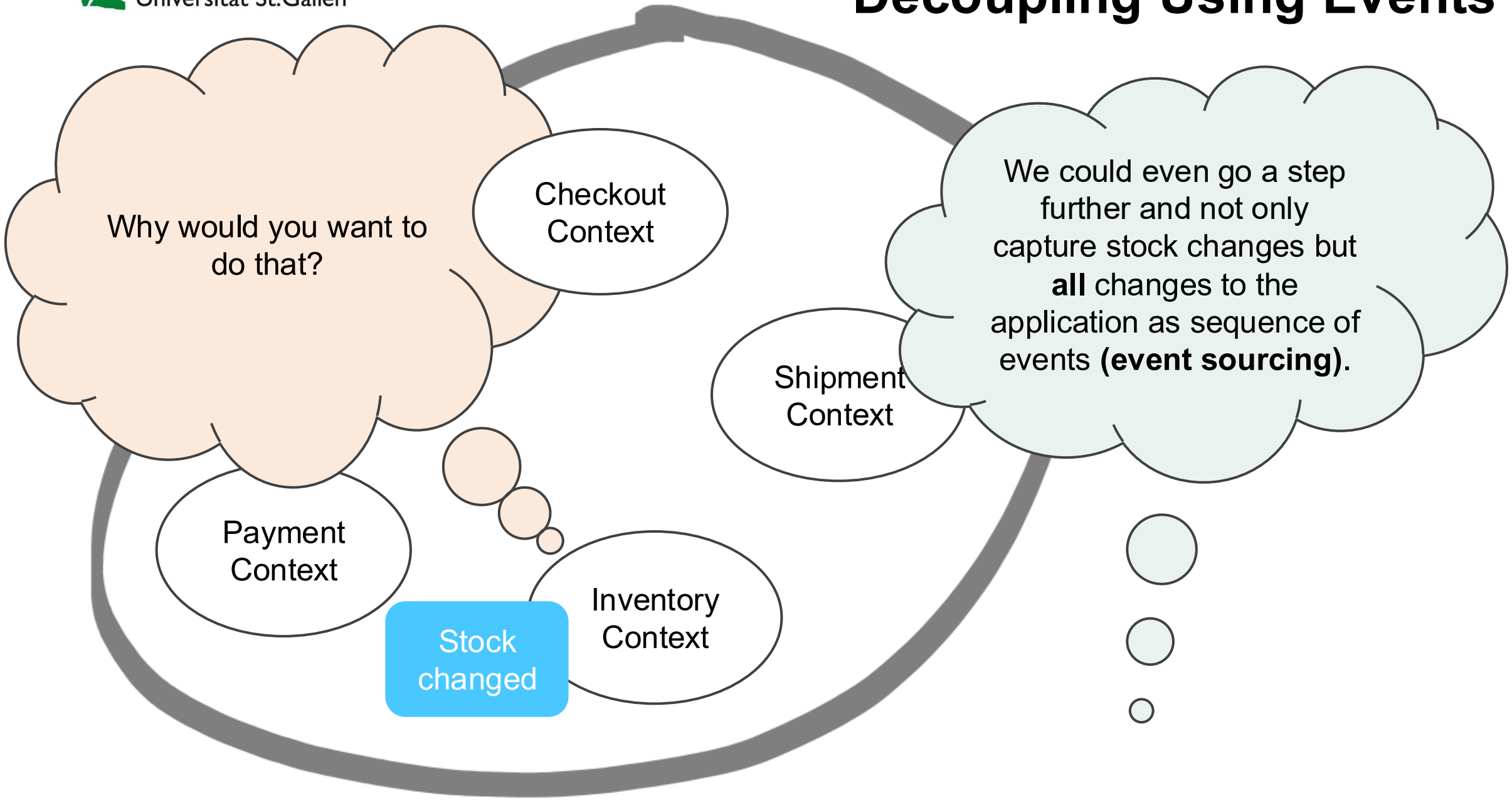
Decoupling using Events



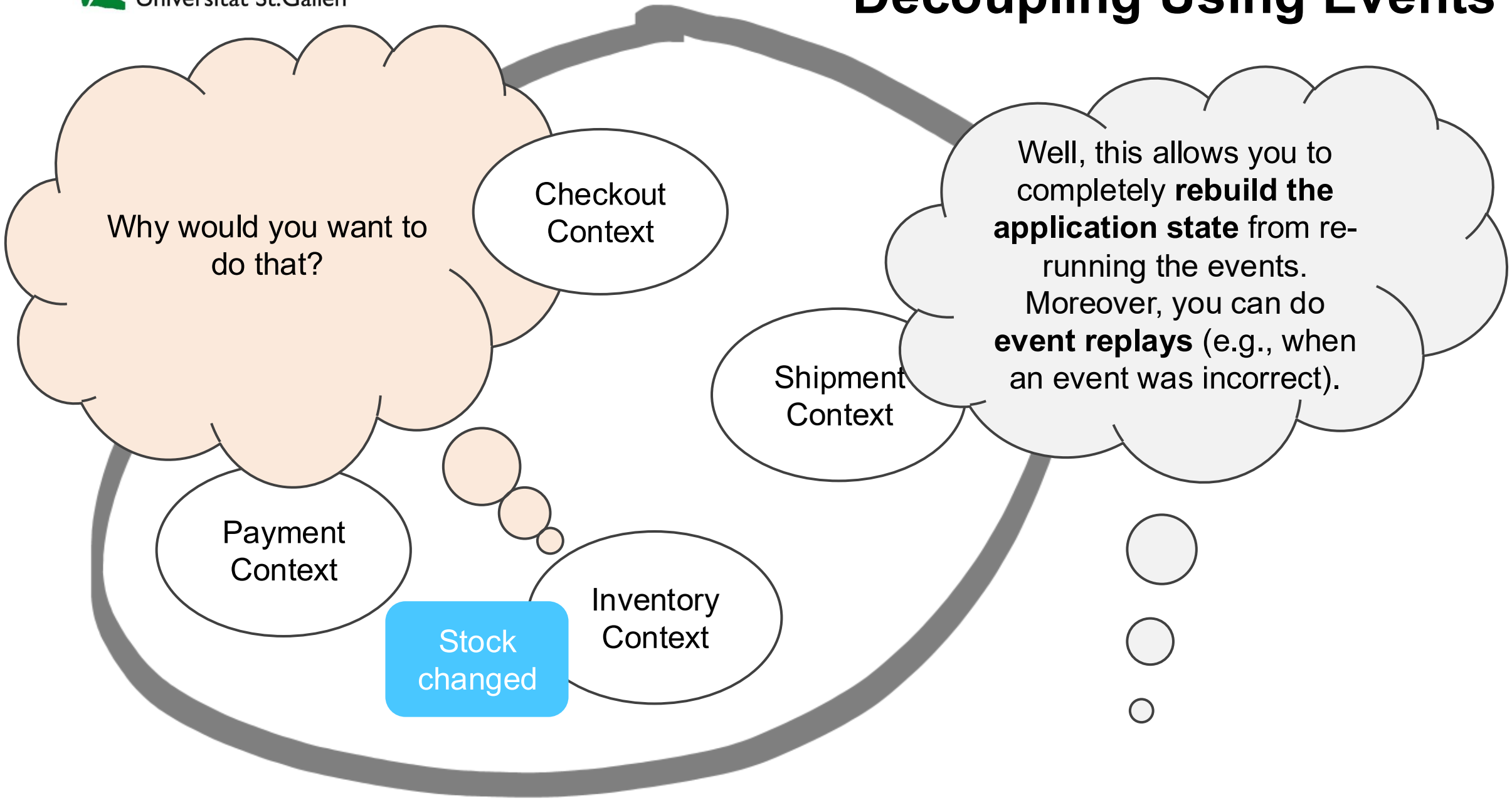
Decoupling Using Events



Decoupling Using Events



Decoupling Using Events

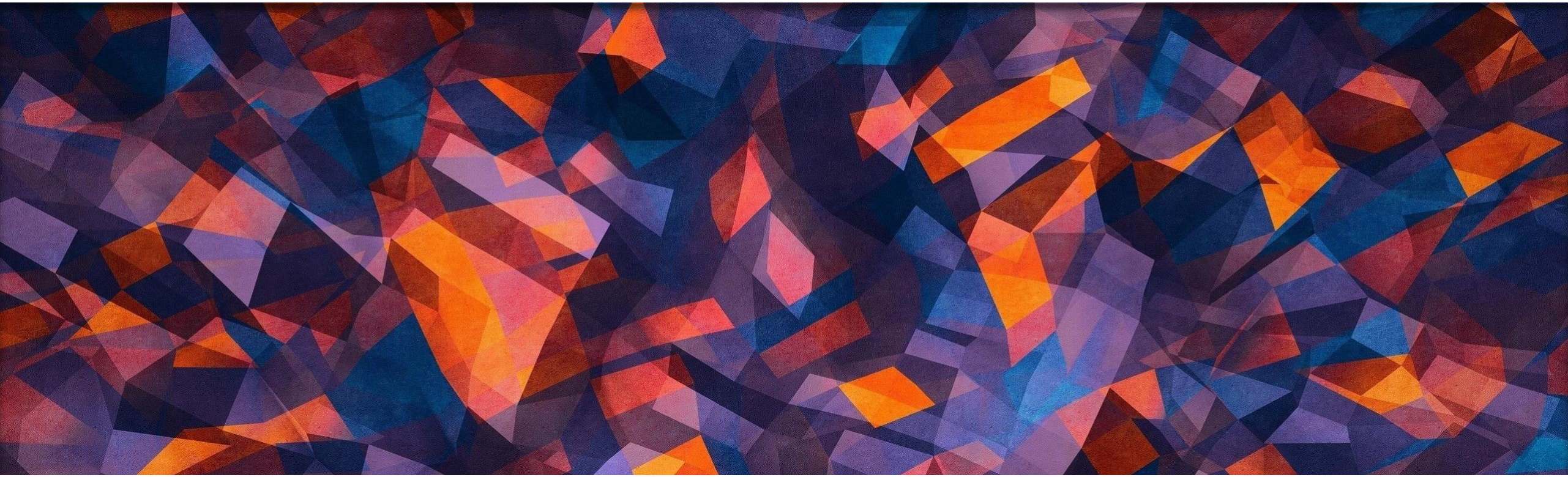


Trade-off: Coupling vs Consistency

- Synchronous request/response
  - Strong consistency
  - Tight coupling
  - Cascading failures
- Event-driven
  - Loose coupling
  - Resilient
  - Eventual consistency

Architecture = choosing the right trade-off.

Universität St.Gallen



Event-driven and Process-oriented Architectures

Event-driven Architecture (EDA) – Four Common Patterns

Lecture 2

Event-driven Architecture

Event–driven Architecture (EDA). An architectural paradigm promoting the production, consumption of, and reaction to events.

Systems communicate by announcing facts about what has happened in the past, instead of calling each other to request work.

- Event-driven architectures have become increasingly popular
- Drivers for using an event-driven architecture
  - Desire for team autonomy
  - Build decoupled systems

Source: The Many Meanings of Event-driven Architecture by Martin Fowler

The Many Meanings of Event-driven Architecture

Event–driven Architecture (EDA). An architectural paradigm promoting the production, consumption of, and reaction to events.

- Four Patterns
  - Event notification
  - Event-carried State Transfer
  - Event sourcing
  - Command Query Responsibility Segregation (CQRS)
- These patterns form a spectrum — from simple notification to full event-sourced systems.

Source: The Many Meanings of Event-driven Architecture by Martin Fowler

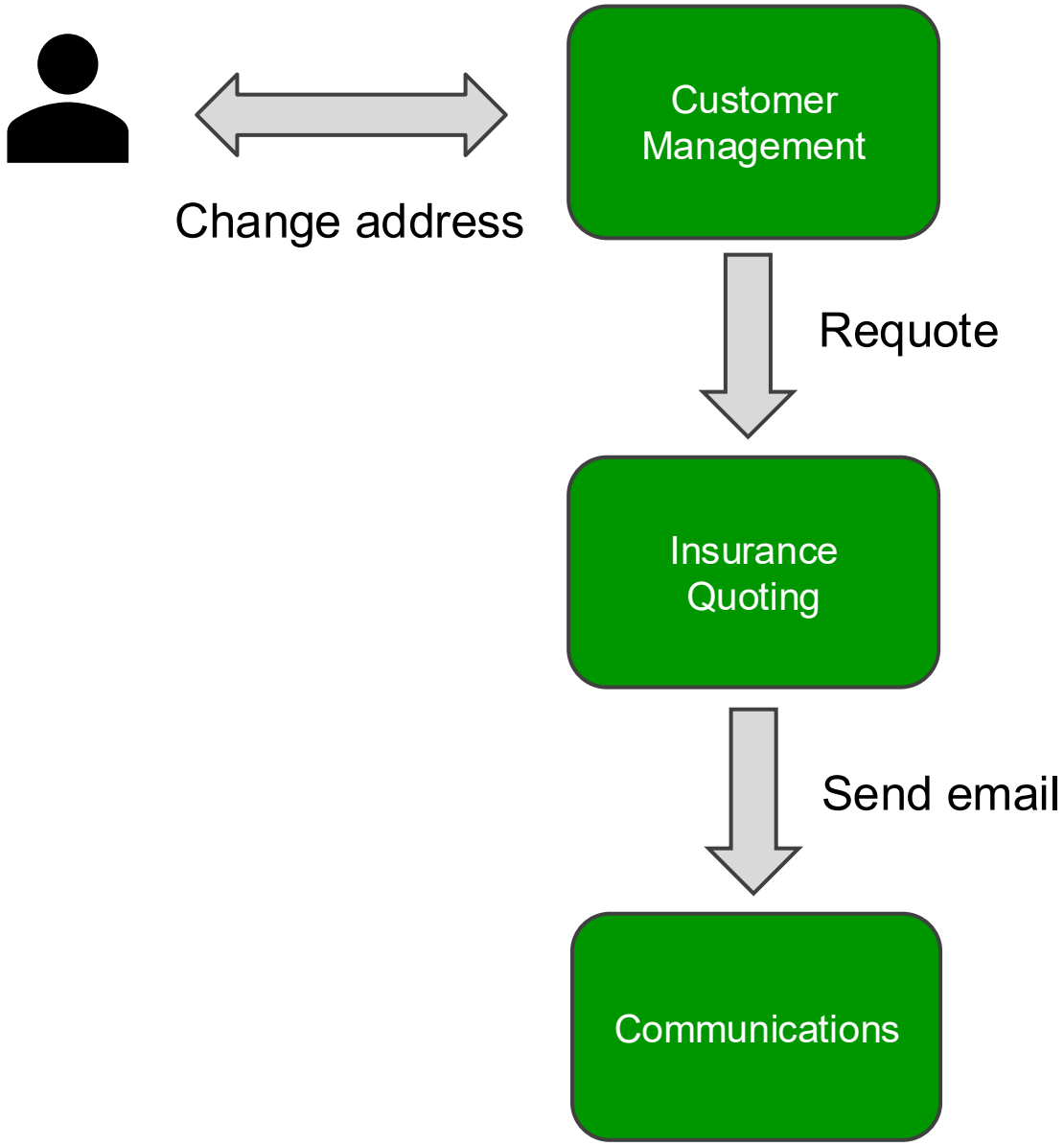
The Many Meanings of Event-driven Architecture

Event–driven Architecture (EDA). An architectural paradigm promoting the production, consumption of, and reaction to events.

- Four Patterns
  - **Event notification**
  - Event-carried State Transfer
  - Event sourcing
  - Command Query Responsibility Segregation (CQRS)

Source: The Many Meanings of Event-driven Architecture by Martin Fowler

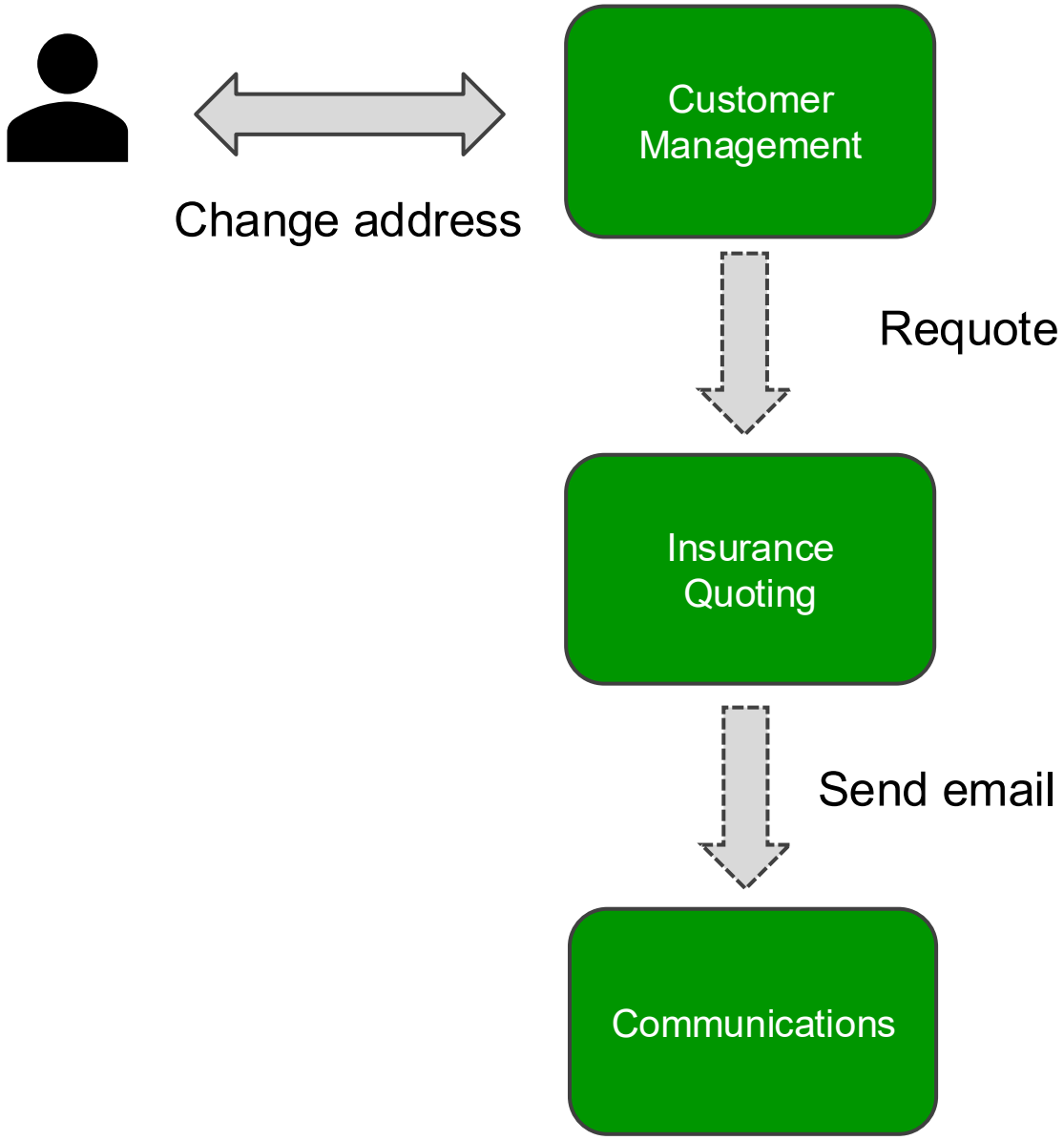
A Fictionary System at an Insurance Company



- Assume change of address affects insurance

Source: The Many Meanings of Event-driven Architecture by Martin Fowler

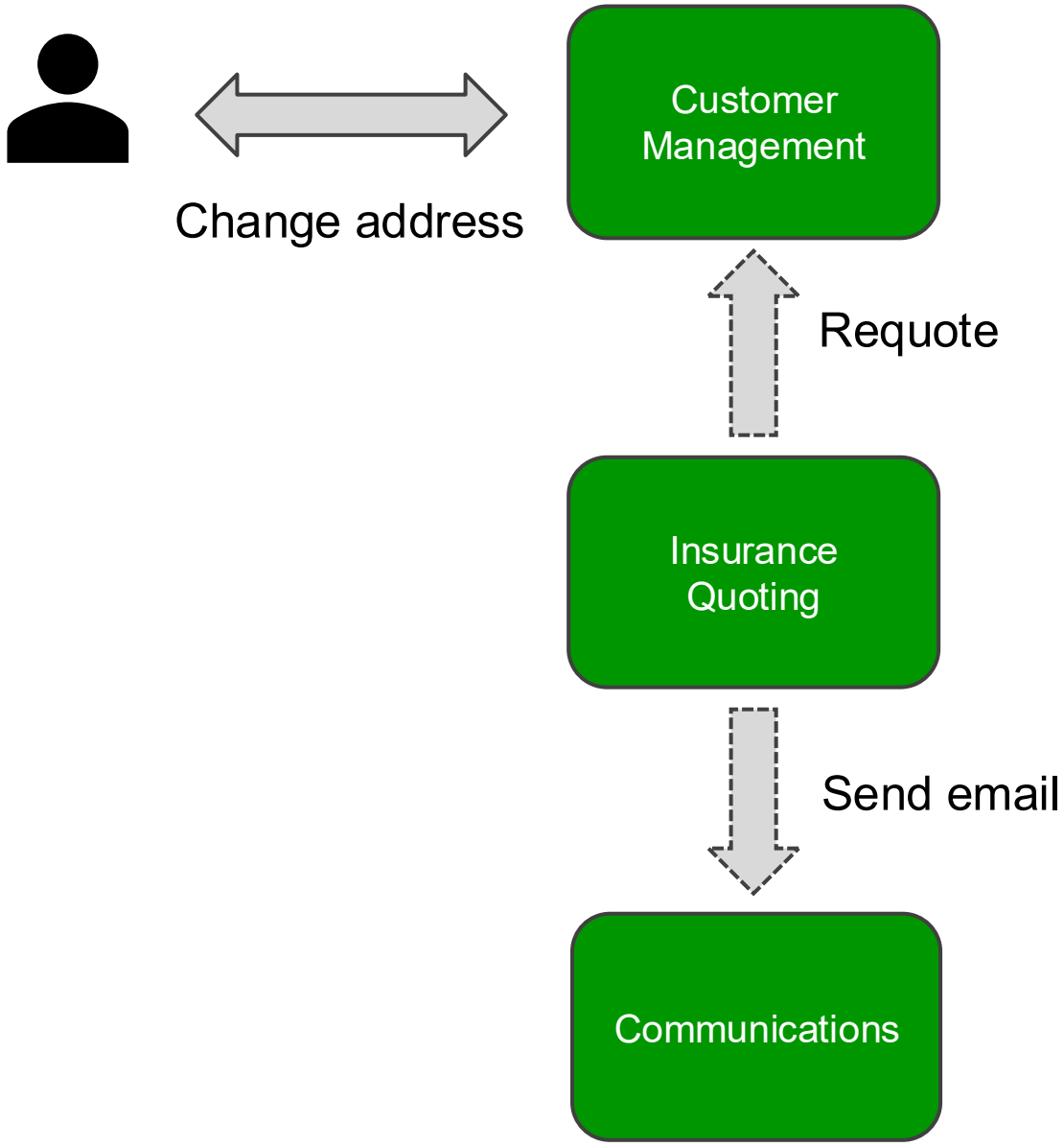
A Fictionary System at an Insurance Company



- Having a request-response communication style in place the Customer Management needs to tell the Quoting System to do a requote
- Creates dependency between Customer Management and Insurance Quoting
- Couples knowledge of how quoting works into Customer Management

Source: The Many Meanings of Event-driven Architecture by Martin Fowler

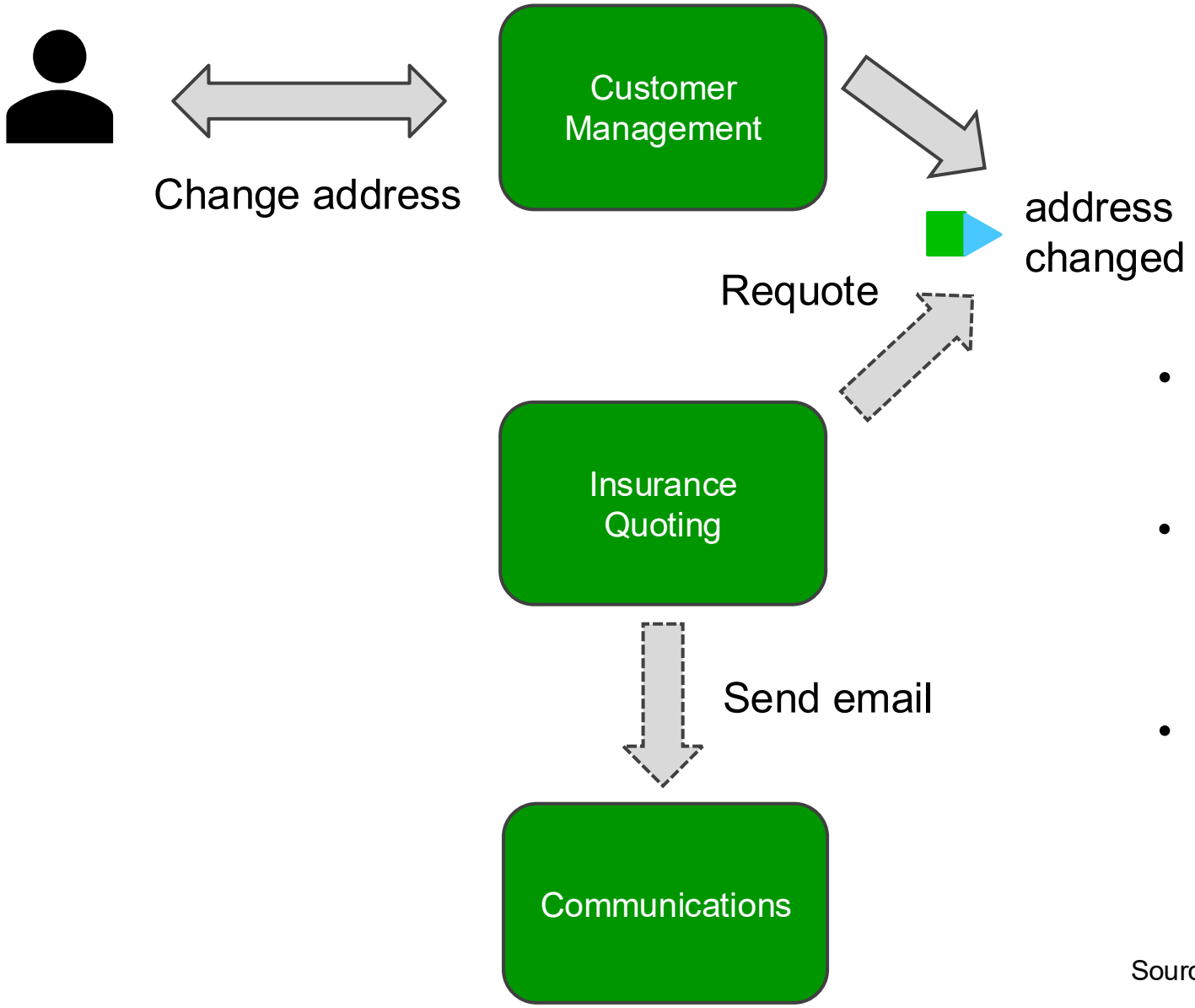
A Fictionary System at an Insurance Company



- A reversed dependency is however much more natural

Source: The Many Meanings of Event-driven Architecture by Martin Fowler

Event Notification



- Using events, the dependencies can be inverted
- The Customer Management emits an event whenever the address changes **(event notification)**
- Instead of the Customer Management depending on the Insurance Quoting, the Insurance Quoting can listen to address changes and respond to them

Source: The Many Meanings of Event-driven Architecture by Martin Fowler

Event Notification

- **What is event notification?**
  - Events used as a notification mechanism between systems
    - Large scale: between different microservices
    - Small scale: between UI and application logic

- **What does it change?**
  - **Dependency Inversion:** producers do not know consumers
  - **Change** becomes **a first-class citizen** - we record the fact that something happened

Source: The Many Meanings of Event-driven Architecture by Martin Fowler

Events or Commands

When do we have events and when do we have commands?

- Requote Insurance Command
  - Phrasing something as a command suggests that the system invoking the command knows what should happen and tells the remote system what to do (i.e., Customer Management knows what Insurance Quoting should do)
- Address Change Event
  - Notifies about the address change and leaves it to other systems to react to this event; the component emitting the event does not know who picks it up and why; no particular response is expected

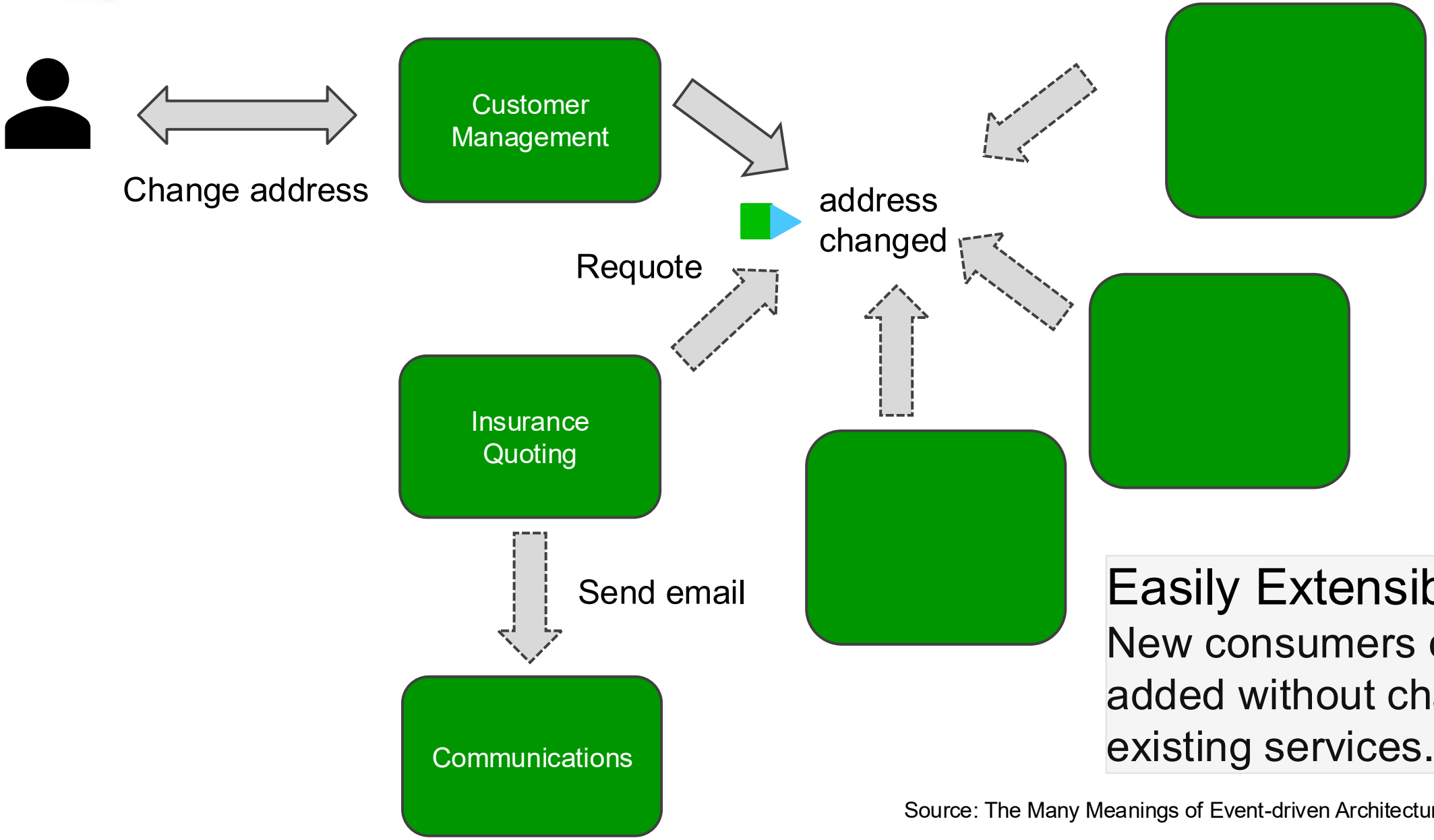
Naming is important!

Source: The Many Meanings of Event-driven Architecture by Martin Fowler

Events or Commands

Commands = synchronous and events = asynchronous?

Commands	Events
Represent the **intent** for something to happen in the system that leads to a desired outcome.	Represent something that **has happened in the past** and other parts of the same domain (domain events) or other domains (integration events) should be aware about.
Commands are characterized by their semantics, not the technical protocol. This leaves open how the command should be triggered or transmitted (synchronous or asynchronous).	Domain events are typically done within the same transactional scope (synchronously or asynchronously), while integration events get published asynchronously via an event bus.



Easily Extensible:
New consumers can be added without changing existing services.

Source: The Many Meanings of Event-driven Architecture by Martin Fowler

Trade-offs Event Notification

Event Notification

Decouple receiver from sender ✓

Does not describe end-to-end system behavior ✗

To describe overall system behavior, we need richer event-driven patterns.

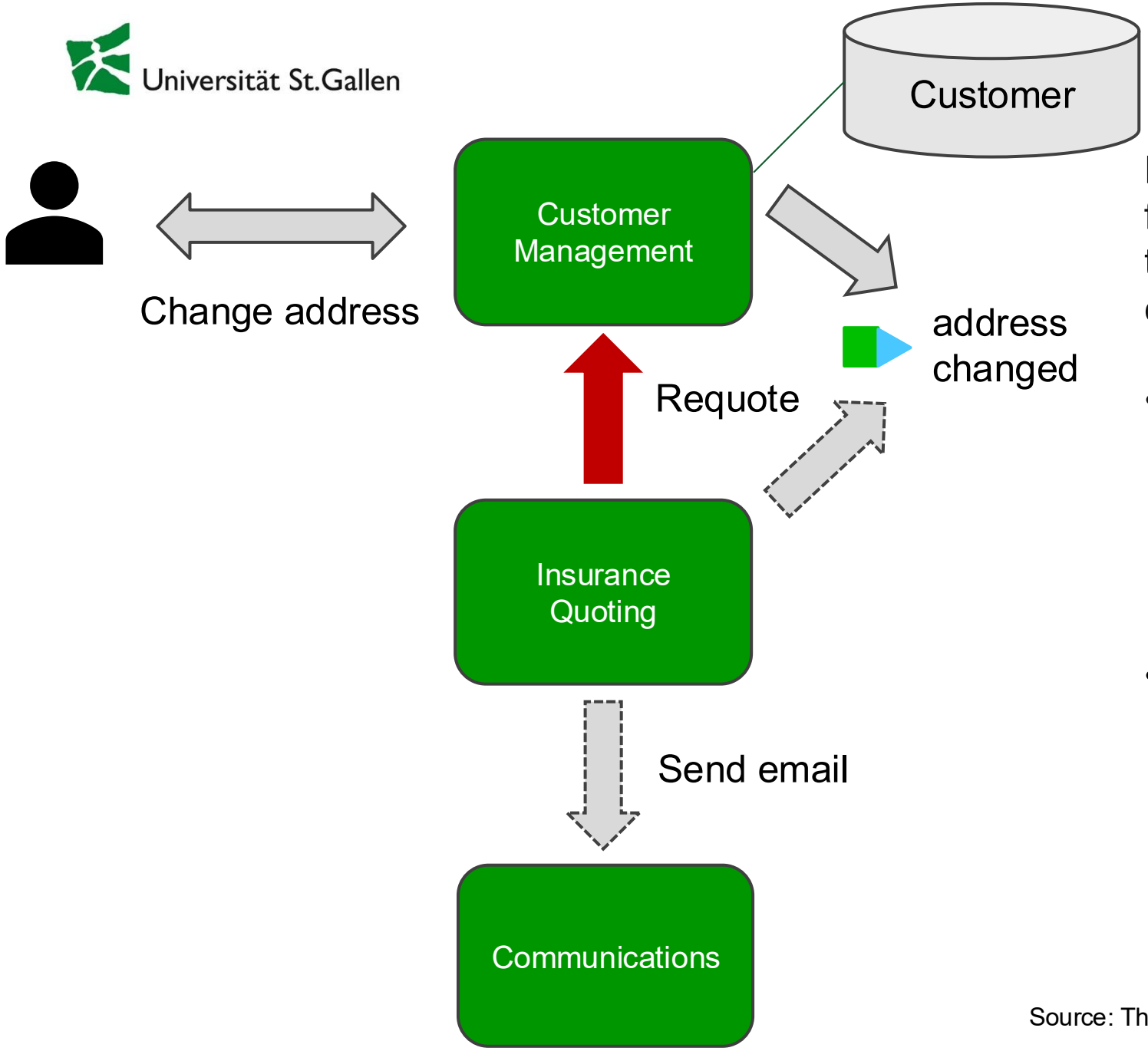
Source: The Many Meanings of Event-driven Architecture by Martin Fowler

The Many Meanings of Event-driven Architecture

Event–driven Architecture (EDA). An architectural paradigm promoting the production, consumption of, and reaction to events.

- Four Patterns
  - Event notification
  - **Event-carried State Transfer**
  - Event sourcing
  - Command Query Responsibility Segregation (CQRS)

Source: The Many Meanings of Event-driven Architecture by Martin Fowler

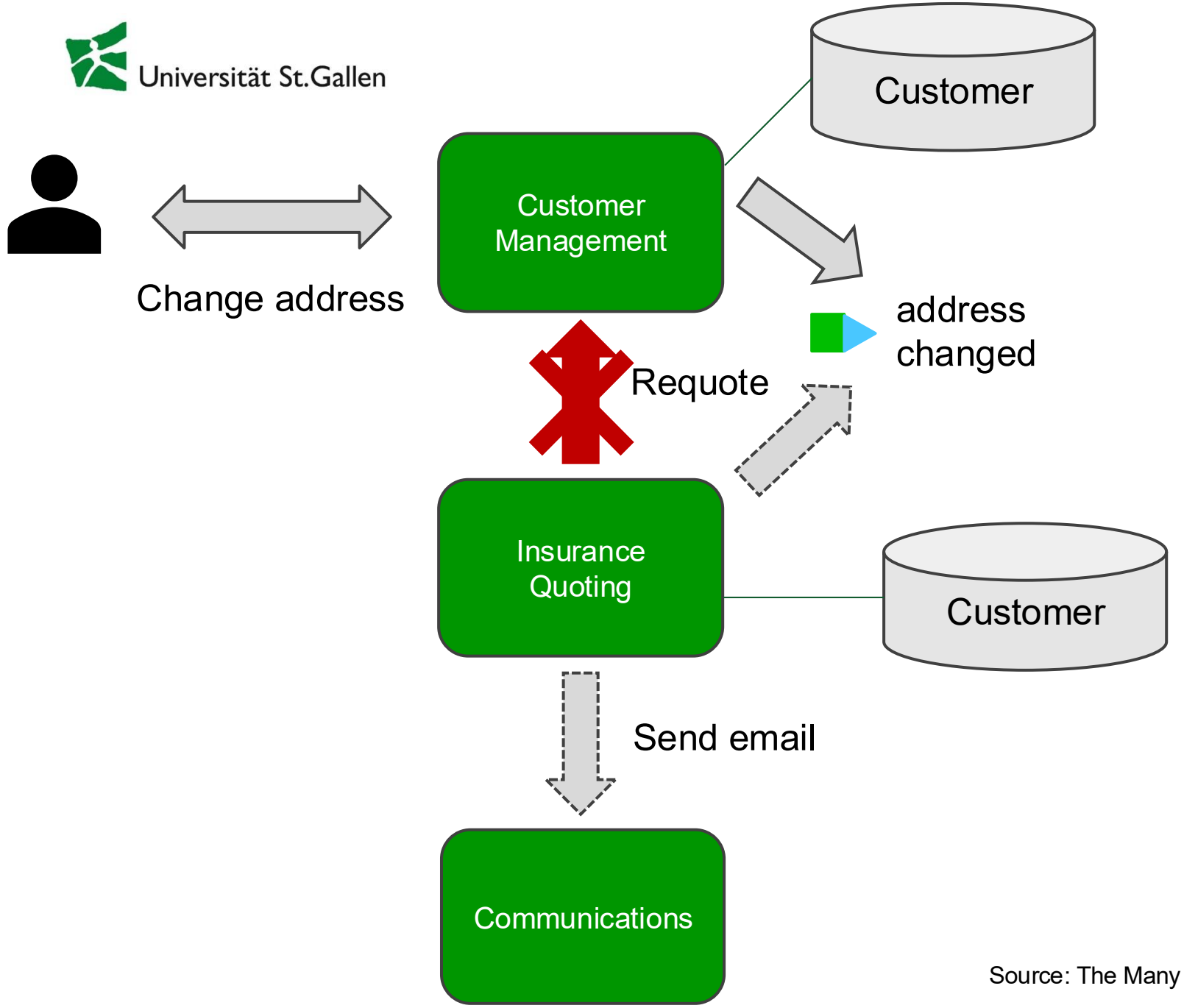


Event notification tells you that a domain fact occurred. Event-carried state transfer tells you the new state needed by consumers

- **With event-notification:**
  - Consumers still need synchronous queries to read state
  - Producers becomes a read bottleneck
  - Higher runtime coupling
- **With event-carried state transfer:**
  - State is included in the event
  - No synchronous read calls required
  - Consumers maintain local read models
  - Lower read-time coupling

Source: The Many Meanings of Event-driven Architecture by Martin Fowler

Event-carried State Transfer



- Events carry the state needed by consumers.
- No communication between Insurance Quoting and Customer Management
- The Insurance Quoting (downstream system) keeps a local copy of the required data
- Customer Management broadcasts all changes using events
- Improves performance, reduces load, improves availability

Source: The Many Meanings of Event-driven Architecture by Martin Fowler

Trade-offs Event-carried State Transfer

Decoupling ✓

Reduced load on supplier ✓

Event-carried State Transfer → Replicated Data → Eventual consistency ✗

Source: The Many Meanings of Event-driven Architecture by Martin Fowler

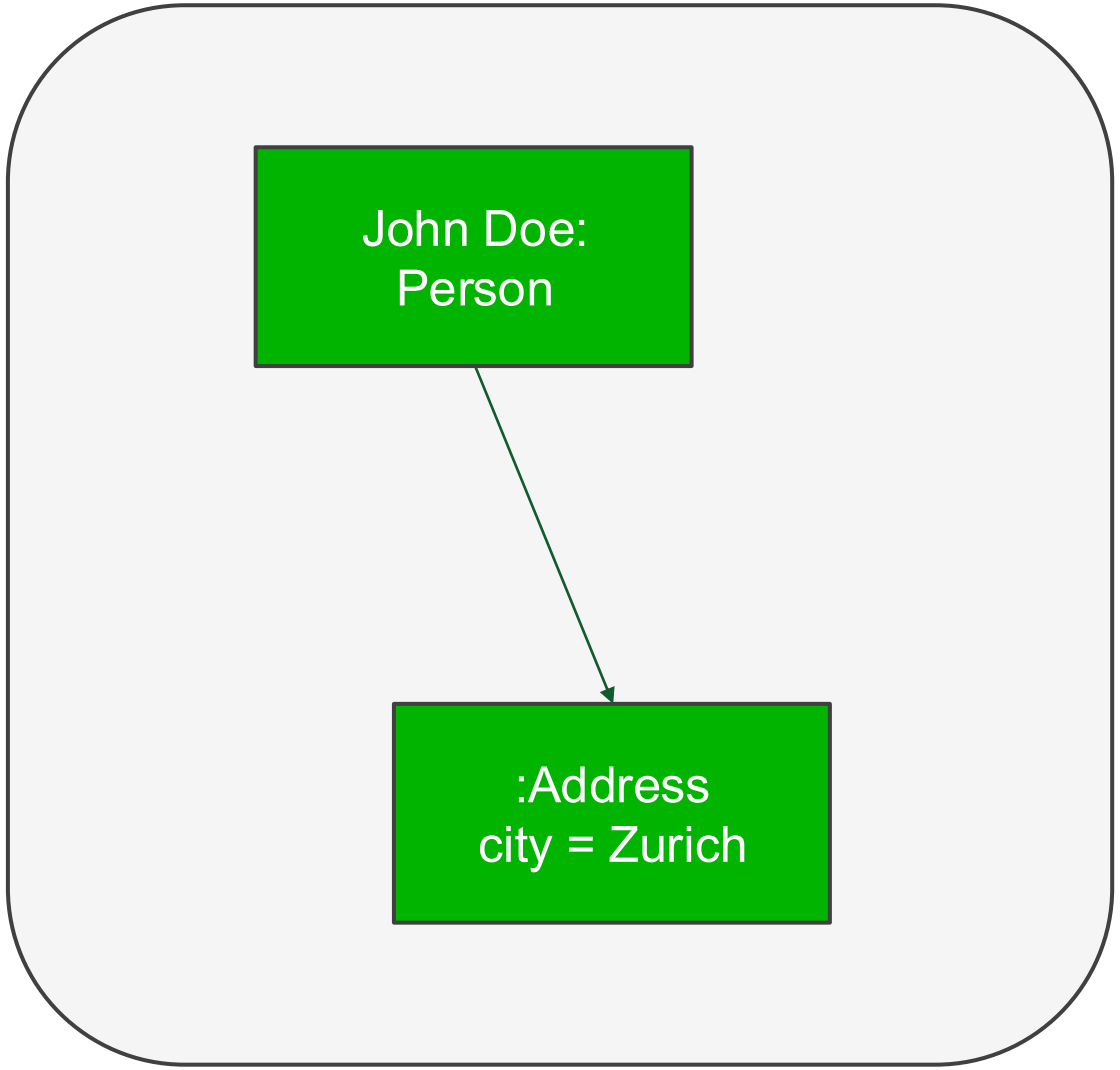
The Many Meanings of Event-driven Architecture

Event–driven Architecture (EDA). An architectural paradigm promoting the production, consumption of, and reaction to events.

- Four Patterns
  - Event notification
  - Event-carried State Transfer
  - **Event sourcing**
  - Command Query Responsibility Segregation (CQRS)

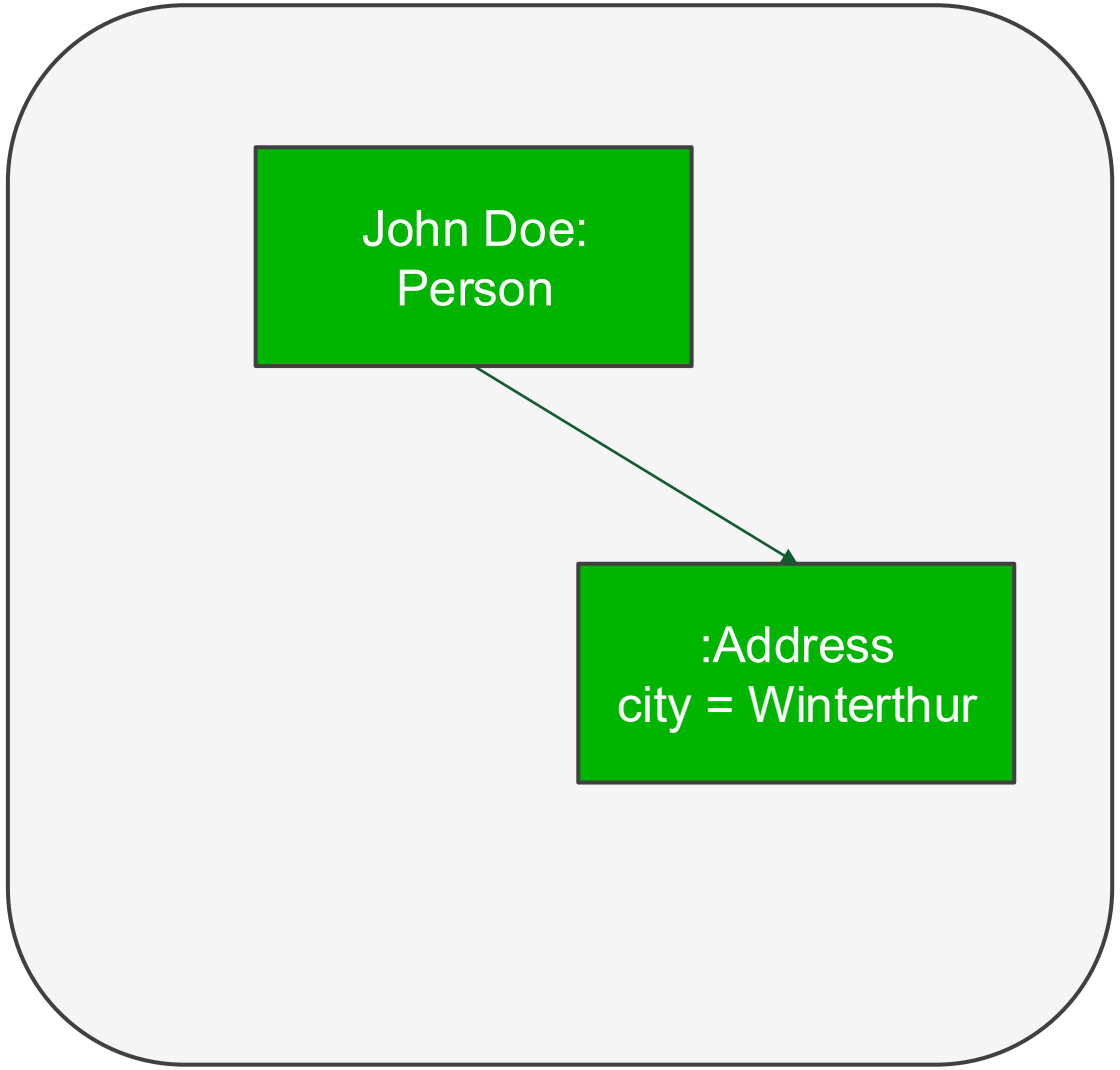
Source: The Many Meanings of Event-driven Architecture by Martin Fowler

Changes in a Traditional System



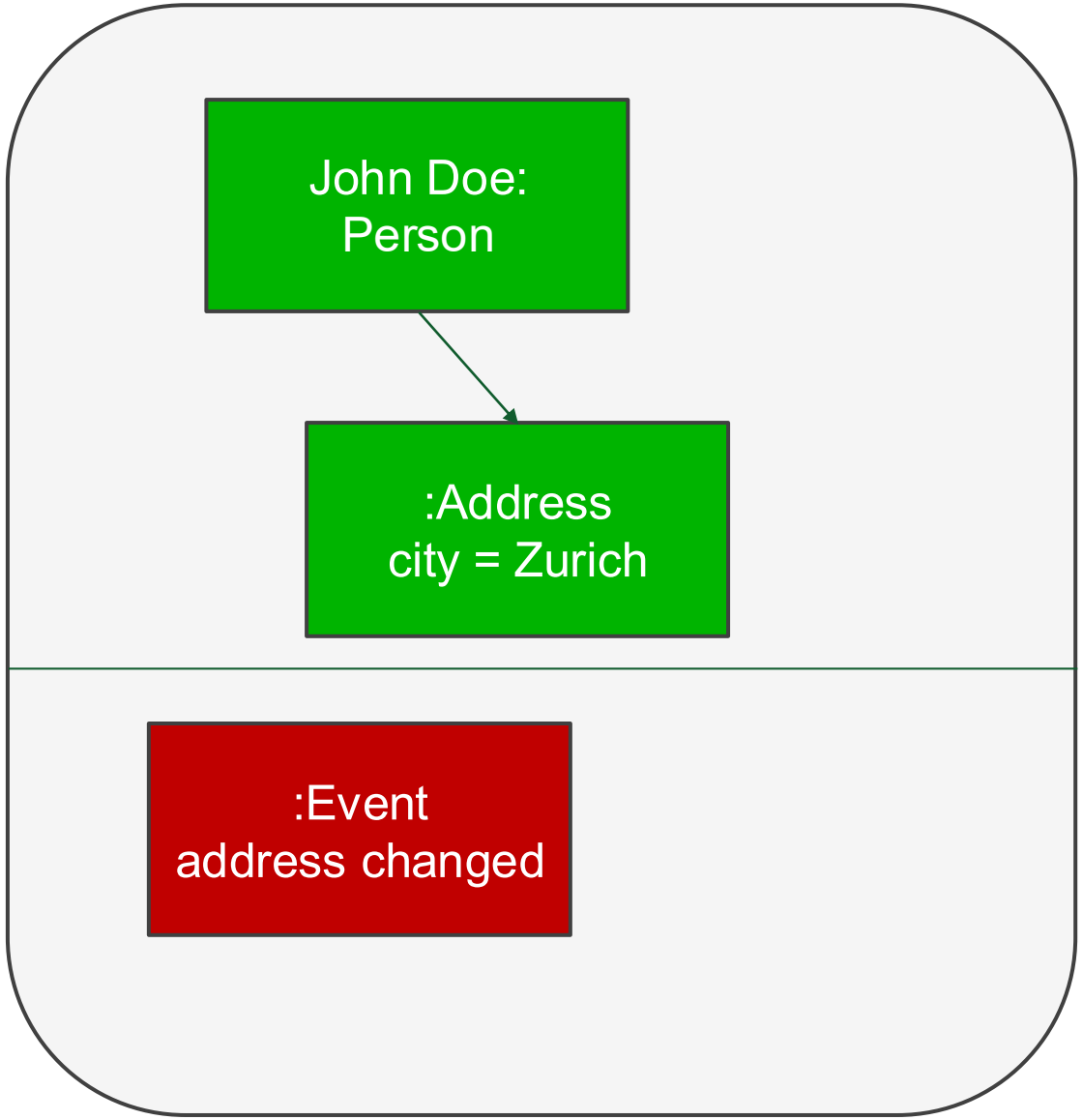
Source: The Many Meanings of Event-driven Architecture by Martin Fowler

Changes in a Traditional System



Source: The Many Meanings of Event-driven Architecture by Martin Fowler

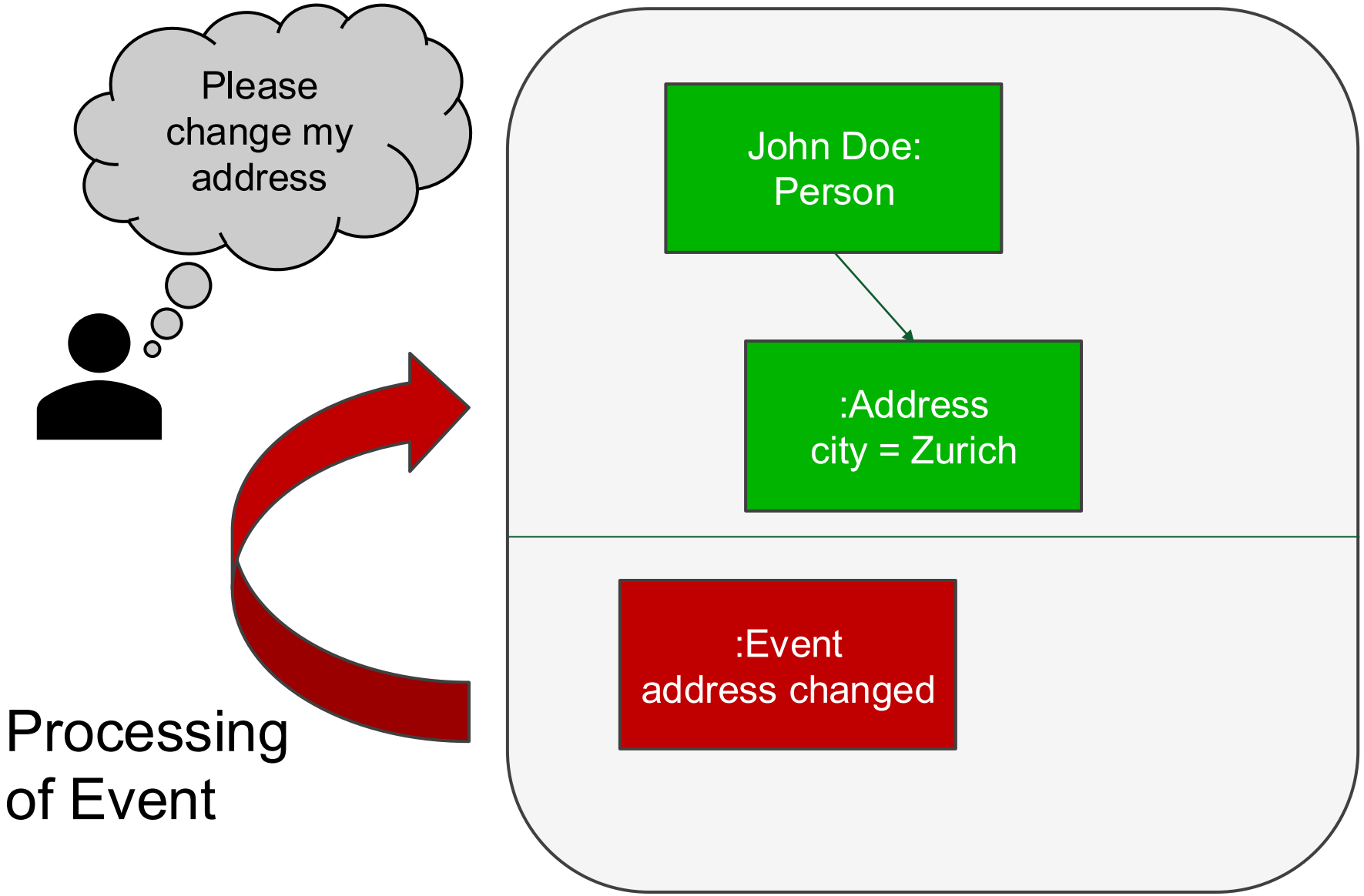
Changes in an Event-source System



Event Sourcing means:
The **event log is the source of truth**, and application **state is a projection** derived from it.

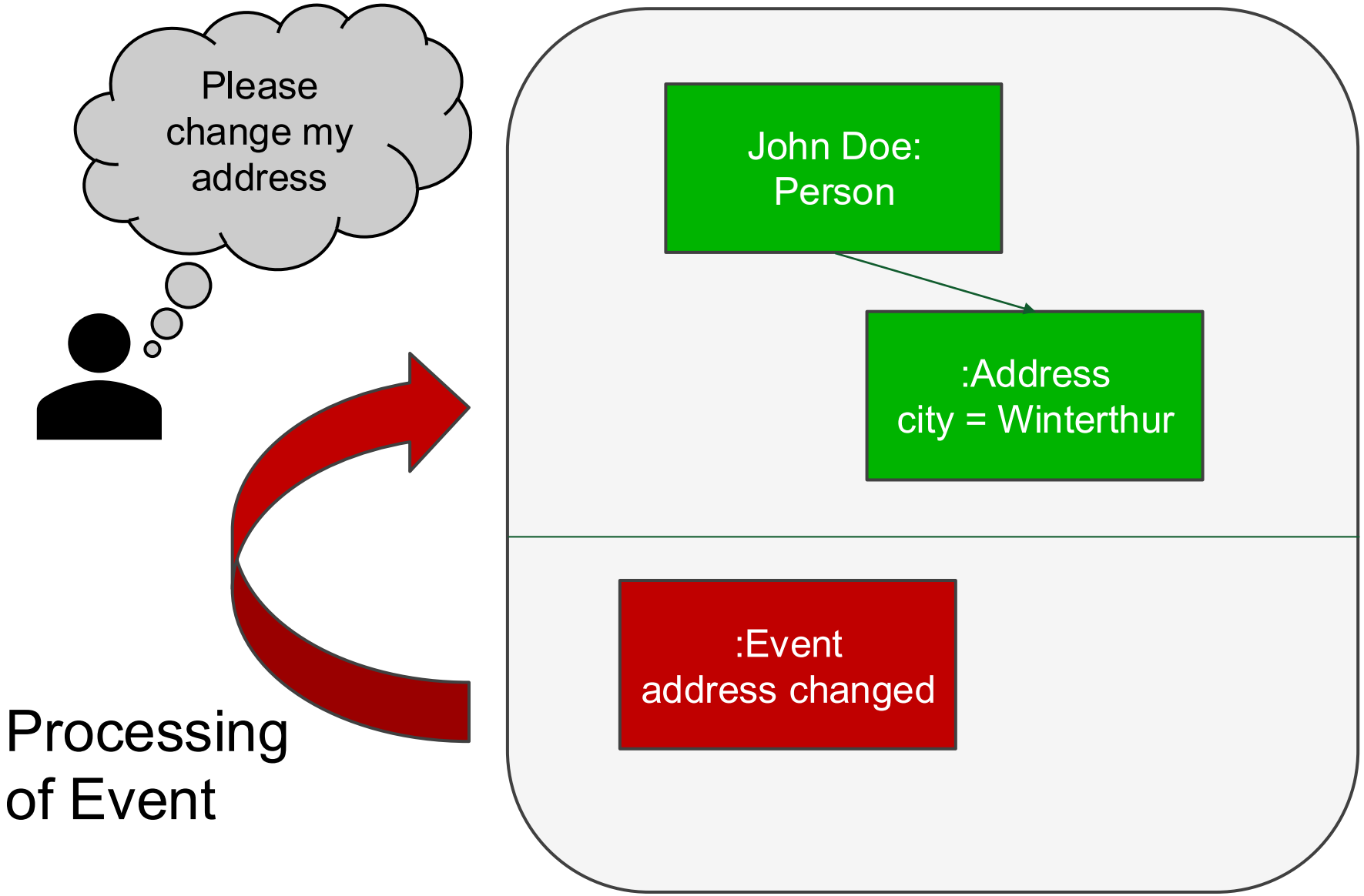
Source: The Many Meanings of Event-driven Architecture by Martin Fowler

Changes in an Event-source System



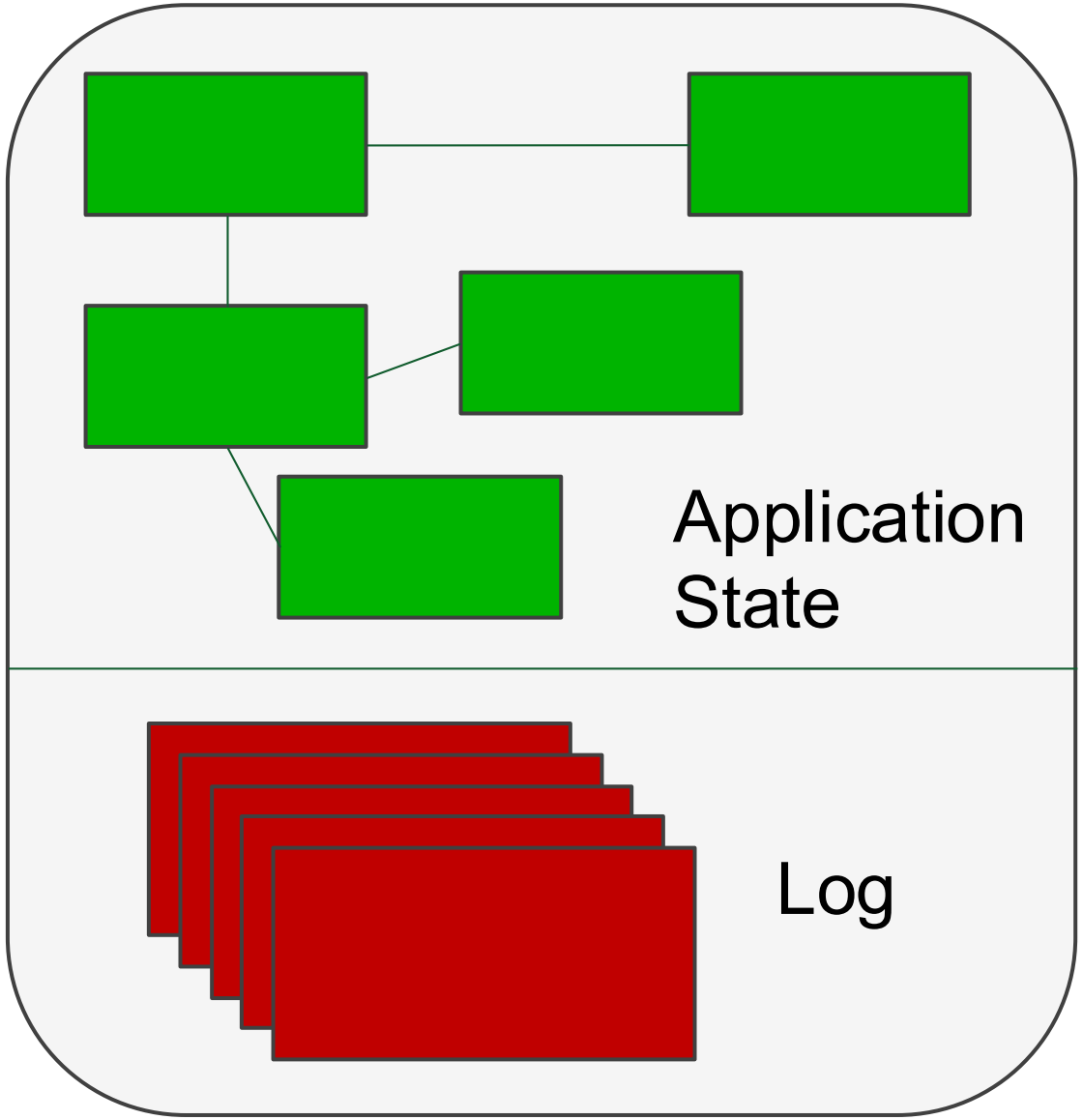
Source: The Many Meanings of Event-driven Architecture by Martin Fowler

Changes in an Event-source System



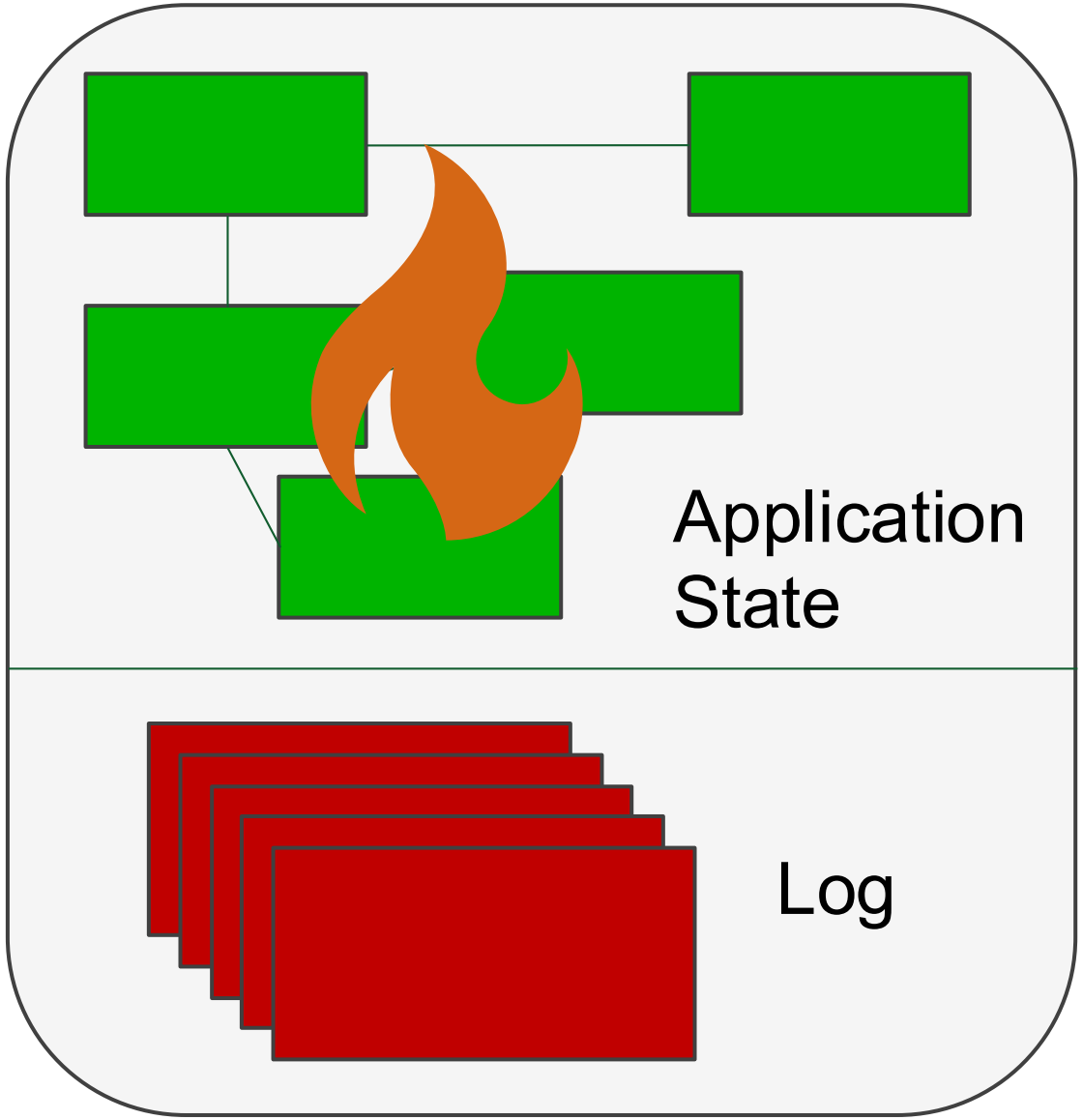
Source: The Many Meanings of Event-driven Architecture by Martin Fowler

Characteristics of an Event-source System



Source: The Many Meanings of Event-driven Architecture by Martin Fowler

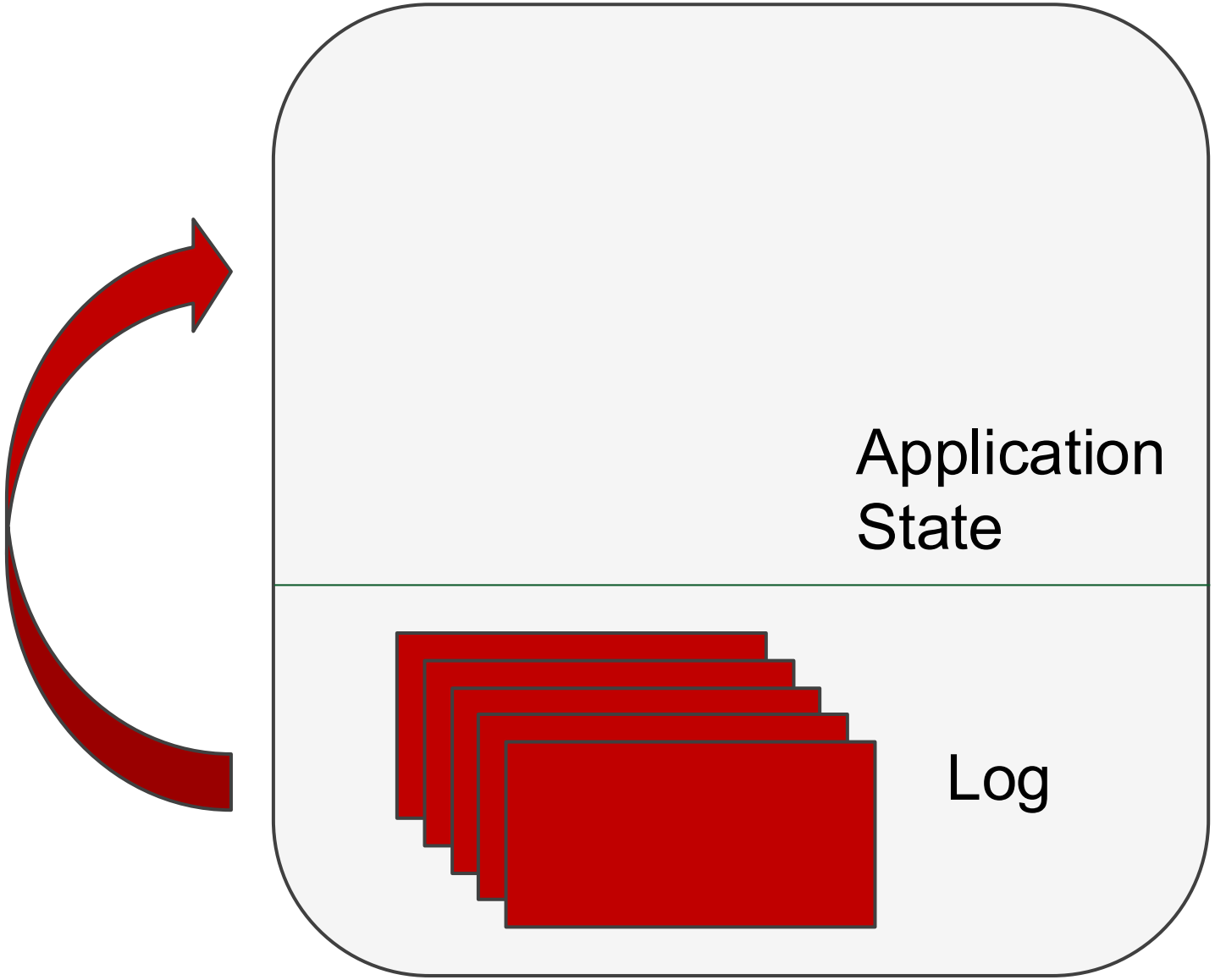
Rebuilding Application State From Log



Source: The Many Meanings of Event-driven Architecture by Martin Fowler

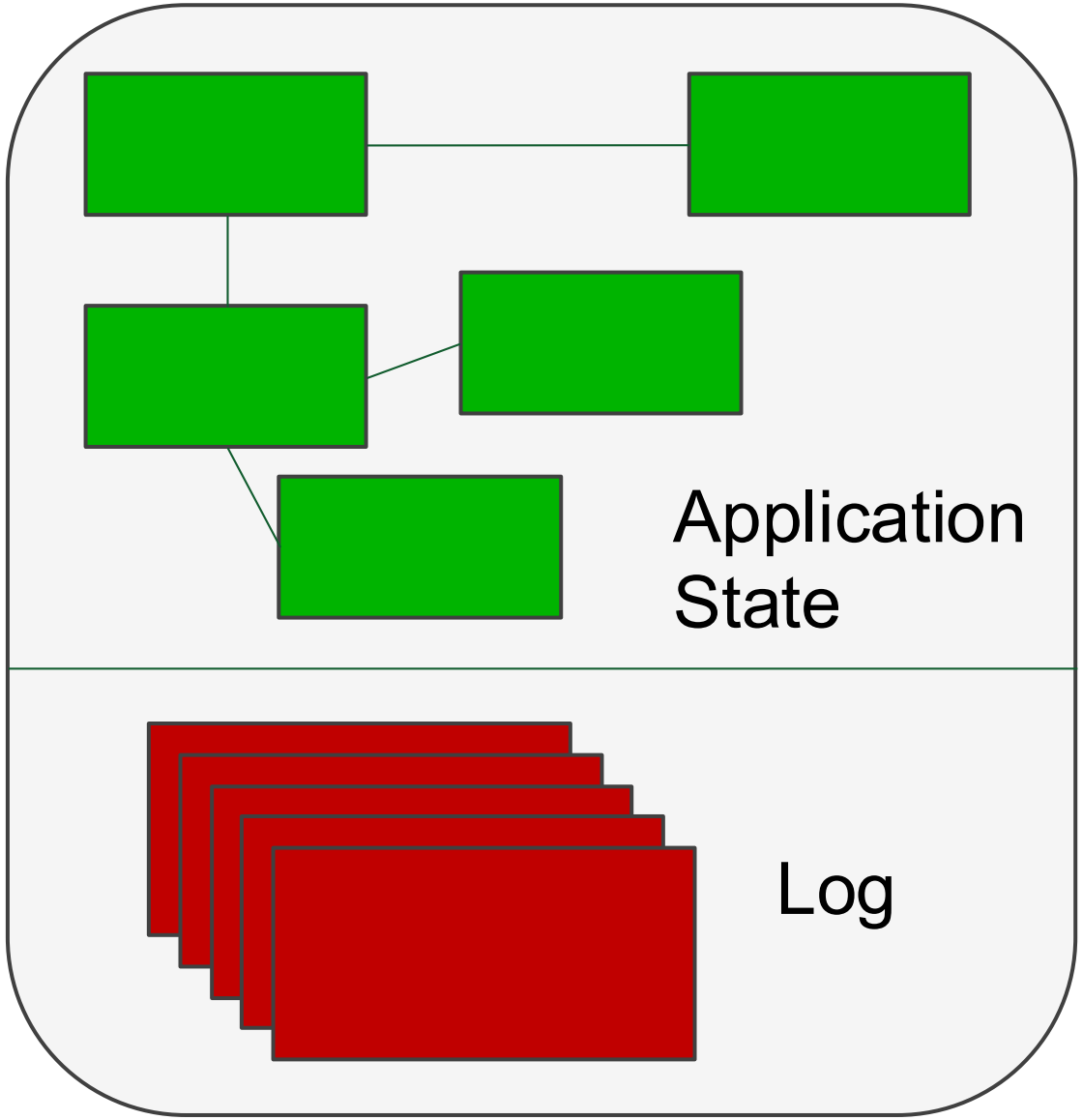
Rebuilding Application State From Log

Confidently rebuild application state from log



Source: The Many Meanings of Event-driven Architecture by Martin Fowler

Rebuilding Application State From Log



Source: The Many Meanings of Event-driven Architecture by Martin Fowler

What we can learn from accounting

- Examples of event-source systems
  - Double-entry bookkeeping in accounting
    - Transactions (events) are recorded and balances (state) are derived from them
  - Version control systems like Git
    - A git commit stores a message describing the intended change and a SHA1 hash which acts as a pointer to the new state of the repository

- Principles of event sourcing
  - events are persisted as the system of record
  - state is a projection derived from these events
  - snapshots are performance oprimizations, not the source of truth

Source: https://dev.to/dealeron/event-sourcing-and-the-history-of-accounting-1aah

Trade-offs Event Sourcing

✓ Benefits

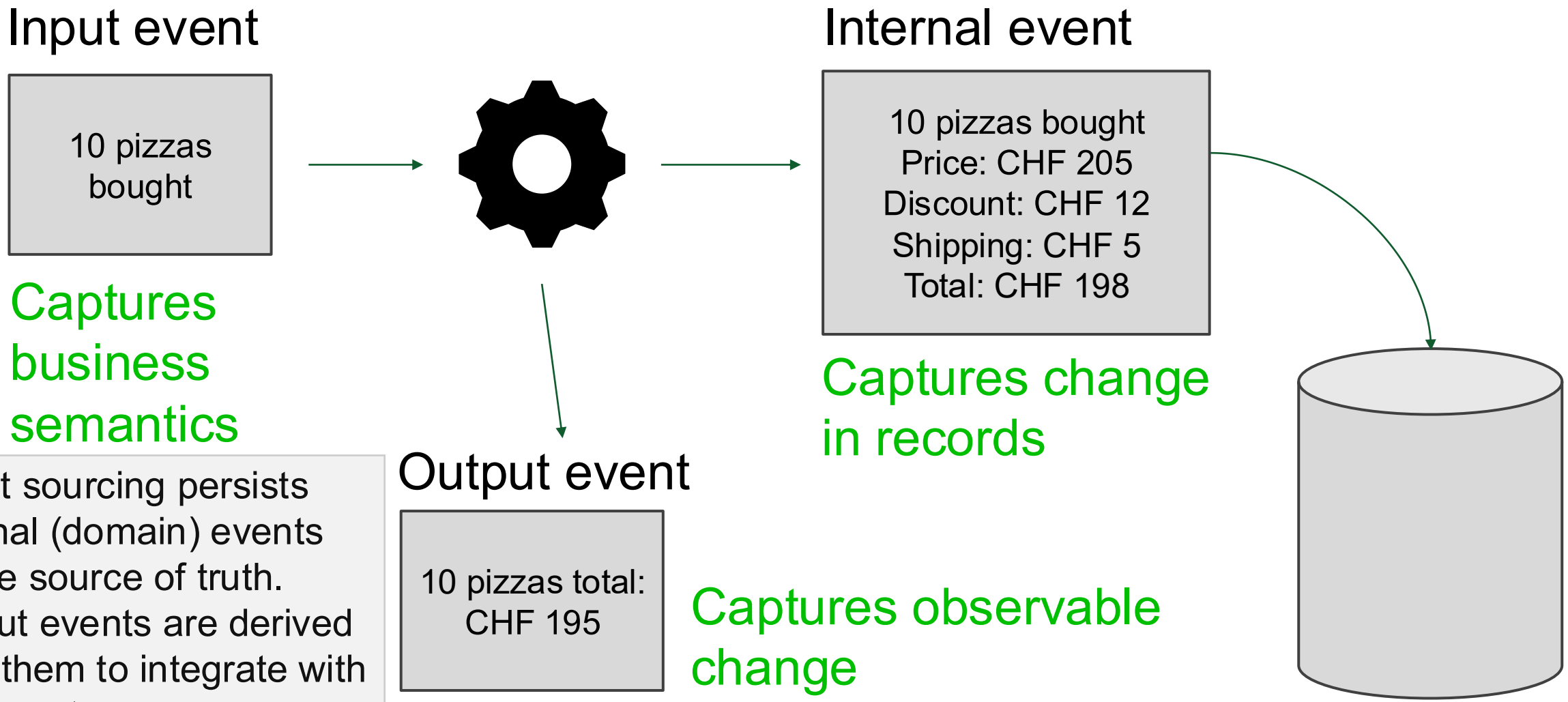
- Auditability
- Powerful debugging & replay
- Full history of state
- Multiple projections / read models
- In-memory reconstruction („Memory image“)

✗ Operational and Design Challenges

- Unfamiliar development model
- Integration with external systems
- Event schema evolution & versioning
- Identifier and ordering complexity
- Higher implementation complexity & tooling needs

Source: The Many Meanings of Event-driven Architecture by Martin Fowler

Input event, internal event, output event



Event sourcing persists internal (domain) events as the source of truth. Output events are derived from them to integrate with other systems.

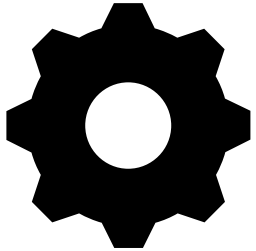
Source: The Many Meanings of Event-driven Architecture by Martin Fowler

Input event, internal event, output event

Input event

10 pizzas bought

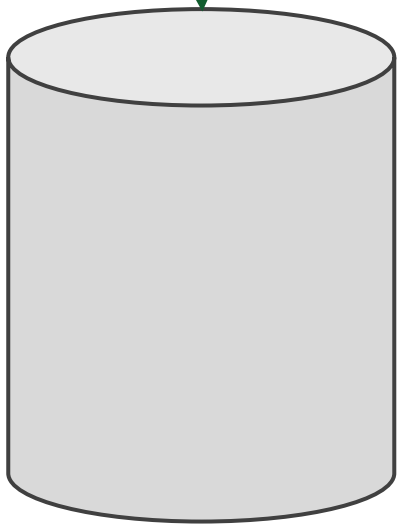
Captures business semantics



Internal event

10 pizzas bought
Price: CHF 205
Discount: CHF 12
Shipping: CHF 5
Total: CHF 198

Captures change in records



Output event

10 pizzas total:
CHF 195

Captures observable change

The event log is the source of truth.
The projection is wrong.
Replaying the log rebuilds the projection → CHF 198.

Source: The Many Meanings of Event-driven Architecture by Martin Fowler

The Many Meanings of Event-driven Architecture

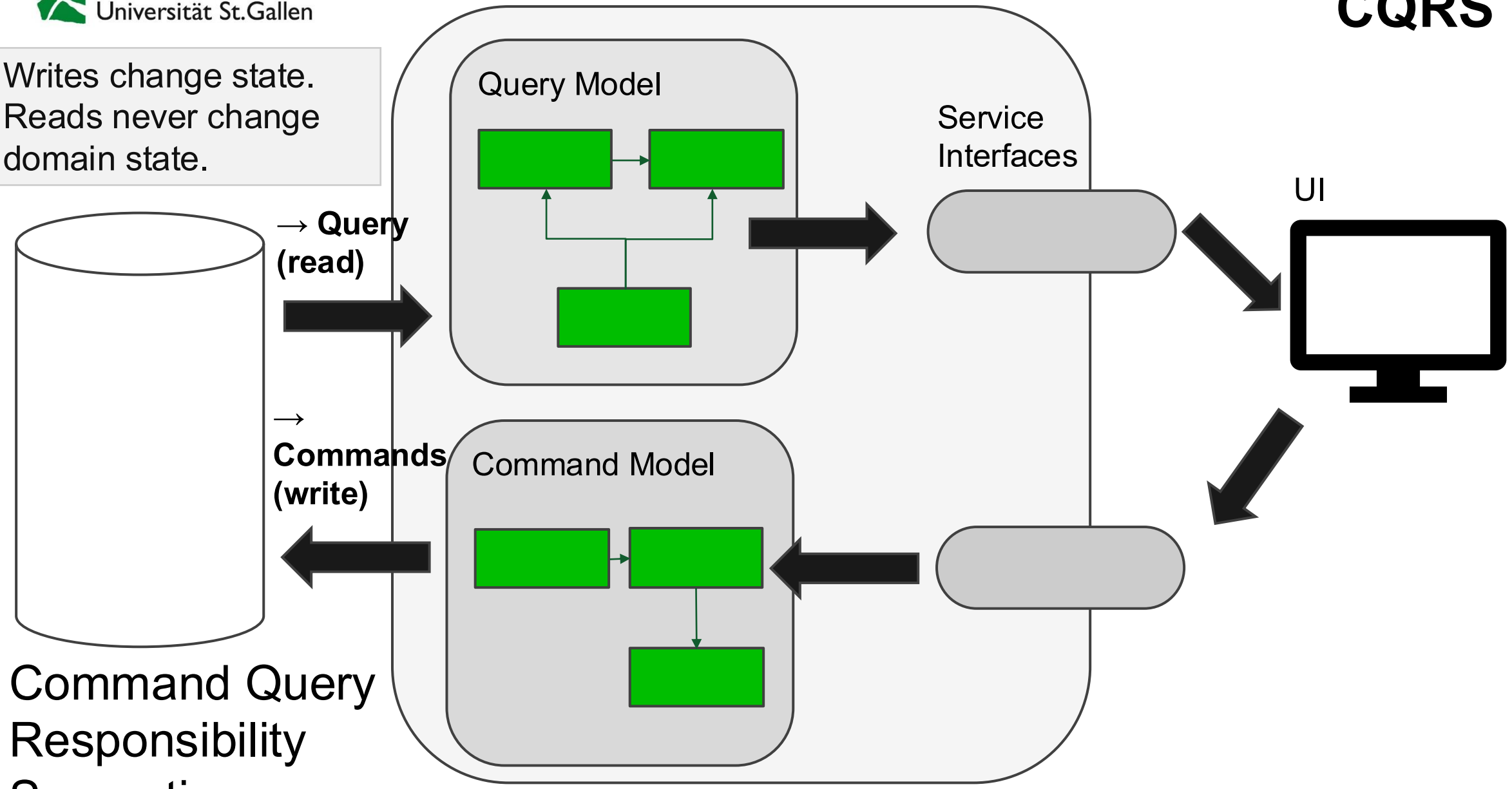
Event–driven Architecture (EDA). An architectural paradigm promoting the production, consumption of, and reaction to events.

- Four Patterns
  - Event notification
  - Event-carried State Transfer
  - Event sourcing
  - **Command Query Responsibility Segregation (CQRS)**

Source: The Many Meanings of Event-driven Architecture by Martin Fowler

CQRS

Writes change state.
Reads never change domain state.

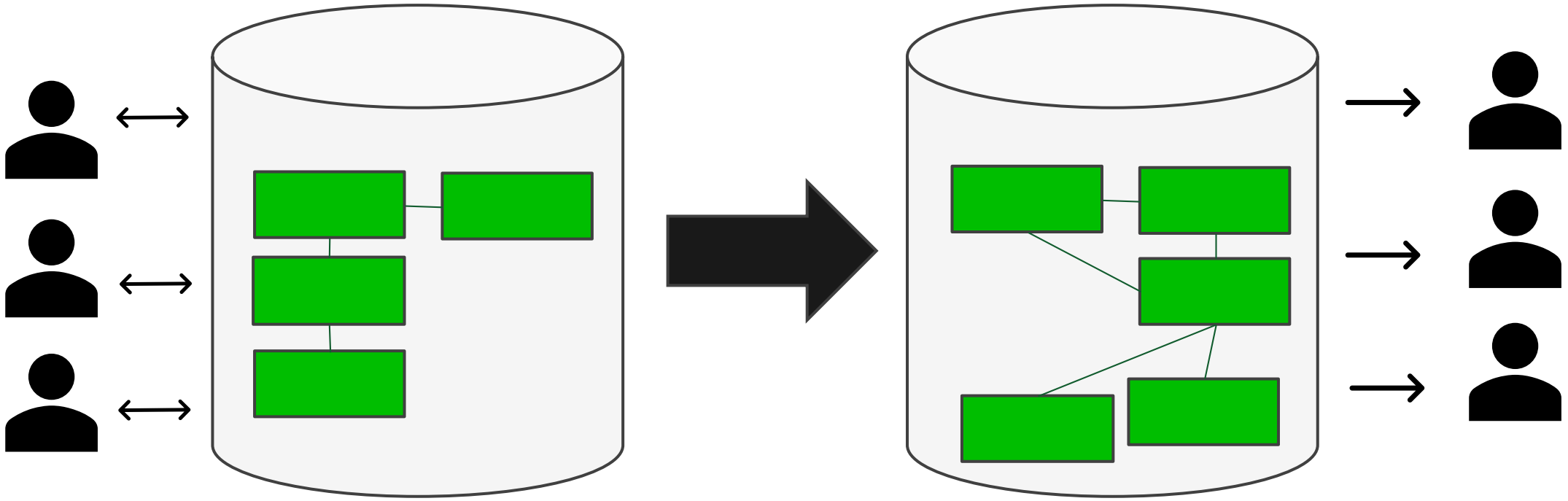


Command Query Responsibility Separation

Source: The Many Meanings of Event-driven Architecture by Martin Fowler

What CQRS is Not

CQRS separates write models from read models so each can be optimized independently. This does not require a separate reporting database.



Operating Database

❌ Reporting Database

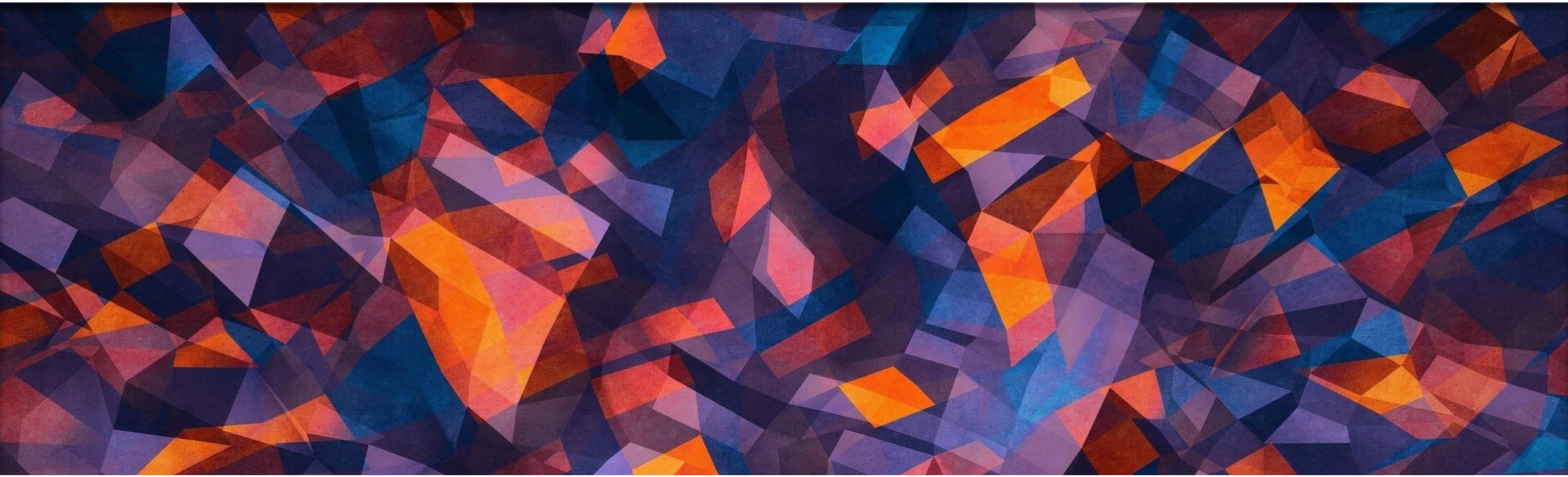
Common misconception: CQRS = reporting replica.

Source: The Many Meanings of Event-driven Architecture by Martin Fowler

Further References

- „The Many Meanings of Event Driven Architecture“
  - https://www.youtube.com/watch?v=STKCRSUsyP0
- Associated blog post by Martin Fowler „What do you mean by-event driven?“
  - https://martinfowler.com/articles/201701-event-driven.html

Universität St.Gallen

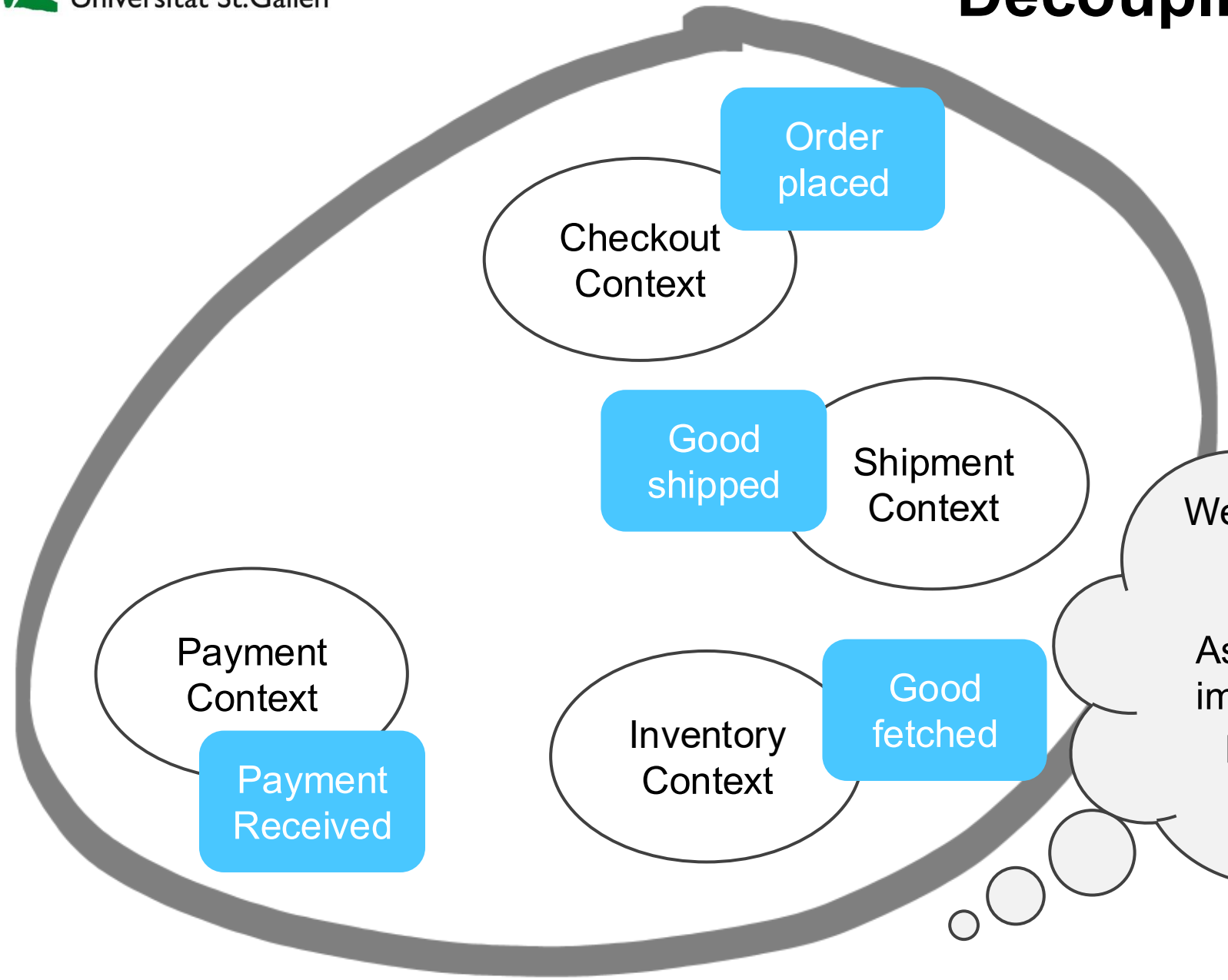


Event-driven and Process-oriented Architectures

Motivating Example (Part 2)

Lecture 2

Decoupling using Events

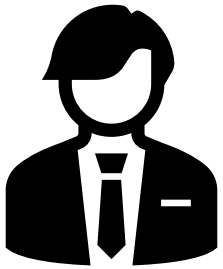


- Services emit events to inform about important state changes

We have seen how events help to decouple and increase autonomy. Assume you now want to implement an end-to-end process like the order fulfillment flow.

- We should only use **choreography** in our distributed architecture to keep **everything decoupled**. I read that it is always a good idea to keep things as much decoupled as possible and choreography is the most decoupled coordination pattern

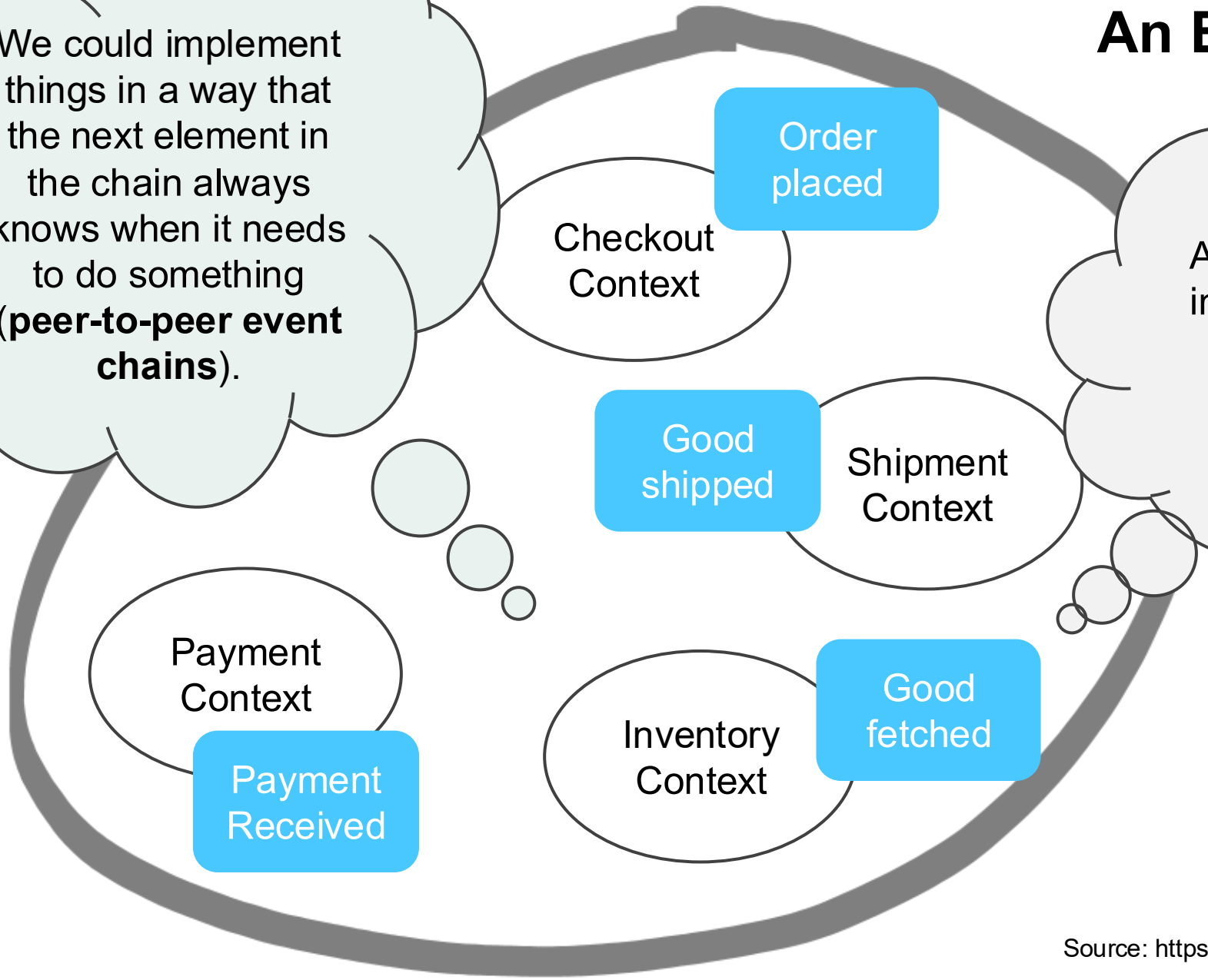
- It‘s never good to say always when it comes to software architecture. My mentor told me to not use absolutes when it comes to software architecture like never or always, since what makes a good architecture always depends. Let‘s first see.



Order Fulfillment Flow
An End-to-end Process

We could implement things in a way that the next element in the chain always knows when it needs to do something (**peer-to-peer event chains**).

Assume you now want to implement an end-to-end process like the order fulfillment flow.

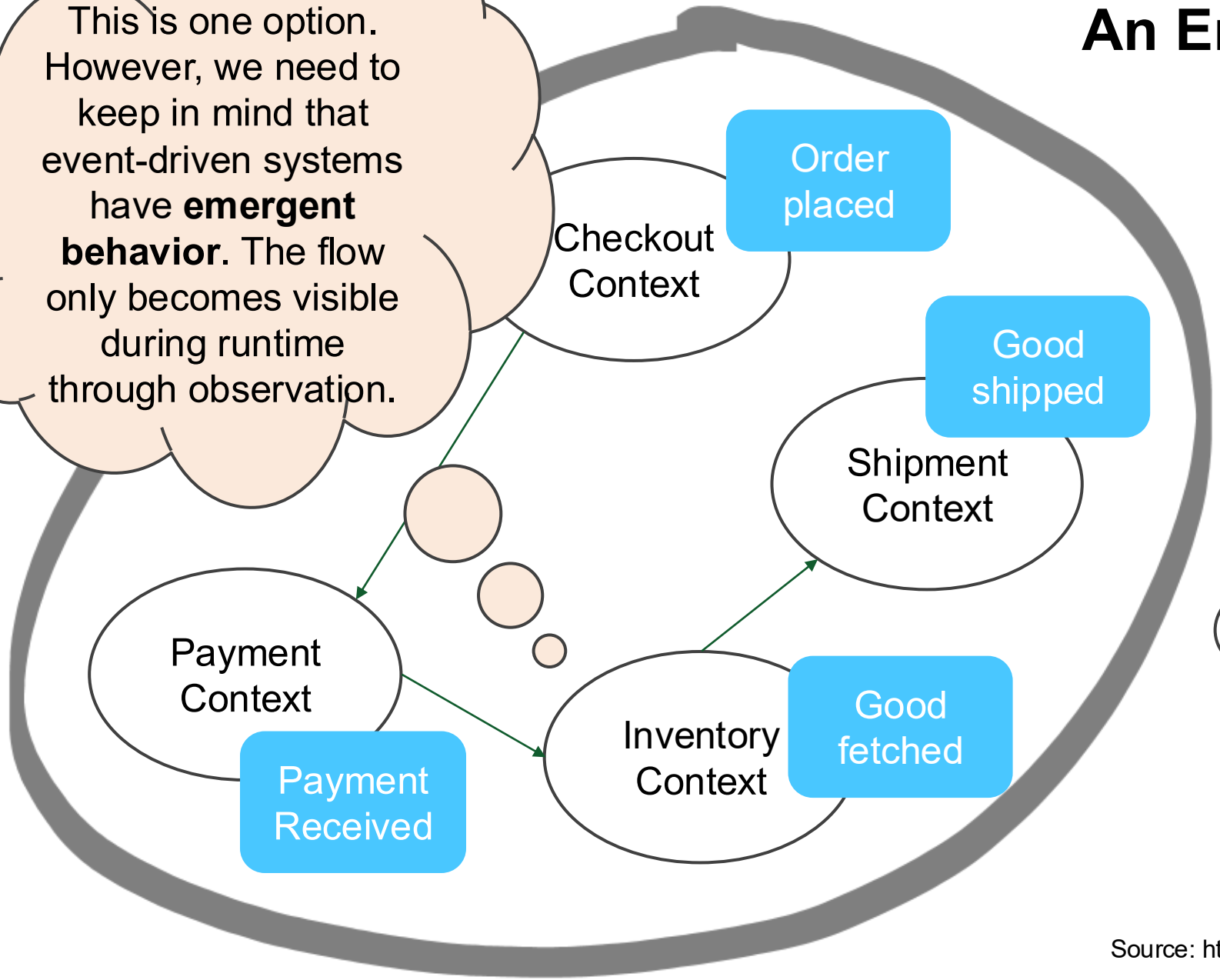


Each service reacts to events and emits new ones – no central coordinator.

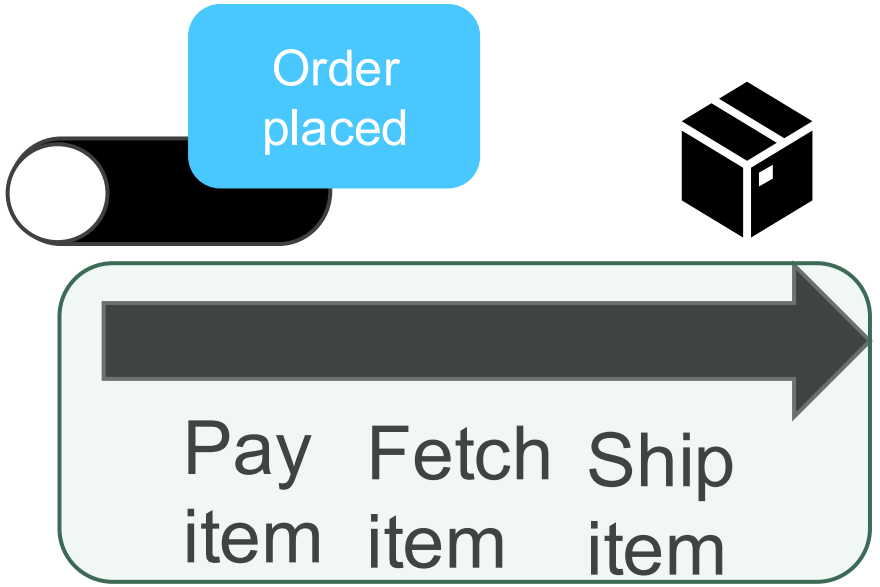
Source: https://www.infoq.com/articles/events-workflow-automation/

Order Fulfillment Flow
An End-to-end Process

This is one option. However, we need to keep in mind that event-driven systems have **emergent behavior**. The flow only becomes visible during runtime through observation.

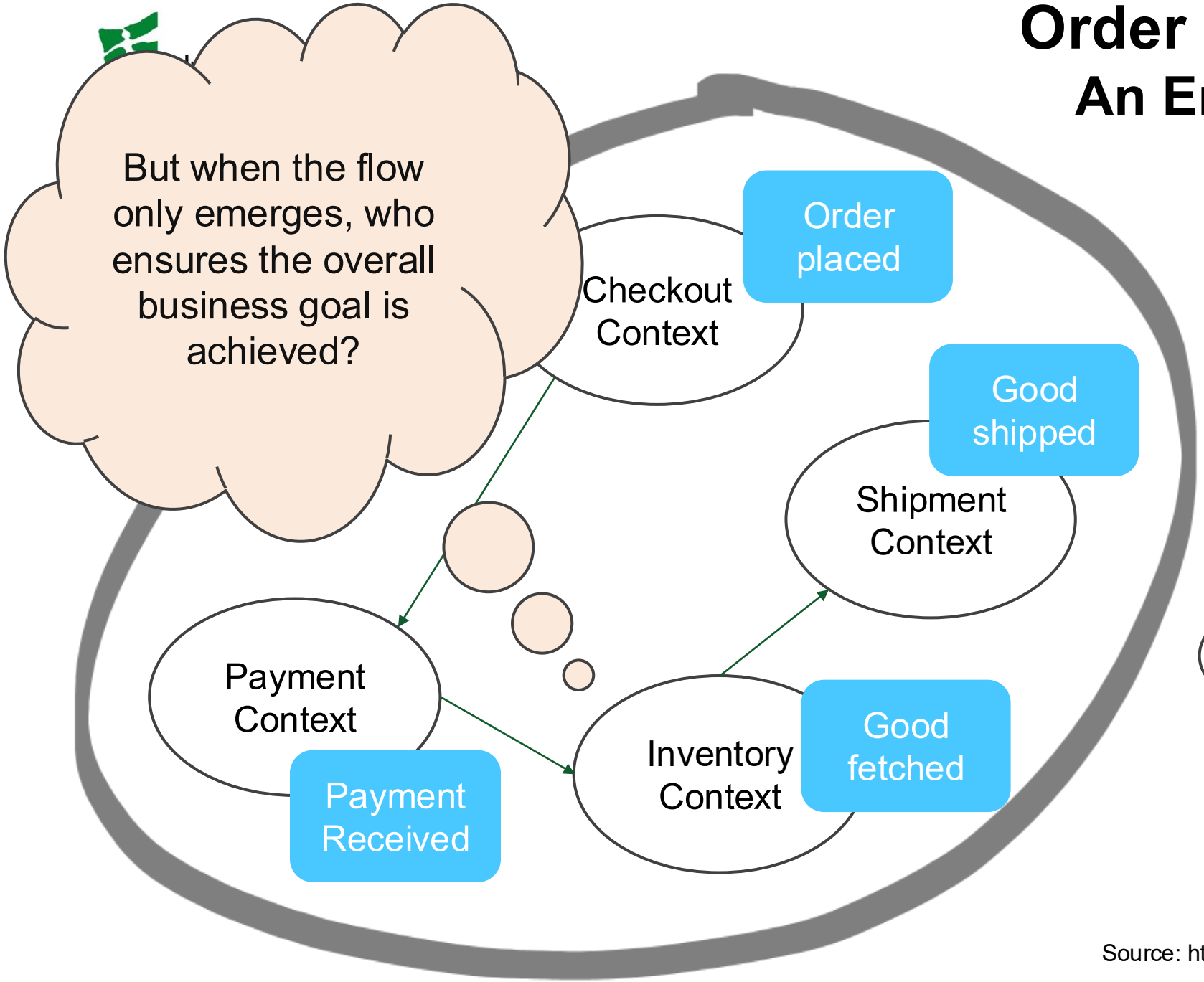


- When the button is pressed an order is placed and fulfilled

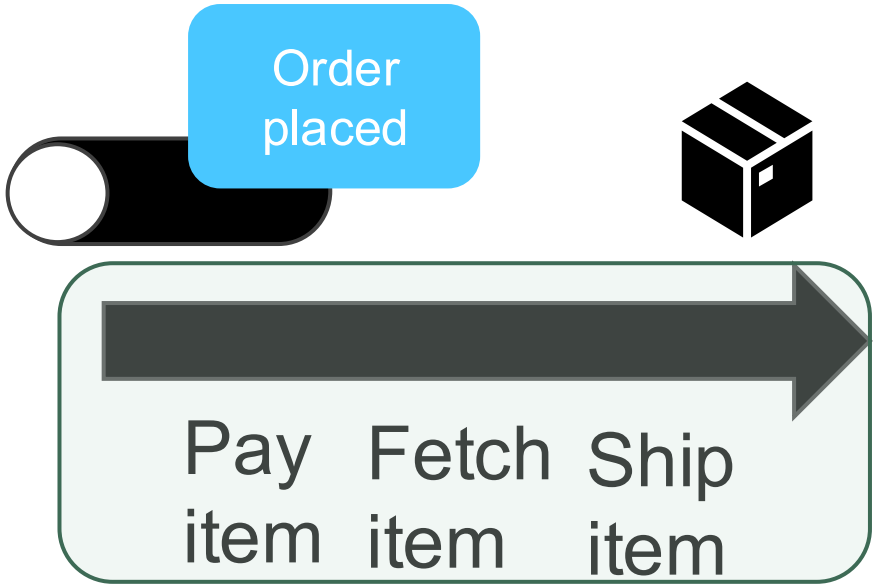


Source: https://www.infoq.com/articles/events-workflow-automation/

Order Fulfillment Flow
An End-to-end Process



- When the button is pressed an order is placed and fulfilled

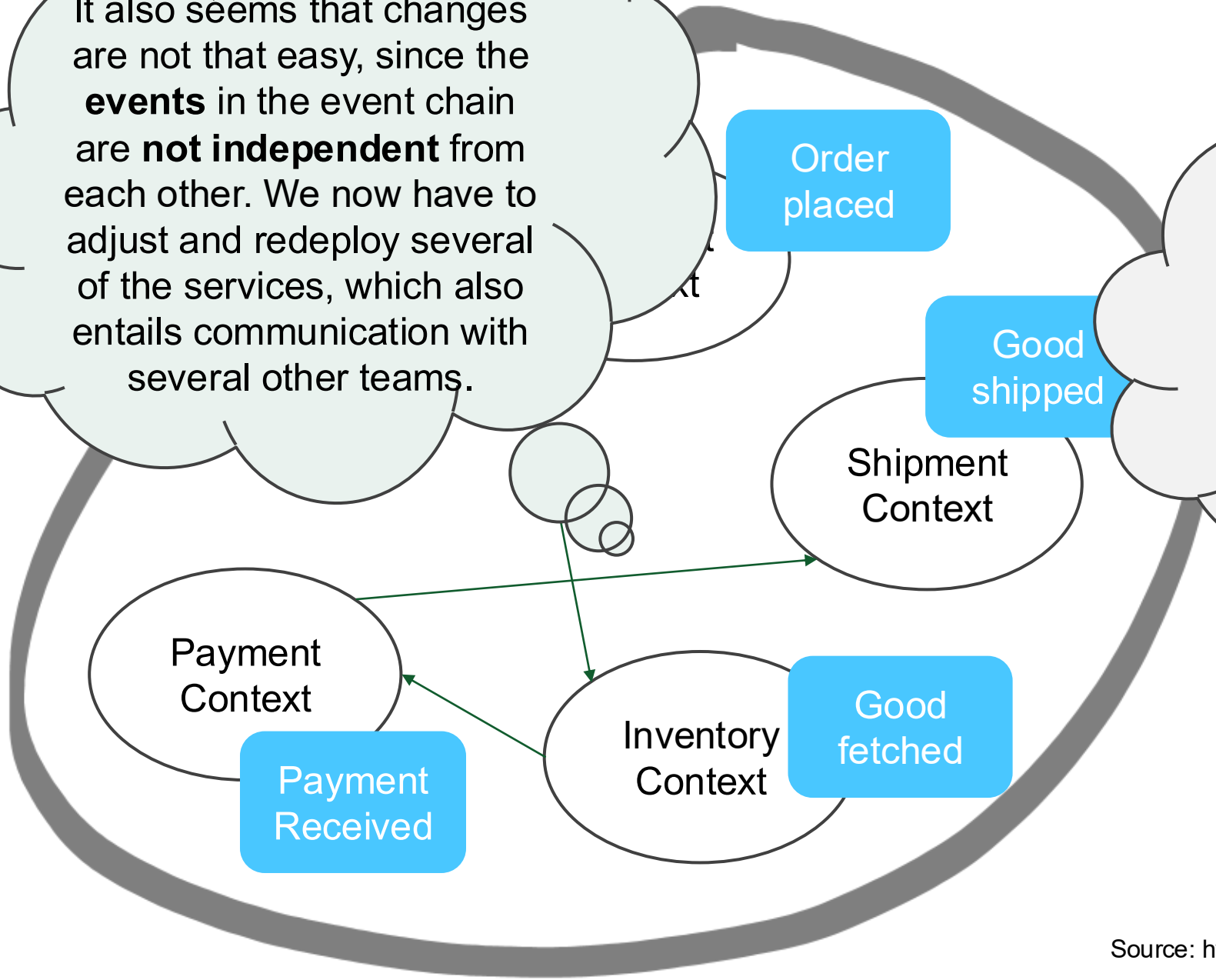


Source: https://www.infoq.com/articles/events-workflow-automation/

Peer-to-peer Event Chains
An Anti Pattern

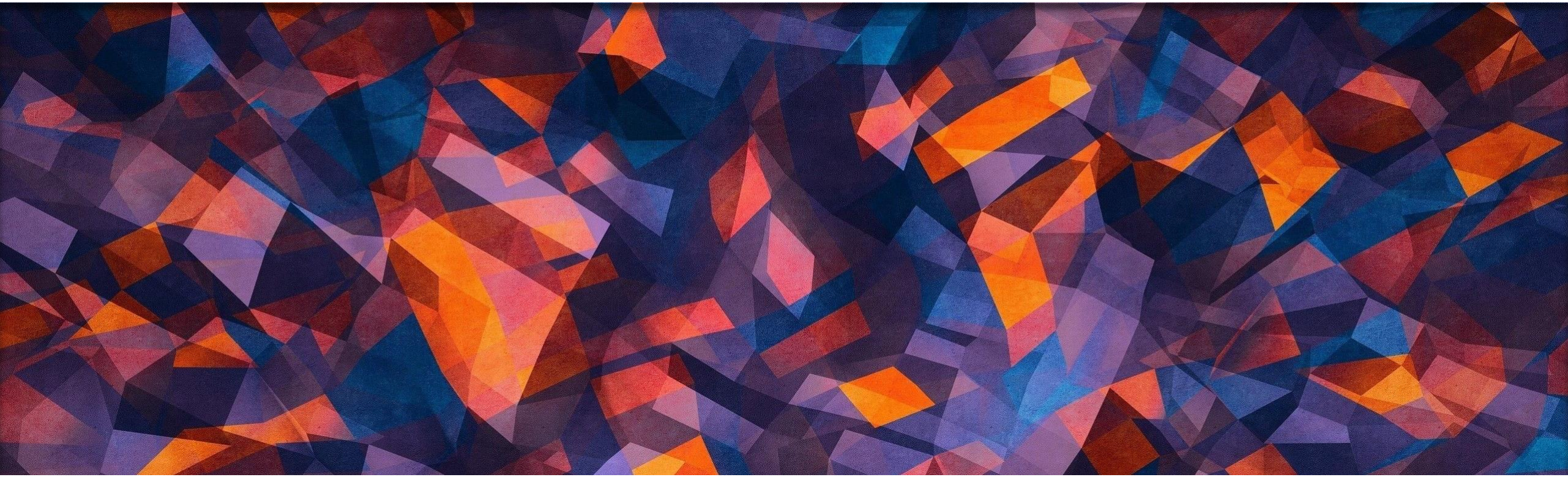
It also seems that changes are not that easy, since the **events** in the event chain are **not independent** from each other. We now have to adjust and redeploy several of the services, which also entails communication with several other teams.

Assume you want to implement a simple change request to fetch good before retrieving the payment .



Source: https://www.infoq.com/articles/events-workflow-automation/

Universität St.Gallen

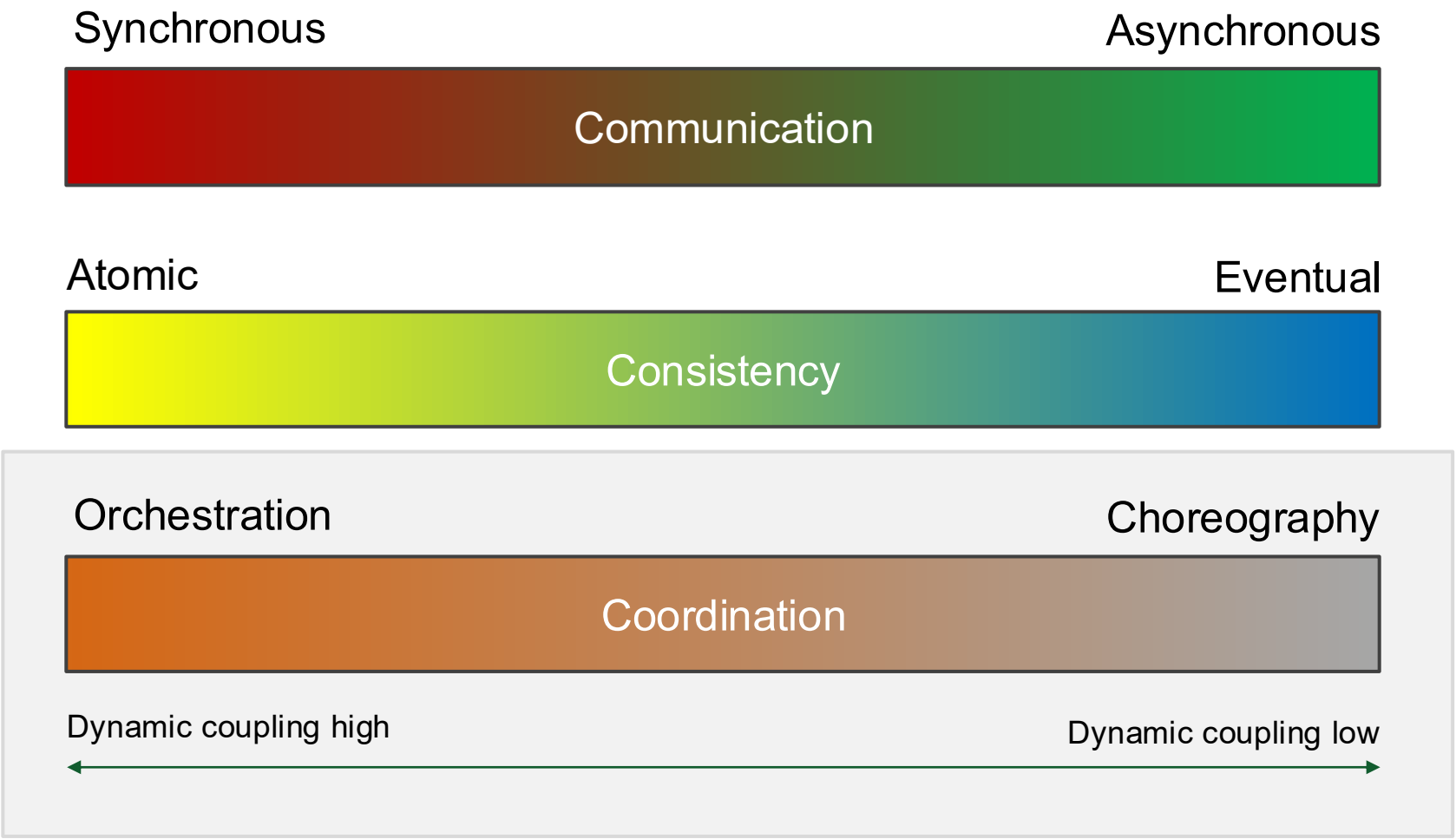


Event-driven and Process-oriented Architectures

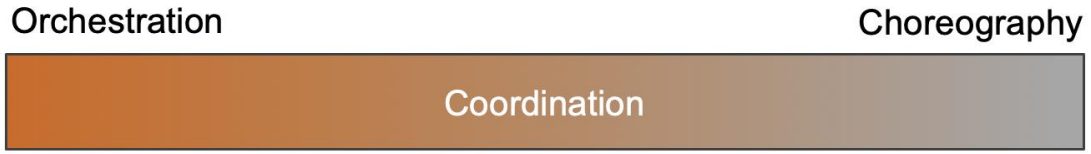
Coordination: Choreography

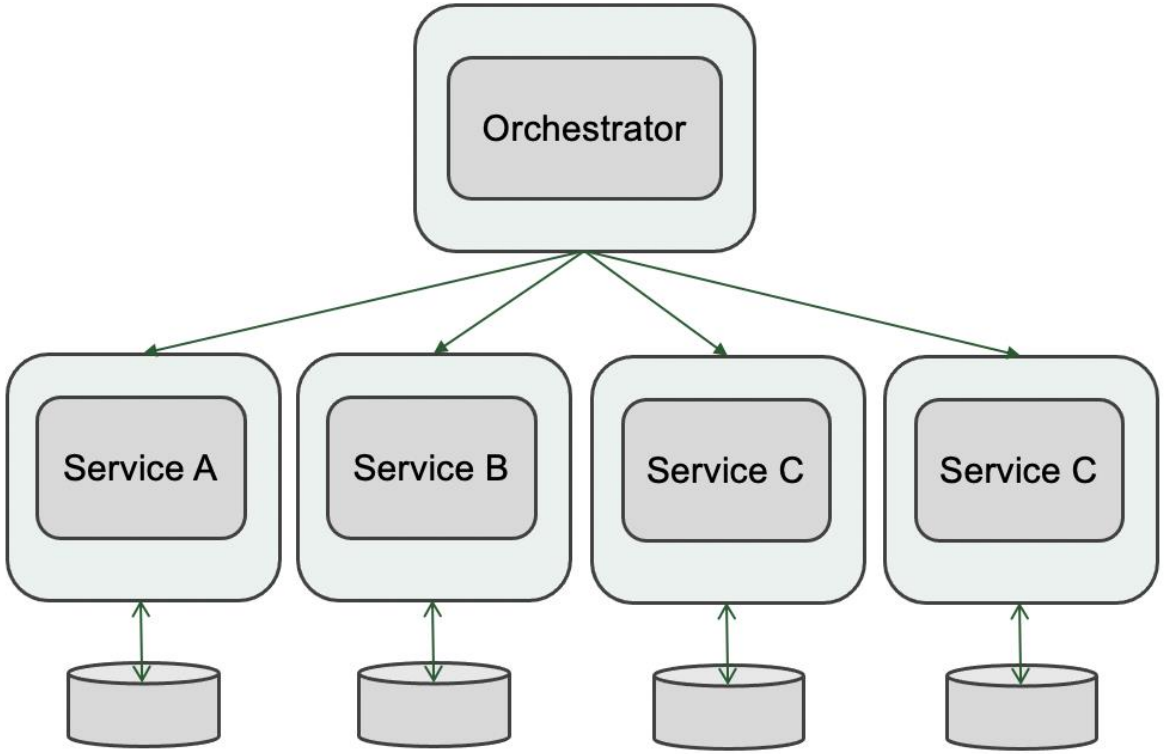
Lecture 2

Dynamic Quantum Coupling (Recap)
Three Coupling Forces

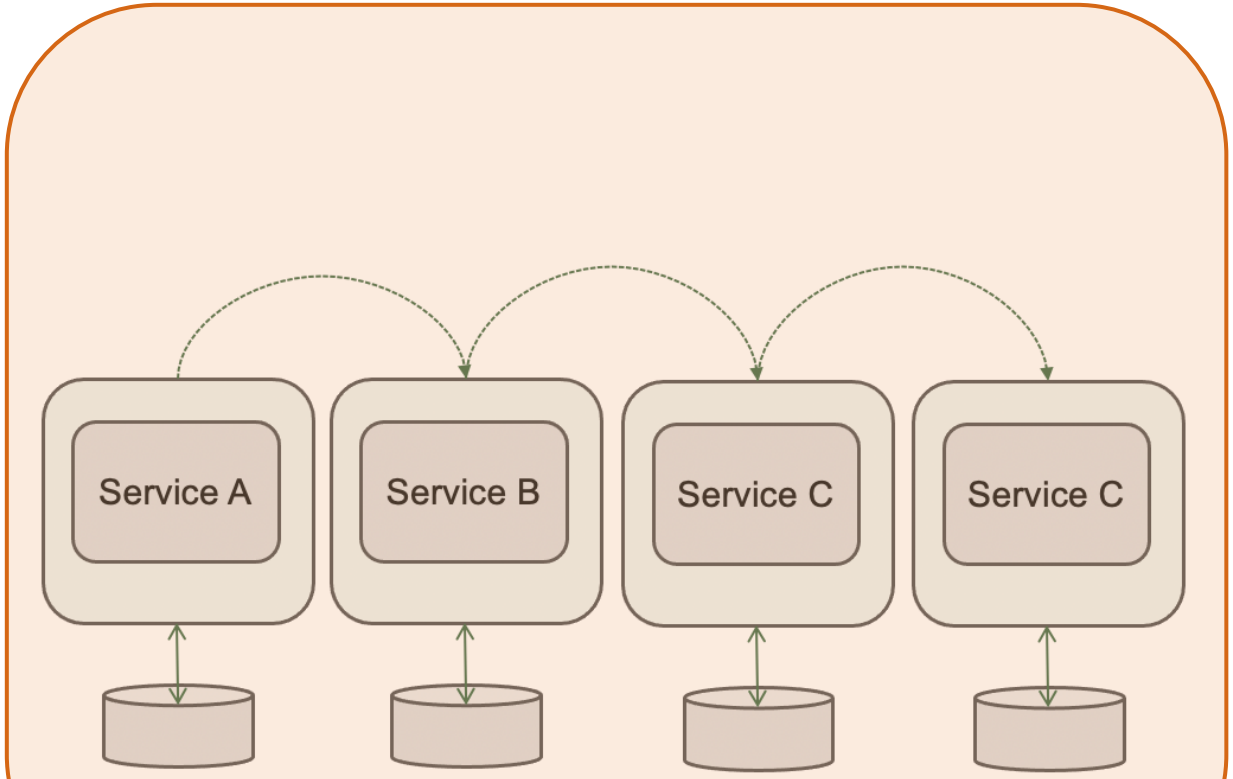


Orchestration versus Choreography
Two Fundamental Coordination Patterns





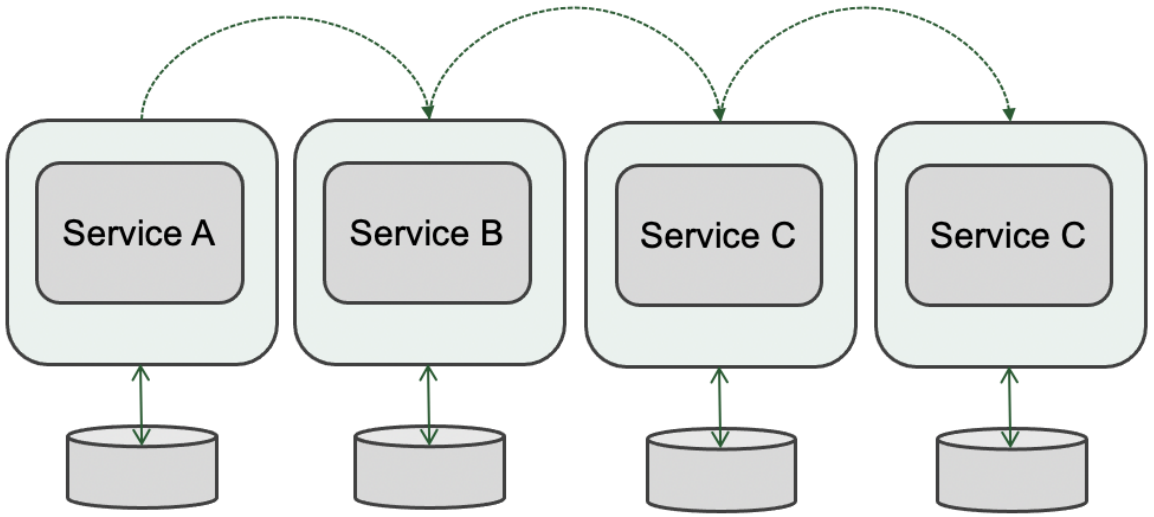
Orchestration



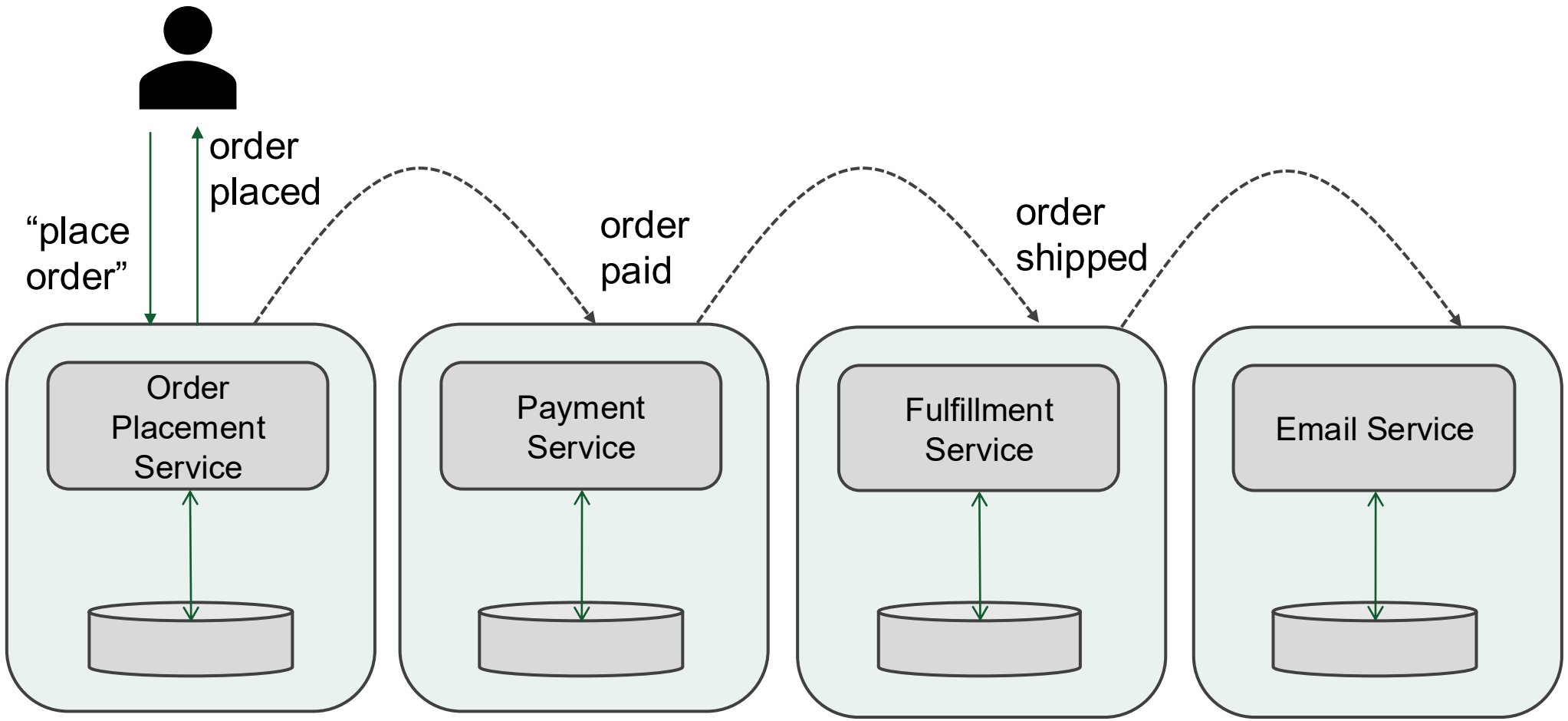
Choreography

Choreography Pattern

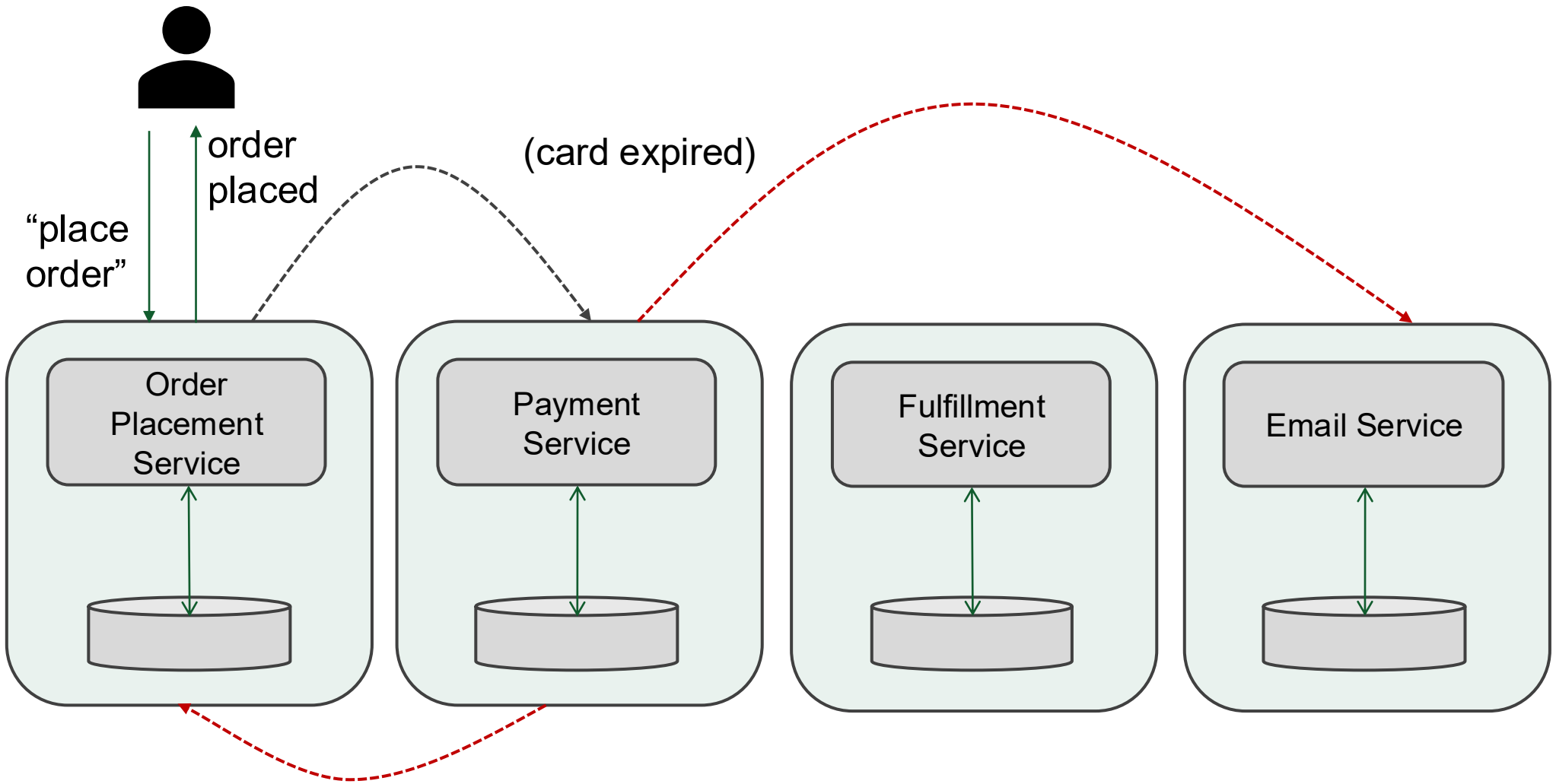
- Lacks central coordination
- Each service participates with each other in the choreography (just like in a dance)
- No single service knows the entire process



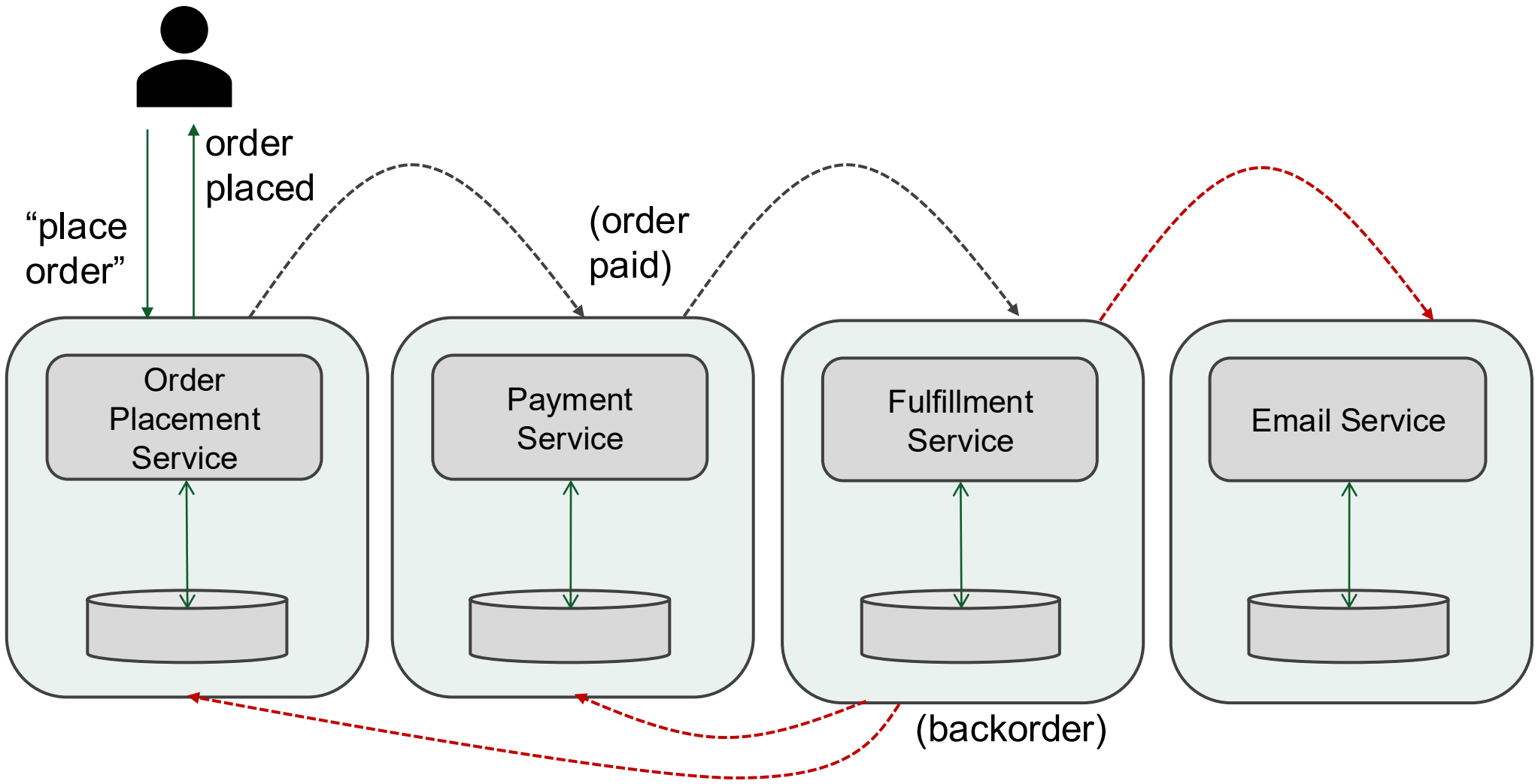
Happy Path Using a Choreography



Payment Rejection Error

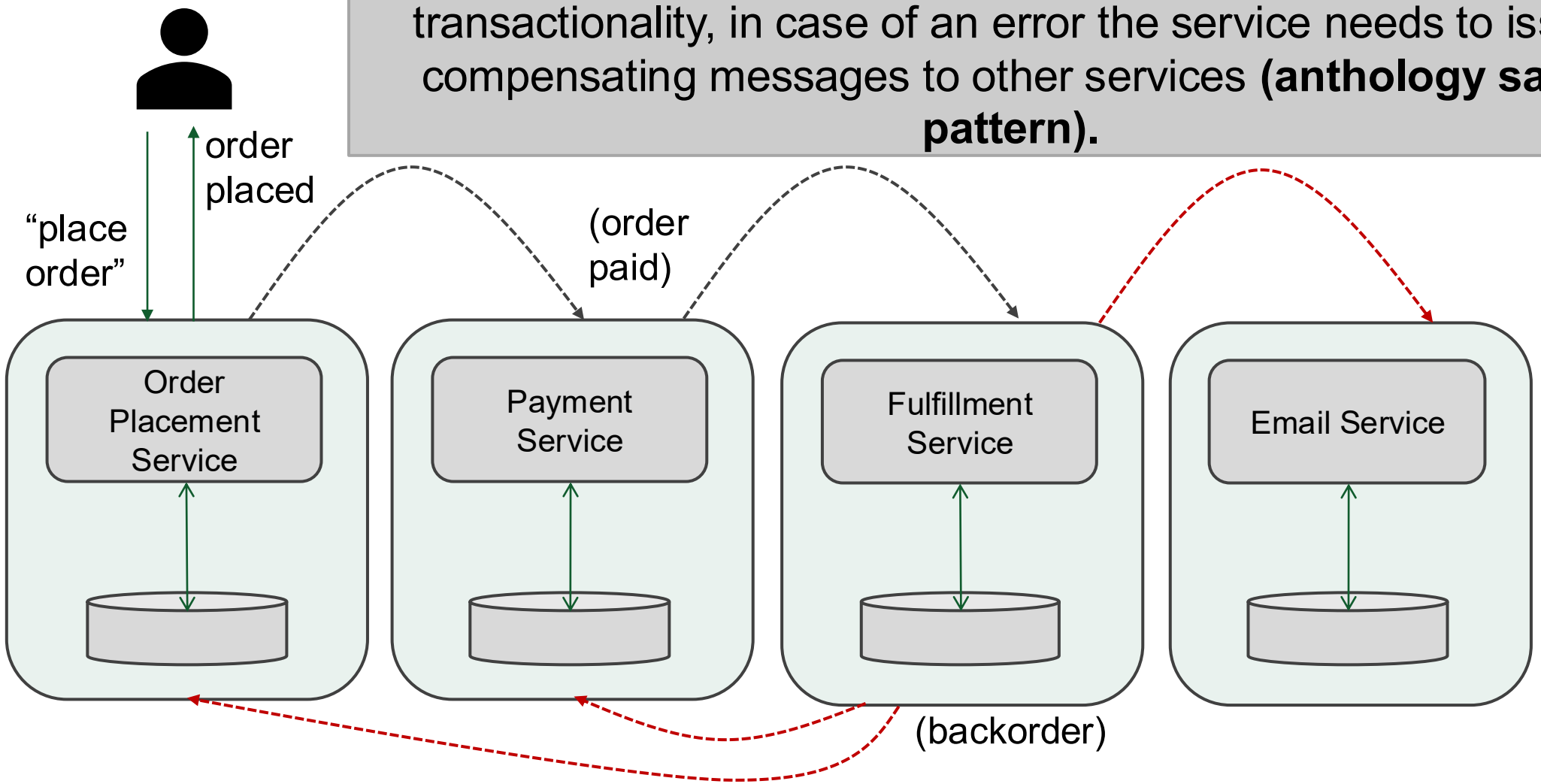


Handling Back Orders

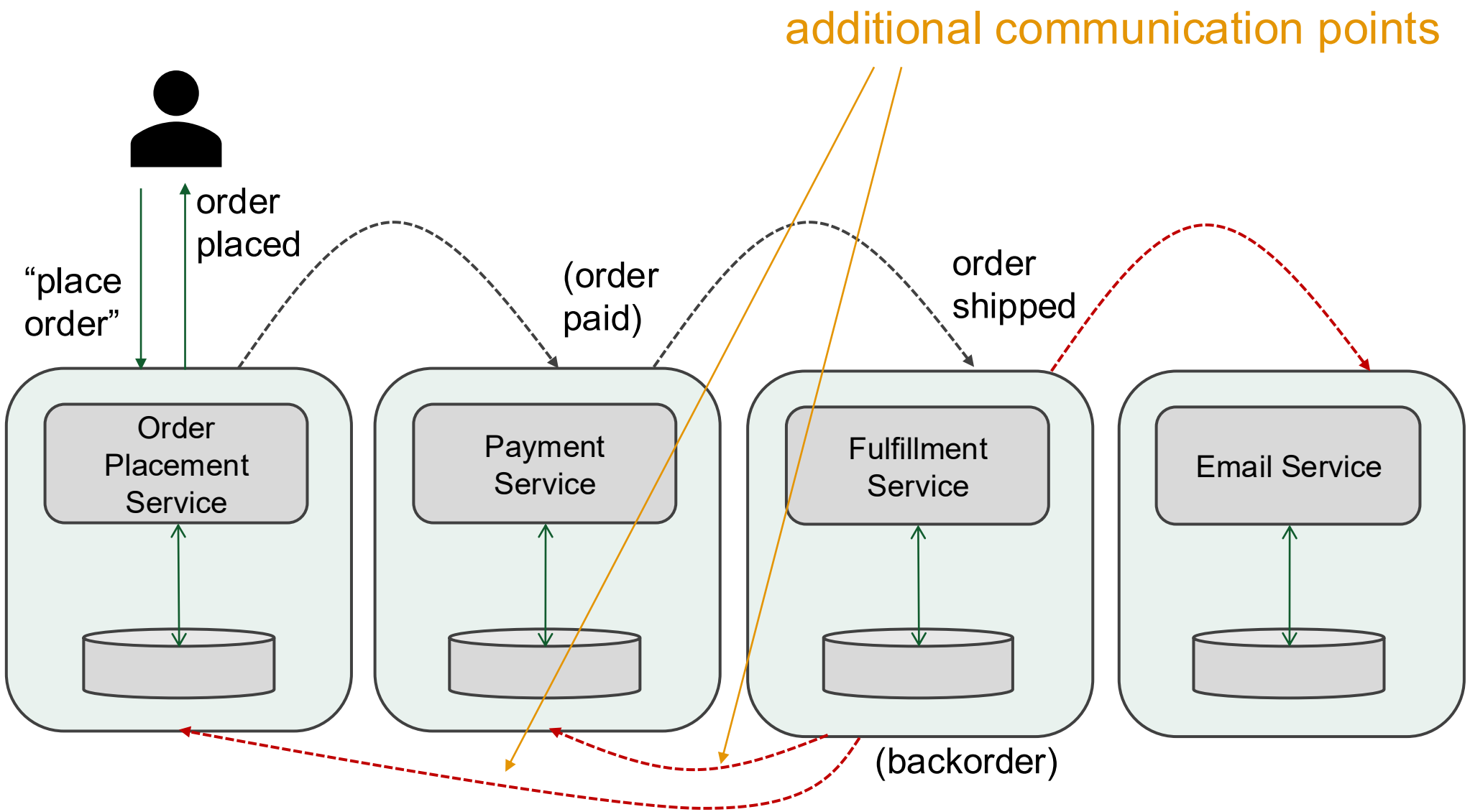


Handling Back Orders

Many steps have already completed before the backorder event causes the error. Since each service implements its own transactionality, in case of an error the service needs to issue compensating messages to other services **(anthology saga pattern).**



Error Handling Adds Communication Links



Semantic Coupling and Implementation Coupling

- Semantic Coupling
  - Inherent coupling that exists in the problem domain
  - When business invariants span services, semantic coupling is unavoidable — regardless of how loosely we implement communication.
- Implementation Coupling
  - Depends on how the interactions are modeled

> "Architects can never reduce semantic coupling via implementation, but they can make it worse".

Tip: Align coordination mechanism with the actual semantic boundaries of the workflow.

Distributed Workflows and the Risk of Event Chains

Event notification is nice because it implies a low level of coupling, and is pretty simple to set up. It can become problematic, however, if there really is a ***logical flow that runs over various event notifications****. The problem is that it can be hard to see such a flow as it's not explicit in any program text. Often the only way to figure out this flow is from monitoring a live system. This can make it hard to debug and modify such a flow. The danger is that it's very easy to make nicely decoupled systems with event notification, without realizing that you're* ***losing sight of that larger-scale flow****, and thus set yourself up for trouble in future years. The pattern is still very useful, but you have to be careful of the trap.*

Martin Fowler

Changing Event Chains is Costly

Scenario: Business requirement changes task sequence

- In choreography, control flow is implicitly distributed across services
- Changes cannot be made locally, but requires several services
- Workflow evolution becomes a distributed coordination problem
- This introduces cross-service versioning and rollout complexity

Decentralized Workflows and Lack of Visibility

- Event chains are hard to understand since the workflow is distributed across several microservices (i.e., multiple codebases)
- Diagnosing and fixing errors during operation is difficult due to lacking context in a choreography
- There is no single place to reason about correctness

Workflow State Management

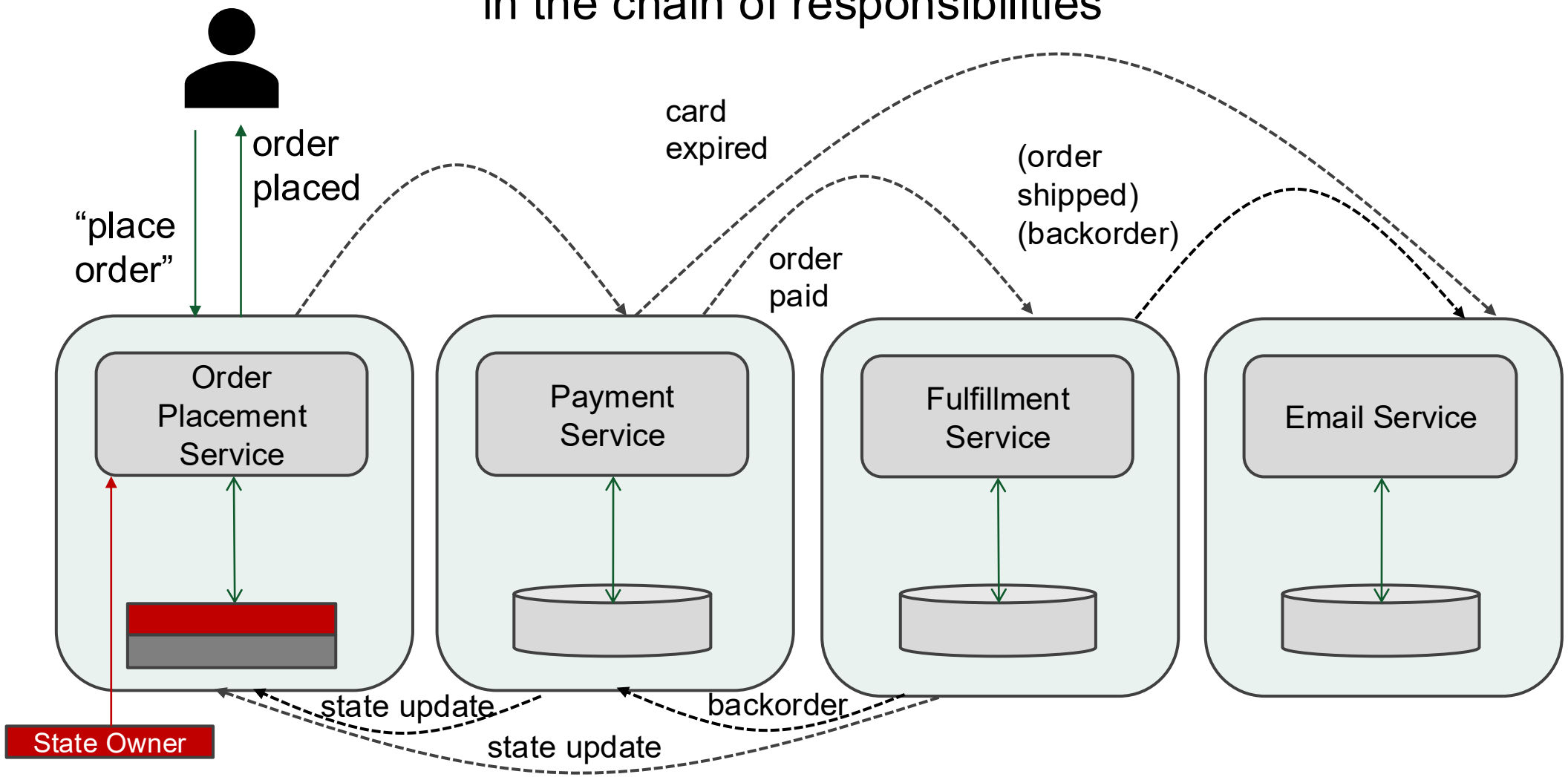
- Long-running workflows create cross-service state dependencies.
- Workflows require explicit state about progress (what has executed, what is pending, compensations, etc.)
- In choreography, no single service owns that workflow state

- Attempts to introduce workflow state
  - Front Controller Pattern
  - Stateless Choreography
  - Stamp Coupling

These workarounds reintroduce coordination implicitly.

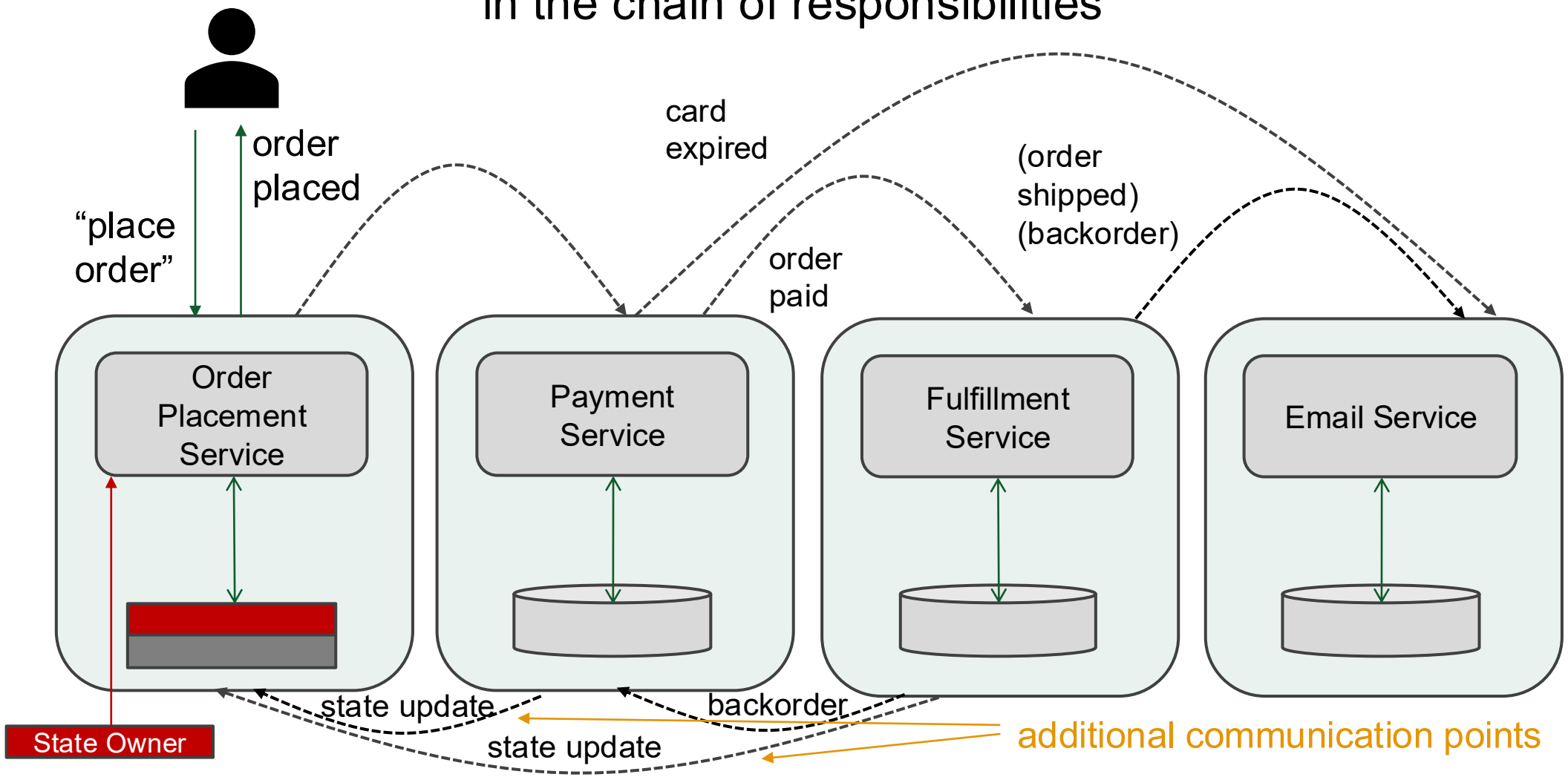
Front Controller Pattern

Responsibility for state is placed on the first called service in the chain of responsibilities



Front Controller Pattern

Responsibility for state is placed on the first called service in the chain of responsibilities



Trade-offs Front Controller Pattern

Pros	Cons
Creates a pseudo-orchestrator within choreography	Introduces workflow state into a domain service
Makes querying the state of an order trivial	Increases communication overhead
	Detrimental to performance and scale as it increases integration communication chatter

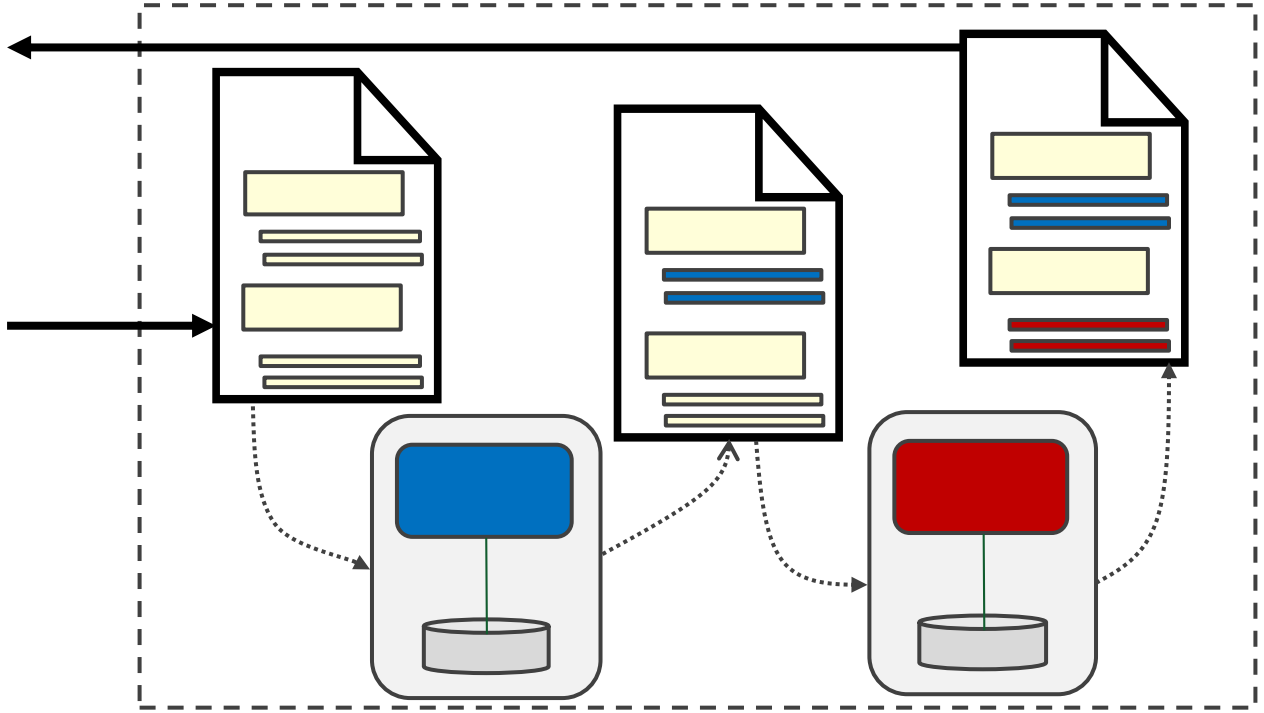
Stateless Choreography

- Keeps no transient state at all and relies on querying individual services to obtain a real-time snapshot

Pros	Cons
Offers high performance and scale	Workflow state must be reconstructed at runtime
Extremely decoupled	Complexity rises swiftly with complex workflows

Stamp Coupling

- Stores extra workflow state in message contracts sent between services
- Each service updates its part of the overall state and passes that to the next service in the chain of responsibilities
- Workflow semantics becomes integrated into the integration contract.



Transactional Boundary

Stamp Coupling

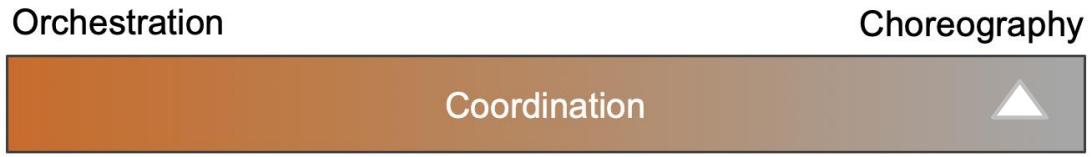
Source: https://www.youtube.com/watch?v=ExbDmyXfBnw

Stamp Coupling

- Stores extra workflow state in message contracts sent between services
- Each service updates its part of the overall state and passes that to the next service in the chain of responsibilities

Pros	Cons
Allows domain services to pass workflow state without additional queries to a state owner	Contracts must be larger to accommodate workflow state
Eliminates need for a front controller	Doesn't provide just-in-time status queries

Trade-offs Workflow-based Choreography



Pros	Cons
Responsiveness	Implicit control flow
Fault tolerance	Cross-service workflow state
Scalability	Complex compensations
Service decoupling	Difficult recovery reasoning

Sweetspot for Choreography

- Choreography is a good fit when:
  - Services are independent
  - Business invariants are local
  - Responsiveness and scalability are important
  - Complex error scenarios are rare

Works best when coordination is lightweight and global ownership is unnecessary.

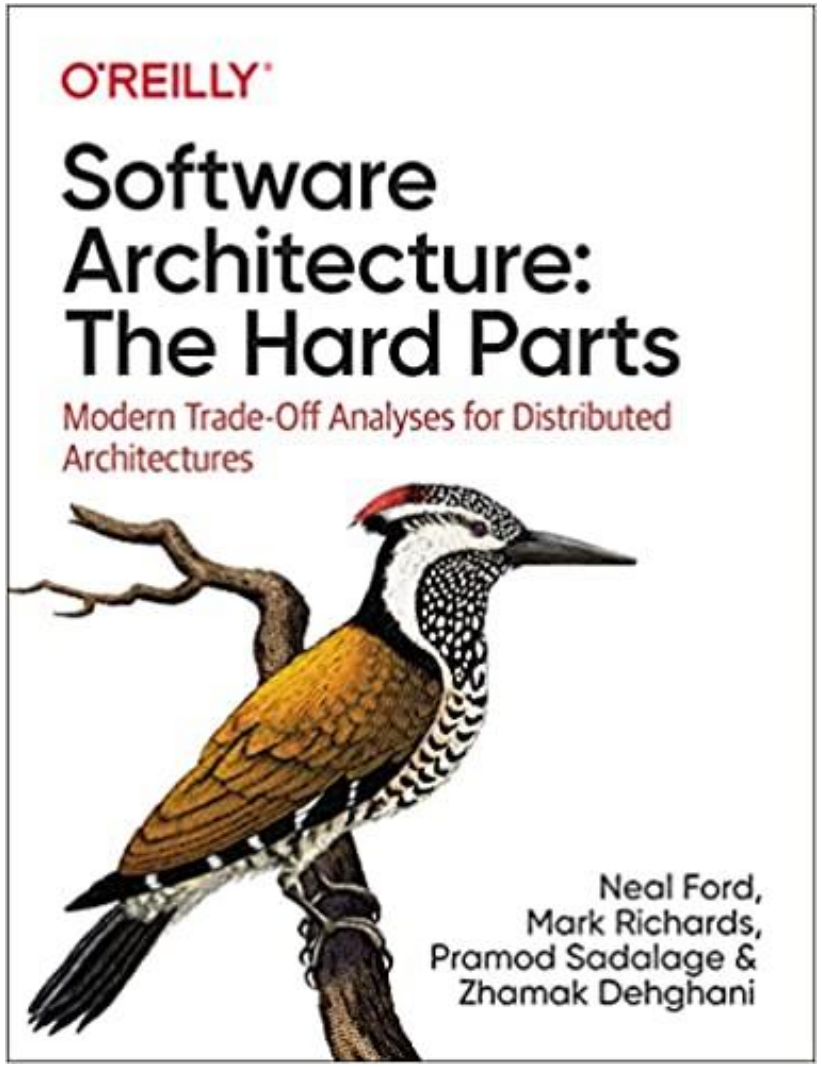
Universität St.Gallen

Choreography in the Dynamic Coupling Spectrum

Pattern Name	Communication	Consistency	Coupling
Phone Tag Saga	Synchronous	Atomic	High
Time Travel Saga	Synchronous	Eventual	Medium
Horror Saga	Asynchronous	Atomic	Medium
Anthology Saga	Asynchronous	Eventual	Very low

Choreography appears across different communication and consistency styles.

References and Further Material

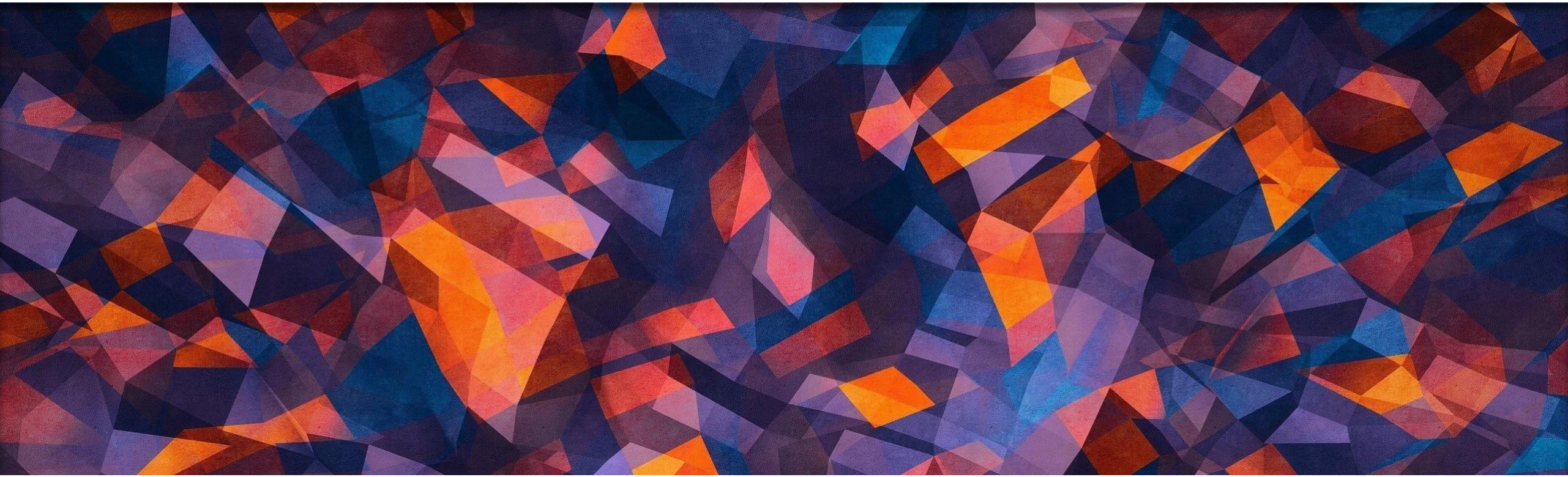


Based on Chapter 11 – Managing Distributed Workflows

More on Stamp Coupling:

Chapter 13 - Contracts

Universität St.Gallen



Event-driven and Process-oriented Architectures

Exercise

Lecture 2

Lab Exercises

- Lab 3 Part 1: Producer & Consumer using Kafka with Spring
  - https://github.com/scs-edpo/lab03Part1-kafka-spring
  - Spring Boot service with an HTTP API
  - Publisher of messages via Kafka, either as String or as JSON object
  - Consumer of corresponding messages
  - Your task: extend, modify, split up consumers and producers in sep. services

- Lab 3 Part 2: Kafka Spring with Eye Tracking Use Case
  - https://github.com/scs-edpo/lab03Part2-kafka-spring-Eye-tracking

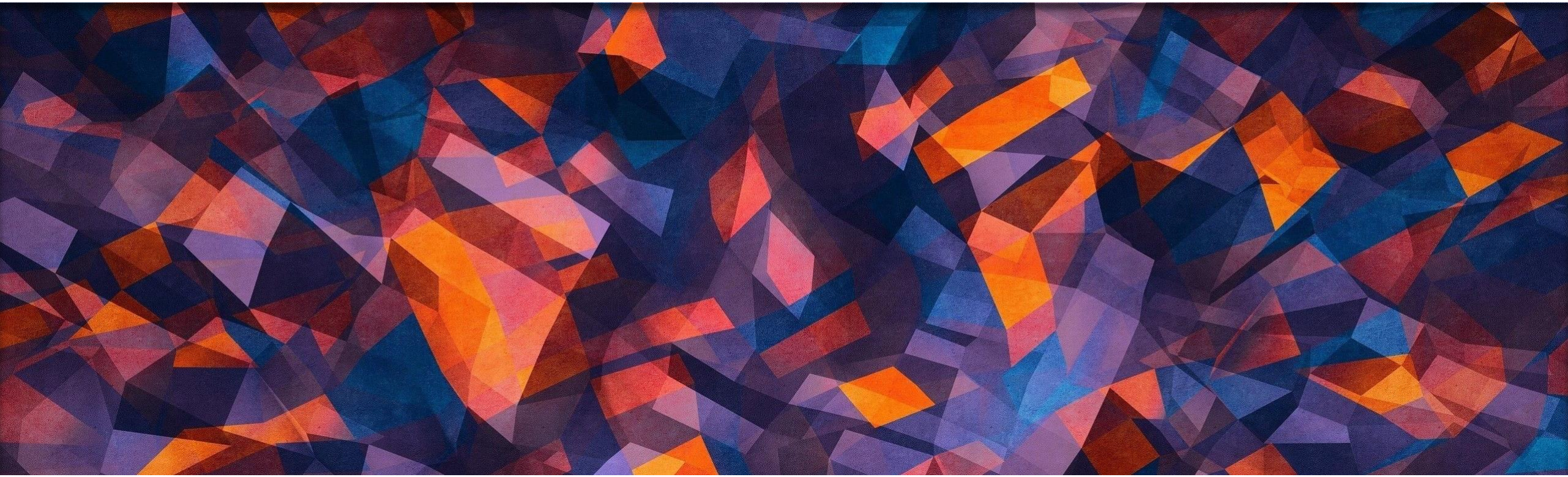
Lab Exercises: Flowing Retail

Lab 4: Discussing Architectural Decisions

- https://github.com/scs-edpo/lab04-flowing-retail/tree/master/kafka/java/checkout
- https://github.com/scs-edpo/lab04-flowing-retail/tree/master/kafka/java/choreography-alternative

- Discuss: Architectural style, Service granularity, Communication/coordination style (synchronous versus asynchronous), Message flow, additional error scenarios and their implications

Universität St.Gallen



Event-driven and Process-oriented Architectures

Technology: CNCF Cloud Events

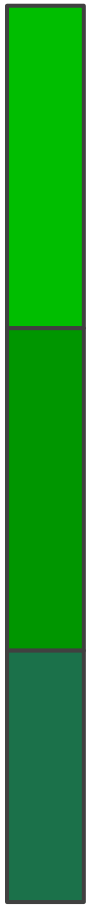
Lab 4

CNCF Cloud Events

- A specification for describing event data in common format to provide **interoperability across services, platforms and systems**
- Open standard developed by the CNCF Serverless Working Group
- Cloud Events specification: https://github.com/cloudevents/spec

Cloud Events – Example Event

```
{
    "id" : "A234-1234-1234",
    "source" : "https://github.com/cloudevents/spec/pull",
    "specversion" : "1.0",
    "type" : "com.github.pull_request.opened",
    "subject" : "123",
    "time" : "2018-04-05T17:31:00Z",
    "datacontenttype" : "text/xml",
    "data" : "<much wow=\"xml\"/>"
    "comexampleextension1" : "value",
    "comexampleothervalue" : 5
}
```



Universität St.Gallen

Flowing Retail Choreography

Starting the Java-Choreography configuration

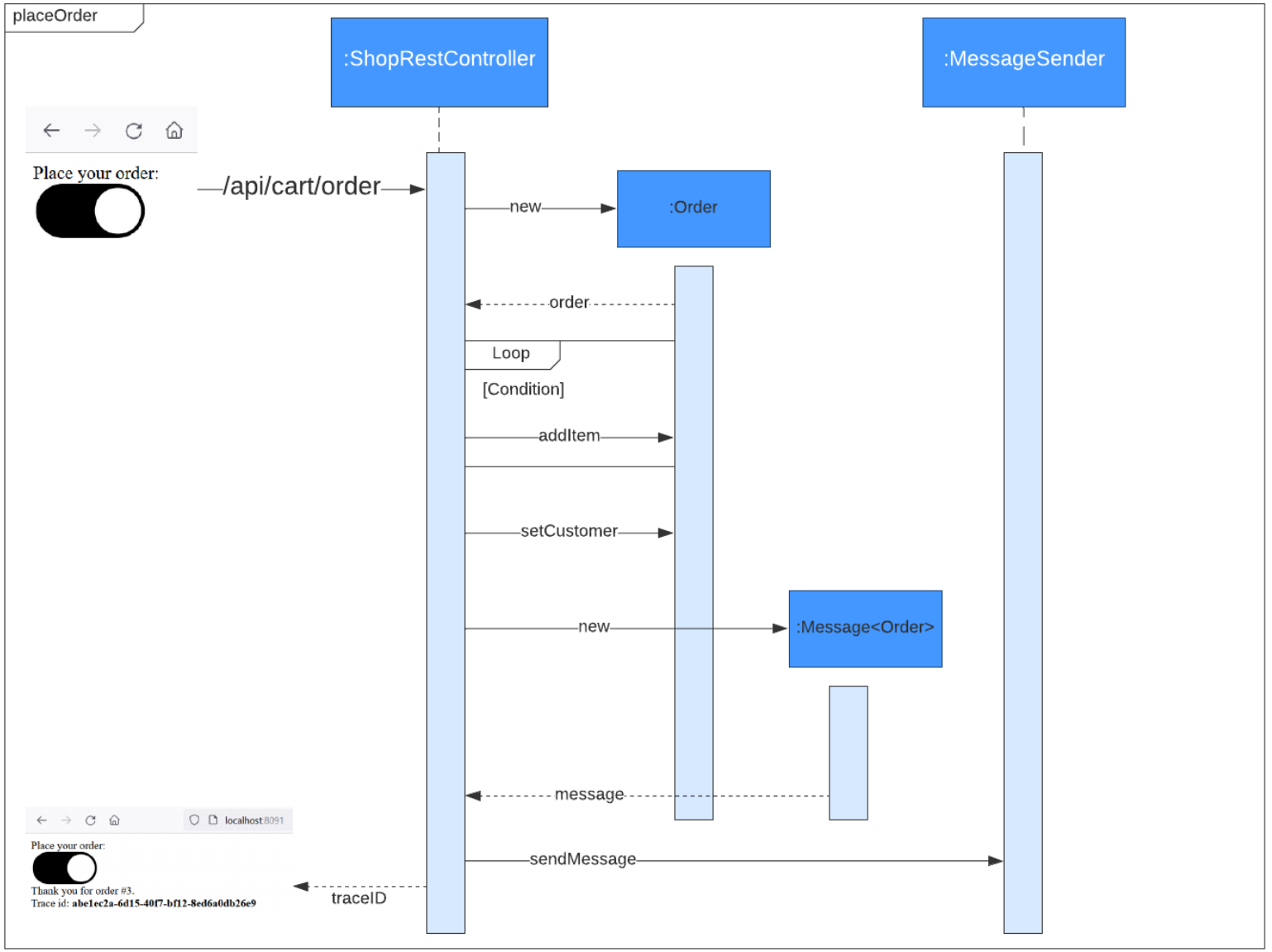
Flowing retail event monitor

Connect websocket Disconnect websocket

Trace Id	Event flow
c2c8017c-0ef7-4788-95ab-9af7c486808f	Event: OrderPlacedEvent (from Checkout)
	Event: PaymentReceivedEvent (from Payment-Choreography)
	Event: GoodsFetchedEvent (from Inventory-Choreography)
	Event: GoodsShippedEvent (from Shipping-Choreography)
	Event: OrderCompletedEvent (from Shipping-Choreography)

Checkout Service

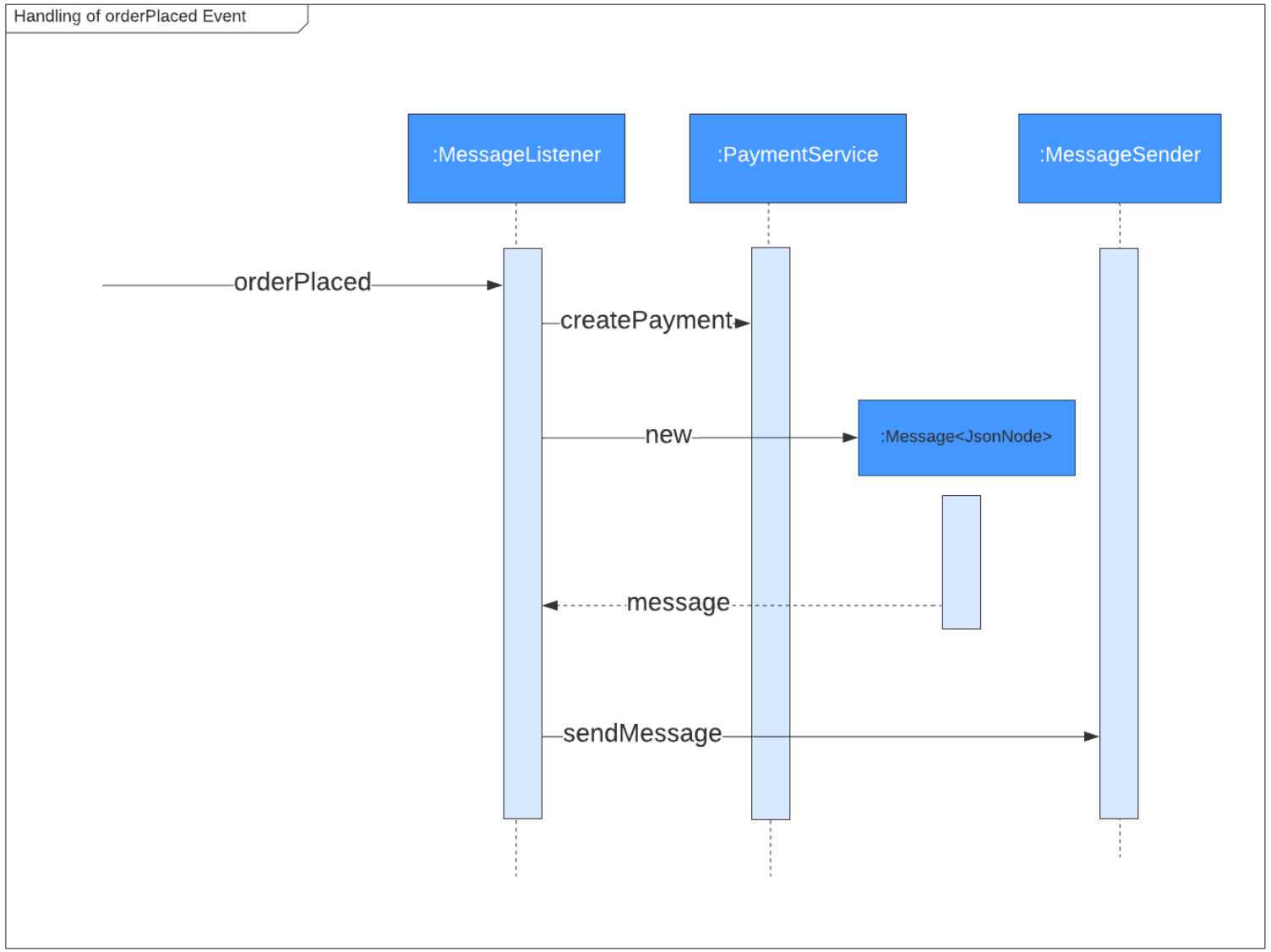
- **Message Sender** uses Spring with Kafka
- Programming model used is annotation-based
- **Message** follows Cloud Events spec



Payment Service

Handling of Order Placed Event and Payment Received Event

- **Message Listener/Sender** use Spring with Kafka
- **Message** follows Cloud Events spec

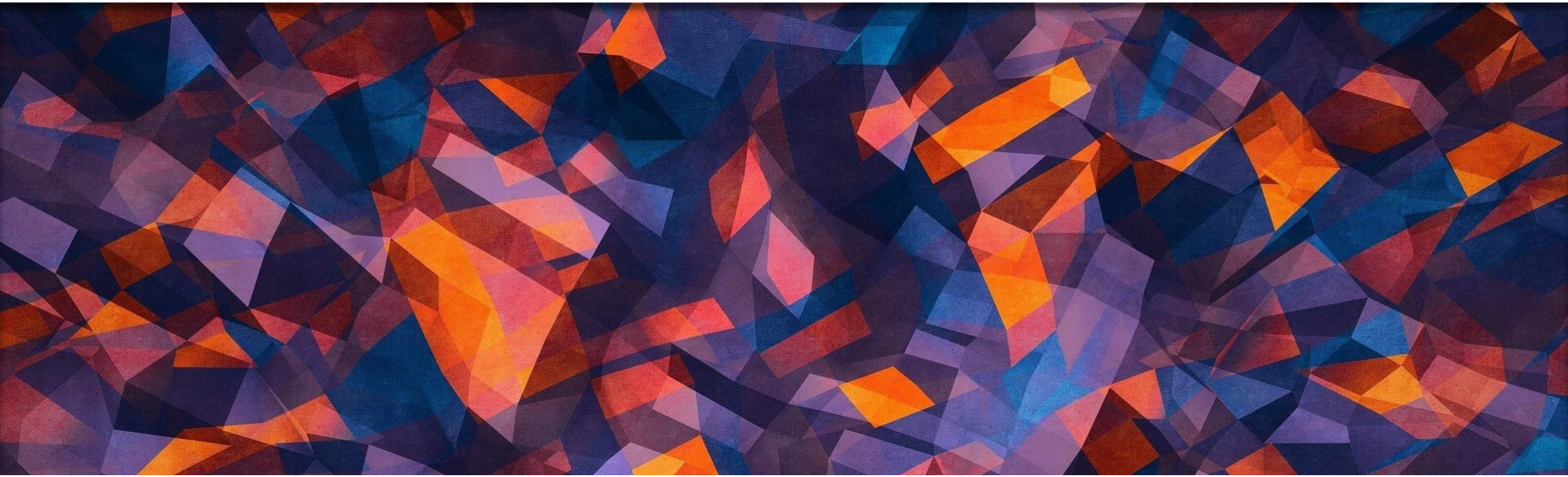


Further References

- Event-Driven Architecture: Getting Started with Kafka
  - https://blog.ippon.tech/event-driven-architecture-getting-started-with-kafka-part-1/
  - https://blog.ippon.tech/event-driven-architecture-getting-started-with-kafka-part-2/

  - https://www.confluent.io/kafka-summit-san-francisco-2019/building-event-driven-architectures-with-kafka-and-cloud-events/

Universität St.Gallen



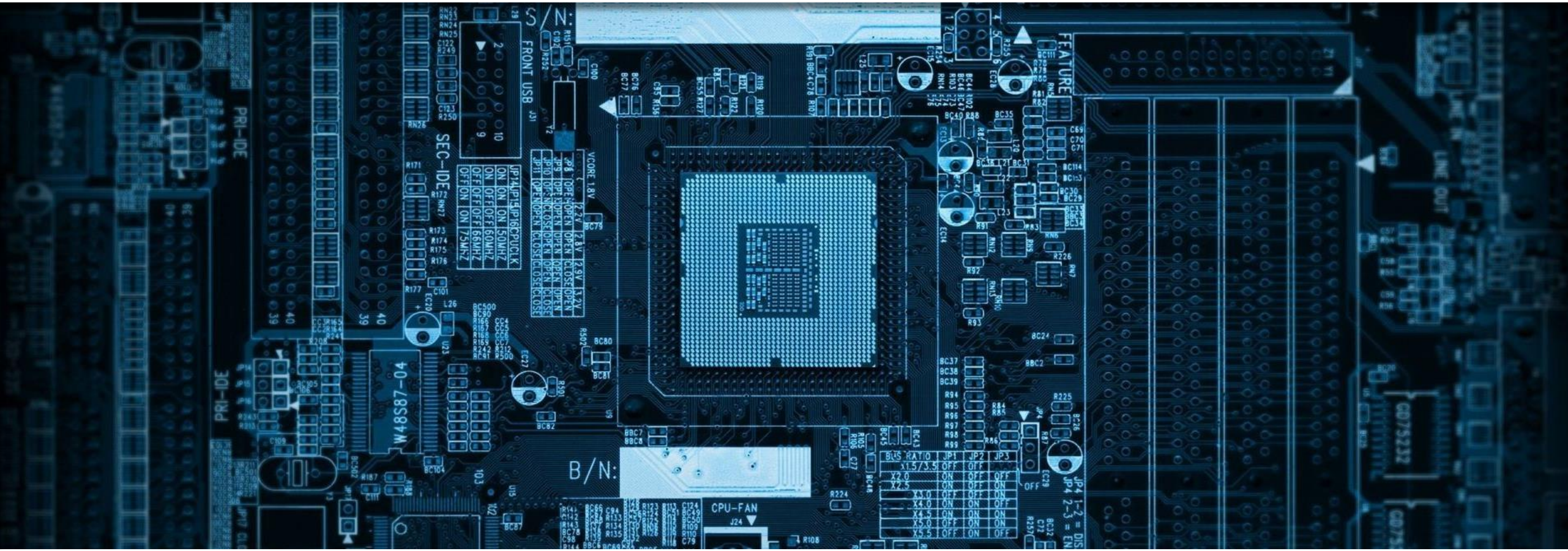
Event-driven and Process-oriented Architectures

Ideas for Project

Lecture 2

Suggestions for Project

- Having seen how to create producer and consumer using Spring with Kafka you could **extend the flowing retail** example from Lab 4 or **think about your own small project** with the goal to implement at least one of the EDA patterns
  - Event notification: notification service for the flowing retail example which sends emails to inform customers about relevant progress in the process
  - Event-carried state transfer: extend the checkout service such that the checkout service is aware about the available stock
    - Requires to keep a copy of the data at the checkout
    - Keep data updated (e.g., using one of the eventually consistency pattern like event-based synchronization)
  - Extension of flow retail example with error scenarios



Questions?